Helix Constitution — Constitution

Helix Universal Constitution

Table of contents

§11.4.19 — Fixed-document column-alignment mandate (User mandate, 2026-05-14)
§11.4.21 — Operator-blocked status + self-resolution exhaustion mandate (User mandate, 2026-05-14)
§11.4.22 — Document-sync commit discipline (User mandate, 2026-05-14)
§11.4.23 — Visual-cue & grouping mandate for Issues docs (User mandate, 2026-05-14)
§11.4.24 — Build-resource stats tracking mandate (User mandate, 2026-05-14)
§11.4.25 — Full-Automation-Coverage Mandate (User mandate, 2026-05-15)
§11.4.26 — Constitution-Submodule Update Workflow Mandate (User mandate, 2026-05-15)
§11.4.31 — Submodule-Dependency-Manifest Mandate (User mandate, 2026-05-15)
§11.4.32 — Post-Constitution-Pull Validation Mandate (User mandate, 2026-05-15)
§11.4.30 — .gitignore + No-Versioned-Build-Artifacts Mandate (User mandate, 2026-05-15)
§11.4.29 — Lowercase-Snake_Case-Naming Mandate (User mandate, 2026-05-15)
§11.4.28 — Submodules-As-Equal-Codebase + Decoupling + Dependency-Layout Mandate (User mandate, 2026-05-15)
§11.4.27 — No-Fakes-Beyond-Unit-Tests + 100%-Test-Type-Coverage Mandate (User mandate, 2026-05-15)
§11.4.33 — Type-aware closure-status vocabulary (User mandate, 2026-05-15)
§11.4.34 — Reopened-source attribution mandate (User mandate, 2026-05-15)
§11.4.35 — Canonical-root inheritance clarity (User mandate, 2026-05-15)
§11.4.36 — Mandatory install_upstreams on clone/add Mandate (User mandate, 2026-05-15)
§11.4.37 — Fetch-before-edit mandate (User mandate, 2026-05-15)
§11.4.38 — Installable-Asset Evidence Mandate (User mandate, 2026-05-17)
§11.4.39 — Per-Feature On-Device End-User Validation Mandate (iter-76, 2026-05-17)
§11.4.40 — Full-suite retest before release tag mandate (User mandate, 2026-05-17)
§11.4.41 — Pre-Force-Push Merge-First Mandate (User mandate, 2026-05-17)
§11.4.43 — TDD-Fix-Discipline Mandate (User mandate, 2026-05-18)
§11.4.44 — Document Revision Header Mandate (User mandate, 2026-05-18)
§11.4.45 — Integration-Status-Doc Maintenance Mandate (User mandate, 2026-05-18)
§11.4.46 — Validate-recent-work-before-post-flash-tests mandate (User mandate, 2026-05-18)
§11.4.47 — Firebase Data Review Mandate (User mandate, 2026-05-18)
§11.4.48 — UI-Driven Video Testing Mandate (User mandate, 2026-05-18)
§11.4.49 — Dual-Approach Testing Mandate (User mandate, 2026-05-18)
§11.4.50 — Deterministic Consistency Mandate (User mandate, 2026-05-18)
§11.4.51 — Live-ADB-First Maximization Mandate (User mandate, 2026-05-18)
§11.4.52 — Autonomous-Validation Mandate (User mandate, 2026-05-18)
§11.4.53 — Fixed_Summary parity mandate (User mandate, 2026-05-18)

Helix Universal Constitution

Field	Value
Revision	4
Created	2026-05-14
Last modified	2026-05-20
Status	active
Status summary	New mandate landed: §11.4.76 Containers-submodule mandate (renumbered from drafted §11.4.75 after concurrent `0a70083` landed §11.4.75 Mechanical Enforcement). For ANY containerized workload (Docker / Podman / Qemu / Kubernetes / emulators), consuming projects MUST install `vasic-digital/containers` as a Git submodule + boot infra on-demand via its `pkg/boot`+`pkg/compose`+`pkg/health` APIs + extend (never reimplement) when features are missing. QWEN.md created in this commit per the user-stated `Constitution.md / AGENTS.md / CLAUDE.md / QWEN.md` propagation set. CLAUDE.md + AGENTS.md mirrors updated in lockstep.
Issues	none
Issues summary	ToC is currently stale (missing entries for §11.4.71/.72/.73/.74/.75/.76) — separate clean-up commit needed per §11.4.61.
Fixed	§11.4.73, §11.4.74, §11.4.76; QWEN.md created
Fixed summary	Operator-mandated additions 2026-05-20 propagated into Constitution.md (this commit); CLAUDE.md + AGENTS.md + new QWEN.md mirrors land in the same commit. §11.4.76 renumber-from-§11.4.75 documented inline (collision with concurrent Mechanical Enforcement landing).
Continuation	(1) ToC regeneration commit (cover §11.4.71..§11.4.76). (2) Submodule-catalogue inventory expansion in

Field	Value
	CLAUDE.md / AGENTS.md / QWEN.md per the 2026-05-20 follow-up user mandate — per-Submodule purpose lines for every repo in `vasic-digital` + `HelixDevelopment`.

Table of contents

> **This document defines mandatory, non-negotiable rules for every project that includes this constitution submodule. Every AI agent and every human contributor MUST comply. Violations are blockers.**
>
> Project-specific constitutions extend this document. When a project Constitution (e.g. `docs/guides/PROJECT_CONSTITUTION.md`) and this universal Constitution differ, the project Constitution may extend or tighten — but NEVER weaken — the universal rules below.
>
> **Last revision**: 2026-05-14 (v1.0 — initial extraction from ATMOSphere Android 15 1.1.5-dev)

This Constitution is the universal source-of-truth for the engineering practices shared across all Helix projects (and any project that opts in by adding this submodule). It is intentionally agnostic about specific hardware, operating systems, languages, vendors, and product domains. Anything in this document that mentions a specific product, hardware SKU, library version, or vendor MUST be moved to the consuming project's own Constitution.

---

1. Test coverage is mandatory for every change

Every code, configuration, documentation, or infrastructure change MUST ship with tests that prove:

Invariant	Mechanism
The change is present in source	pre-build / pre-merge gate that scans the source tree for the expected pattern
The change survives compilation / packaging	post-build gate that inspects the produced artifact (binary, archive, image) and verifies the change actually shipped
The change behaves correctly at runtime	runtime / integration / on-device test that exercises the user-visible behaviour
The gate itself is not bluffing	meta-test (mutation) entry that mutates the asserted condition and proves the gate catches the break

A change without all four layers of coverage is not ready to merge.

1.1 False-positive immunity is an invariant

A test that always returns PASS because its regex never matches, its input path is wrong, its assertion target does not exist, or its comparison is tautological is **worse than no test**. Every new gate MUST be paired with a mutation entry in the project's meta-test harness that:

1. Temporarily breaks the assertion (sed out a line, rename a symbol, etc.).
2. Re-runs the gate.
3. Asserts the gate now reports FAIL.
4. Restores the original.

If the mutation round does not turn PASS → FAIL, the gate is a sham and must be rewritten.

This is the **single most important rule in this Constitution.** Every subsequent §11.4.x clause is downstream of it.

---

2. Commit and push mechanics — single entrypoint, locked

All commit and push work uses the project's official multi-remote commit wrapper (e.g. `scripts/commit_all.sh`) as the single entrypoint. Direct `git commit` / `git push` / `git add` on the main repo is prohibited in normal workflow. Exception: tag creation + tag-ref push is done via explicit `git tag -a` and `git push <remote> <tag>`.

The commit and push wrappers MUST hold an advisory `flock` so two invocations cannot race against each other. Lock files self-clean on process exit via `trap`.

When a contributor sees the "another commit/push wrapper is already running" error, they MUST NOT `rm -f` the lock unless they have just killed the owning process.

2.1 Multi-upstream push is the norm

Every project hosted on multiple Git providers (GitHub primary, GitLab / GitFlic / GitVerse / Bitbucket / Gitea mirrors) MUST push every commit to ALL configured upstream remotes. A commit that lives on only one remote is a future operational risk: when that one provider has an outage, the project is unreachable.

The constitution submodule itself ships an `install_upstreams.sh` helper (and an `Upstreams/` directory with one declaration script per remote). Running it from the submodule root configures all the remotes locally. Consuming projects SHOULD do the same for their own multi- remote topology.

---

3. Submodule changes propagate through submodule commits first

When a change lands in any submodule of the project:

1. **Commit inside the submodule first** (the submodule's own `git add` + `git commit`), since the parent project's commit wrapper does not commit submodule source — it only captures the updated submodule pointer.
2. **Push the submodule commit** to **all** remotes of that submodule.
3. **Then** run the parent project's commit wrapper to capture the updated submodule pointer in main and push main.

Skipping step 1 produces parent commits / tags that point at old submodule HEADs without the actual source.

---

4. Every tag on the main repo MUST be mirrored on every owned submodule

When a version tag is created on the main repo at a specific commit, the same tag MUST be created on **every owned submodule** at that submodule's currently-pointed-to HEAD, and pushed to **every remote** of every owned repo. Third-party submodules (libraries not under the project's control) are excluded. The consuming project's Constitution declares its owned-submodule set explicitly.

---

5. Changelog discipline and multi-format export

Every tagged release MUST ship:

1. A new `docs/changelogs/<tag>.md` entry describing user-visible changes, developer-visible changes, and known caveats.
2. Exports of that entry to `docs/changelogs/<tag>.{html,json,txt}` so external tooling (release portals, ticketing systems, package indices) can consume without needing a Markdown parser.
3. An update to a cumulative `docs/changelogs/CHANGELOG.md` (reverse chronological).

The project SHOULD provide a script (`scripts/testing/export_changelog.sh <tag>` or equivalent) that wraps `pandoc` + `jq` + plain-text Markdown strip.

---

6. Documentation up to the nano-details

Every new feature, fix, or infrastructure change MUST update:

1. The project's CLAUDE.md / AGENTS.md Applied Fixes table (one row, one commit, one release tag).
2. `docs/guides/` for any user-visible or developer-reachable behaviour change.
3. Architecture diagrams, flowcharts, and any reference docs that touch the changed subsystem.
4. The per-version changelog (§5).

Documentation drift after a fix is itself a Constitution violation: undocumented behaviour change is the same defect class as un-tested behaviour change.

---

7. Making false-success results literally impossible

All validation output MUST satisfy the following invariants:

1. Every gate reports concrete PASS / FAIL / SKIP with explicit reason text. A gate that just says "ok" is non-compliant.
2. Every SKIP reason MUST be mechanically distinguishable from a PASS. Summary counters track them separately.
3. FAILs are counted towards exit status. A script that hides FAILs in logs and returns 0 is a bug.
4. Meta-tests (§1.1) catch any gate that always PASSes regardless of condition.
5. Runtime results MUST be cross-referenced against host-side evidence (capture rig, screen recording, log analyzer, or equivalent) when the system's own introspection cannot directly verify the behaviour.

§7.1 NO BLUFF — positive-evidence-only validation

Every runtime test MUST satisfy ALL FIVE of the following non-negotiable constraints. A test that violates any one of them is a **bluff** and is forbidden from being committed to the test suite:

1. **Real ACTION**: the test MUST invoke at least one user-visible action via a project anti-bluff helper (e.g. `ab_send_action()` in shell-based suites). A test that records a PASS without ever calling such a helper is automatically failed by the summary function.
2. **State DELTA**: every PASS that claims to verify a feature MUST capture state BEFORE the action and AFTER, and assert the delta matches expectation. `before == after` on success is a bluff.
3. **POSITIVE EVIDENCE**: the PASS condition MUST be POSITIVE evidence (a value match, a state delta, a captured frame analysis, a captured audio frame analysis), NEVER absence-of-error. If a probe returns nothing, that's a FAIL not a SKIP.
4. **UNIQUE EVIDENCE TOKEN**: tests that interact with mutable framework state MUST embed a per-run UUID into the action AND look for that token in the resulting state. Cached results predating the token cannot match — this defeats the stale-cache false-pass.
5. **Audio / video features REQUIRE captured evidence**: features that produce audio output MUST be validated by capturing the actual output via a capture pipeline (loopback, hardware capture rig, or equivalent) — NOT by metadata-only checks. Features that produce video output MUST be validated via frame capture + OCR or pixel-difference analysis.

Each consuming project SHOULD ship a shared anti-bluff helper library (typically `tests/lib/anti_bluff.sh` or equivalent) that codifies these rules; every new runtime test MUST source it and use its assertion helpers.

---

§8. Bleeding-edge ultra-perfection quality bar

The acceptance bar for shipping a project change is **not** "tests pass and source compiles" — that is the floor. The acceptance bar is:

> **The user, on a freshly-deployed system, sees the expected effect when they perform the documented action — confirmed end-to-end, with captured evidence, on the actual target environment.**

The following are **forbidden**:

1. **"Source rebrand without shipped artifact rebuild"**: every source-side rename or rebranding MUST flow through to a shipped artifact that the runtime actually loads. Pre-build source-string gates are insufficient by themselves; shipped-artifact gates must accompany them.
2. **"Tests pass but feature broken"**: this is the failure mode §7.1 exists to eradicate. Any test that PASSes despite the feature being non-functional is a critical defect; its mutation pair must FAIL on the breakage.
3. **"Configuration-only tests"**: a test that only verifies a config file exists / has expected contents is downgraded to a pre-build gate. Runtime tests MUST exercise the actual user-visible behaviour and capture the result.

4. **“It works on my workstation”**: every fix is validated on every supported target before any release. No exceptions.
5. **“Will fix in next cycle”**: deferrals are documented in the changelog with concrete blockers + implementation plans, not just handwaved.

---

§9. Absolute codebase and data safety — zero risk, zero loss

Every destructive operation on the repository (history rewrite, force-push, branch deletion, bulk file removal, submodule de-init, pack repack that drops objects) is a **safety-critical** operation. Data loss from a wrong force-push is irreversible once the remote garbage-collects dangling objects. This section is non-negotiable.

§9.1 Mandatory safety protocol for destructive operations

Every destructive operation MUST execute, in strict order:

1. **Full backup before touching anything.** Hardlinked mirror of `.git` to a sibling backup directory:

   ```
   cp -al .git <backup>/<repo>.git.mirror
   ```

   This is near-instant on the same filesystem and uses zero additional disk. If `.git` is small, also create `git bundle create <backup>/main.bundle --all`.

2. **Record critical metadata into the backup dir**:

   - `git show-ref > refs.txt`
   - `git tag > tags.txt`
   - `git submodule status > submodules.txt`
   - `git rev-parse HEAD > head.txt`
   - `git rev-parse HEAD^{tree} > head_tree.txt`
   - `git ls-tree -r HEAD --full-tree | sha256sum > head_tree_content.sha256`
   - `git ls-tree -r HEAD --full-tree > head_tree_listing.txt`

3. **Identify the target state**: the expected HEAD commit, tree hash, and tree content hash after the operation completes. The operation MUST preserve these unless it is explicitly changing HEAD content.

4. **Run the operation** (filter-repo, rebase, etc.). NEVER use `--no-verify` / bypass-hooks flags. NEVER auto-force on failure.

5. **Post-operation verification gate** (all MUST pass):

   - HEAD tree hash matches pre-op target (content identical for non-content-changing rewrites)
   - HEAD tree content sha256 matches pre-op sha256
   - All tags from `tags.txt` are preserved and resolve to valid commits
   - All submodule pointers from `submodules.txt` match current state

    - Project's pre-build / pre-merge gates pass with zero FAIL and zero unclassified WARN

6. **Only if every check in §9.1.5 passes** may the operation be considered successful. Otherwise: restore from backup immediately (`rm -rf .git && mv <backup>/<repo>.git.mirror .git`).

7. **Then and only then** may a force-push proceed — and still not without explicit user authorization per §9.2.

§9.2 Force-push requires explicit user authorization every time

Force-pushing (including `push --force`, `push --force-with-lease`, or any divergence-recovery code path inside an automated wrapper) overwrites the remote's view of history. On any shared branch this is irreversible.

Rules: - Automated push wrappers MUST NOT auto-force-push as a fallback to any kind of failure (size rejection, divergence, server error, anything). The historical divergence-recovery code that silently force-resets the remote is forbidden and must be removed or gated behind an explicit flag. - A human must affirmatively say "force-push" (or equivalent) in the session to authorize each force-push. - Force-push to `main` (and any equivalent primary branch on owned submodules) is only authorized after the §9.1.5 gate has passed. - Every force-push event is recorded in `docs/changelogs/<tag>.md` under a "Force-push audit" section (target refs, backup location, validation gate output).

§9.3 Hardlinked backup is the standard — there is no excuse

Because hardlinked `.git` copies are near-instant and use zero additional disk (same-filesystem inodes), the cost of making a backup is trivial. Any destructive operation without a fresh hardlinked backup is a Constitution violation regardless of how small the repo is or how confident the operator.

§9.4 Commit-message audit trail for history rewrites

After any `git filter-repo` run (or equivalent), commit a record under `docs/changelogs/` (or `docs/history-rewrites/`) containing: - What was stripped and why - Pre-op `.git` size vs post-op `.git` size - Pre-op HEAD commit vs post-op HEAD commit - Pre-op HEAD tree hash vs post-op HEAD tree hash (MUST match for non-content-changing rewrites) - Backup location

---

§10. Enforcement

Any commit, tag, or release that violates §1–§9 is non-compliant. The fix is to amend/revert and re-land with compliance. No exceptions for speed pressure. Data-safety violations (§9) and host-session-safety violations (§12) are treated as catastrophic and block the entire release cycle until fully remediated.

---

§11. End-user quality covenant

§11.4 End-user quality guarantee — forensic anchor (User mandate, 2026-04-28)

The reason §7.1 + §8 exist — captured verbatim from the project owner so that future engineers and AI agents understand the historical failure mode the covenant exists to eradicate:

> "We had been in position that all tests do execute with success and all Challenges as well, but in reality the most of the features does not work and can't be used! This MUST NOT be the case and execution of tests and Challenges MUST guarantee the quality, the completion and full usability by end users of the product!"

This is the canonical motivation for every gate, every mutation pair, every meta-test, every anti-bluff helper, every captured- evidence requirement, every multi-environment validation rule. The covenant exists because the project has historically shipped broken features behind PASS-ing tests, and that outcome is no longer tolerated.

Operative rule: the bar for shipping is not "tests pass" but **"users can use the feature."** Every PASS in this codebase MUST carry positive evidence captured during execution that the feature works for the end user. Metadata-only PASS, configuration-only PASS, "absence-of-error" PASS, and grep-based PASS without runtime evidence are all critical defects regardless of how green the summary line looks.

Propagation requirement: every `CLAUDE.md` and every `AGENTS.md` in the project tree (parent + every submodule recursively) MUST contain a clearly-titled "MANDATORY ANTI-BLUFF COVENANT — END-USER QUALITY GUARANTEE" section that quotes this user mandate verbatim and points back to this §11.4 as the canonical authority. The consuming project enforces this via a pre-build gate (typically `CM-COVENANT-PROPAGATION`).

§11.4.1 — FAIL-bluffs are equally forbidden

The covenant must be enforced symmetrically. The historical failure mode the covenant eradicates is the **PASS-bluff**: a green test result on a feature that doesn't work for users. But the inverse failure mode is just as toxic — the **FAIL-bluff**: a red test result caused by a script bug (shell crash, missing argument, regex error, malformed assertion) rather than by an actual product defect.

Examples of FAIL-bluffs: - A test using `set -eu` calls `log_pass` without arguments → the helper references `$1` → script crashes with "1: parameter not set" → orchestrator records FAIL → engineer assumes product bug. - A test pre-condition check uses an unbounded sed range → a comment line in the source matches the range terminator → the intended mutation never fires → audit reports PASS — bluff.

A FAIL produced by a test bug is no more useful than a PASS produced by a test bug — both mislead investigation, waste cycles, and ultimately let real defects ship undetected.

Operative rule (extended): every test MUST fail ONLY for genuine product defects. A test that crashes for a script-internal reason — undefined variable under `set -u`, unhandled error, malformed pipe, regex syntax error, division by zero, etc. — is a critical defect in the test suite itself and MUST be fixed at the source layer (helper library, shared lib, test source) so the failure mode disappears project-wide, not patched in individual call sites.

§11.4.2 — Recorded-evidence requirement

A test that emits PASS without **captured visual or audio evidence of the user-visible feature actually working on the screen the user would see** is a §11.4 PASS-bluff. Closing this gap requires a project-wide recording + analyzer infrastructure consisting of (at minimum):

- **Recording wrapper** that captures the user-visible output to time-synchronized media files. Multi-display systems record every output stream in parallel; sync metadata (host wall-clock at spawn) is written alongside the recording.
- **Action-timeline emitter**: every test emits structured timeline events (JSONL or equivalent) at every user-visible transition.
- **Frame / audio analyzer**: scans the recordings, extracts samples at a configurable interval, feeds them to recognizers (OCR for text overlays, speech-to-text for audio, pixel-diff for frame health), correlates against the timeline, and emits findings.

Closure criterion: every PASS for a user-visible feature MUST be cross-checked by the analyzer against the recording + action timeline. A PASS that lacks at least one matched timeline event in the analyzer findings is treated as a §11.4 PASS-bluff.

§11.4.3 — Per-environment-topology test dispatch

Tests that depend on environment topology (presence/absence of secondary displays, microphones, network interfaces, peripheral devices, cloud services, etc.) MUST detect topology at test entry and dispatch the topology-appropriate variant. A test that runs the WRONG variant for the actual topology and PASSes is a §11.4 PASS-bluff: it claims a feature works while never having exercised it on the configuration the user actually has.

A topology-touching test that does NOT have a dispatcher AND a per- topology variant is a §11.4 violation regardless of how it "passes" on any specific configuration. Single-test-multi-topology scripts MUST gate every topology-dependent assertion behind an explicit topology check that emits SKIP-with-reason if the required topology is absent — never PASS-by-default.

§11.4.4 — Test-interrupt-on-discovery + retest-from-clean-baseline

A test cycle that *continues running past a freshly discovered defect* is itself a §11.4 PASS-bluff: it produces "all green" summaries that the operator might tag as a release while the codebase under test is known-broken at the moment those greens were recorded.

The mandatory protocol — non-negotiable:

1. **Interrupt immediately.** The moment a defect is re-discovered, re-produced, or newly identified during a test cycle, the cycle MUST stop. No "let it finish for the data" — the data is contaminated.
2. **Systematic debugging before any fix.** Identify root cause at source layer (not symptom), blast radius across related tests/features/subsystems, and regression-protection seam. Symptom patching is forbidden.
3. **Fix at root cause** per §11.4.1: source-layer fix, not symptom patch in the calling site. The fix MUST be safe (no new failure modes), minimal (no surrounding cleanup smuggled in), and non-regressive.
4. **Four-layer test coverage per fix.** Every fix lands with positive-evidence coverage in every applicable layer:
   - **Pre-build / pre-merge gate** (source-level catch)
   - **Post-build / packaging gate** (artifact-level catch)
   - **Runtime / on-device test** (end-user-behaviour catch)
   - **Meta-test paired mutation** (proves the gate is not itself a bluff)
5. **Documentation MUST be updated for every fix** — Issues → Fixed migration on closure, Applied Fixes table row, user-facing guides, architecture diagrams, per-version changelog.
6. **Rebuild the system.** Full build via the canonical build script. Skipping rebuild — even when the fix is host-only — invites stale-artifact contamination of the retest baseline.
7. **Re-deploy on every supported target.** A target left on the pre-fix build is a confounder.
8. **Repeat the full test suite** from the beginning on every target, sequentially per host memory cap. Partial reruns are a §11.4 violation.
9. **No-bluff certification per cycle.** Before tagging: meta-test harness returns all gates green AND every gate's paired mutation FAILs.

§11.4.5 — Captured-evidence quality analysis

§11.4.2 mandates *captured* evidence; §11.4.5 mandates the **content** of that evidence be analyzed for quality, not merely for presence.

Audio quality analysis — every audio test that PASSes MUST verify: 1. **Presence** — non-trivial RMS amplitude in captured output. 2. **Channel count** — analyzer reports the exact channel count the test claims (2.0 stereo, 5.1 surround, etc.). Stereo downmix in a "5.1 PASS" claim is a PASS-bluff. 3. **Sample rate + bit depth** — match the codec / pipeline under test. 4. **Glitch census** — underruns, XRUNs, dropouts above tolerance MUST be explicitly classified (PASS within budget, WARN above, FAIL on hard limits). Silently ignoring counts is a PASS-bluff. 5. **Coexistence-artifact census** — radio / IPC / shared-resource contention above tolerance.

Video quality analysis — every video test that PASSes MUST verify: 1. **Presence** — captured recording has non-zero size AND decoded- frame total > 0. A 0-byte file is the canonical PASS-bluff. 2. **Routing target** — analyzer confirms video appeared on the intended display / surface. 3. **Frame health** — drop count, jitter, freeze detection, tearing. 4. **Obstruction census** — OCR scan for hostile overlays (error dialogs, sign-in dialogs, geo-restriction overlays, paywall, etc). 5. **Resolution + codec** — captured dimensions match what the test claims.

§11.4.6 — No-guessing mandate

Tests, gates, status reports, closure narratives, commit messages, and any operator-facing text MUST NOT use words like `likely`, `probably`, `maybe`, `might`, `possibly`, `presumably`, `seems`, `appears to`, `guess`, `seemingly`, `apparently`, `perhaps`, `supposedly`, `conjectured`, or their synonyms when describing CAUSES of test failures, system behaviour, or fix effectiveness.

Either:

1. **Prove the cause** with captured forensic evidence (logs, kernel ring buffer, kernel ramoops, structured snapshots, OS journals, strace, etc.) and state it as fact, OR
2. **Explicitly mark UNCONFIRMED + PENDING_FORENSICS** with a tracked-task ID for follow-up.

Every "X likely caused Y" sentence in the codebase or documentation is a §11.4.6 violation. Every "appears to be benign" without a concrete forensic trace is a §11.4.6 violation.

§11.4.7 — Demotion-evidence rule

A demotion from any FAIL classification (`OPEN`, `POSSIBLE PRODUCT DEFECT`, `FAIL`) to a lower-severity classification (`INVESTIGATED`, `MITIGATED`, `RESOLVED`, `WORKING-AS-INTENDED`) requires positive evidence captured under the SAME CONDITIONS that originally exposed the defect:

1. **Same target** — defect on Target-A needs Target-A evidence; cross-target claims need every target.
2. **Same build** — evidence captured BEFORE a relevant fix landed does not validate post-fix behaviour.
3. **Same cycle position** — a stress-soak FAIL is not refuted by an isolated-test PASS.
4. **Same load profile** — a contention-mode FAIL is not refuted by a quiescent-mode PASS.

"I cannot reproduce in isolation" is a HYPOTHESIS, not a finding. Per §11.4.6 it MUST be tagged `UNCONFIRMED:` until same-conditions retest produces positive evidence.

§11.4.8 — Deep-web-research-before-implementation

Before designing a non-trivial fix, before implementing a new feature, before declaring an architectural choice — perform deep web research to verify the chosen approach is informed by current state-of-the-art. The research surface includes:

1. **Official documentation** of the platform, framework, language, or service.
2. **Vendor technical guides** for the hardware / cloud / library.
3. **Open-source codebases** that already solved analogous problems.
4. **Coding tutorials + technical articles**.
5. **Issue trackers** for the relevant projects.

Operative rule. A fix that re-invents a wheel (or reproduces a known-broken pattern) when the open-source community has already solved the problem is a §11.4 violation by omission.

Documentation requirement. Every non-trivial fix's commit message (or accompanying entry in the project's Issues / Fixed file) MUST cite the research sources that informed it: at least one external link OR the literal "NO external solution found — original work".

§11.4.9 — Batch-source-fixes-before-rebuild

When working through a multi-defect closure cycle, all source-side fixes that DO NOT require runtime validation to design MUST be landed BEFORE the next artifact rebuild. The anti-pattern this mandate eliminates is `Fix A → rebuild → flash → cycle → fix B → rebuild → ...` which serializes operator time onto rebuild latency.

Exceptions where a fix DOES require interim rebuild MUST be documented in the fix's commit message as `REQUIRES_REBUILD: <reason>`. The default is the batch path; rebuild-now is the operator-authorized exception.

§11.4.10 — Credentials-handling mandate

Forensic anchor — direct user mandate:

> "Credentials or any secret and sensitive data MUST NOT leak! This MUST BE added as the mandatory constraint into all Submodules and main project's Constitution, CLAUDE.MD and AGENTS.MD."

All credentials, secrets, API tokens, passwords, phone numbers, OAuth refresh tokens, signing keys, encryption keys, and any other authentication-or-authorization-bearing data used by test scripts, build automation, or any project tooling MUST be handled per the following rules without exception:

1. **No tracked-file storage.** Credentials NEVER live in any file that git tracks. Placeholder templates with `<replace ...>` or `EXAMPLE_VALUE_DO_NOT_USE` placeholders ARE allowed in `.example` files committed to show operators what keys to populate.
2. **Repository-wide ignore.** `.gitignore` MUST exclude all credential-bearing paths:
   - `.env`, `.env.*`, `*.env`, `.<service>.env`, `*.<service>.env`
   - `scripts/testing/secrets/*` with `.example` + `README.md` exception
3. **Runtime-load-only.** Tests load credentials AT EXECUTION TIME from operator-populated files under `scripts/testing/secrets/` (or per-project equivalent). If the file is missing, the test SKIPs with "credentials not configured for " — it does NOT proceed without credentials and does NOT use defaults.
4. **No echo / no log.** Test scripts MUST NEVER print, log, or include credentials in error paths, stack traces, screen recordings, or output artifacts.
5. **Per-service file separation.** One file per service keeps blast radius small.
6. **Filesystem permissions.** `.env` files are `chmod 600`, parent directory is `chmod 700`.

7. **Rotation on suspected leak.** Rotate at provider, update local files, audit captured artifacts.

The consuming project's `.gitignore` MUST be verified before every commit: `git ls-files --cached | grep -E "\\.env$"` MUST show no real `.env` files (only `.env.example`).

§11.4.10.A — Pre-store credential leak audit (User mandate, 2026-05-17)

Forensic anchor — verbatim user mandate (2026-05-17):

> "Us these for all future testing (full automation testing) and make sure they are not leaking anywhere or get git versioned!"
>
> [Discovered during execution: the operator-provided test credentials ALREADY existed in 5 tracked files dating to prior project cycles. §11.4.10 + §11.4.30 catch new commits but did not detect the pre-existing leak when the operator re-provided the same values for a new use.]

When an operator provides credentials, API tokens, signing keys, or any other secret material to be stored in the project's gitignored configuration (.env, scripts/testing/secrets/*, secrets/*, etc.), the agent or human storing them MUST FIRST execute a repo-wide audit for prior leaks of THOSE specific values BEFORE storing.

The audit MUST:

1. **Grep every tracked file** for the literal credential value(s). `git ls-files | xargs grep -l <value>` is the canonical form. Search MUST cover all file types — `.md`, `.json`, `.yaml`, `.kt`, `.go`, `.py`, `.sh`, `.sql`, build configs, docs.
2. **Grep the entire git history** for the literal credential value(s). `git log -S<value> --all --source --remotes` reveals historical commits even if the current tree is clean. A history-only leak is a §11.4.10 violation of equal severity to a tree-leak — anyone who pulled an older commit still has the value in their working copy of any branch they cloned.
3. **Report findings to the operator BEFORE storing the new value in any operator-controlled location.** The operator may then decide whether to (a) rotate the credential at the upstream provider before reusing, (b) accept the historical leak as a known-compromise and proceed, or (c) abort the storage. Storing the value without surfacing the audit is itself a §11.4 PASS- bluff at the security layer.
4. **If a leak is found:** open a forensic incident record per the project's §6/§7 sixth-law-incidents discipline (or equivalent), redact the literal values from all tracked files in-place (replace with `<redacted-per-§11.4.10>` placeholders), and record an OPERATOR ACTION REQUIRED for rotation per §11.4.10 sub-clause 7. The redaction commit lives on master IMMEDIATELY; the history-purge follow-up (git-filter-repo / BFG / equivalent + force-push to every upstream) requires explicit per-operation operator authorization per §9.2.
5. **Strengthen pre-push hook detection** when a previously-undetected class of literal credential is discovered. The pre-push hook's credential-pattern grep MUST be extended to catch the specific pattern class that escaped (literal usernames matched with literal passwords in the same file, well-known credential SHA-256 hashes, org-specific naming conventions for service accounts, etc.). The extension MUST land in the same commit as the redaction.

Pre-build gate (recommended) `CM-PRE-STORE-CREDENTIAL-AUDIT` (when implemented in consuming project) checks the commit message body of any commit that touches `.env`, `.env.example`, `scripts/testing/ secrets/`, or equivalent for an `Audit-Pre-Stored:` stamp citing the audit's outcome. Paired mutation (§1.1) strips the stamp and asserts gate FAILs.

Composes with §11.4.10 (the core credentials mandate this extends), §11.4.30 (.gitignore + no-versioned-artifacts — overlapping enforce- ment surface), §11.4.6 (no-guessing — audit findings stated as fact or as PENDING_FORENSICS), §11.4.7 (demotion-evidence — "I think it's not leaked" without grep evidence is a guess). Classification: universal (per §11.4.17). No escape hatch.

§11.4.11 — File-layout discipline

Project files MUST be organised by purpose, not by historical accident. Source code goes under canonical project roots. Tests go under canonical test directories. Logs and forensic artifacts go under operator-controlled directories (`~/Documents/`, `qa-results/`, `/tmp/`, OS-equivalent) — NEVER scattered at the repo root, NEVER inside working source trees, NEVER tracked unless they are reference assets explicitly required for evidence-replay.

Discovered drift is fixed by **moving** files to their canonical location and updating every caller — never by adding redirect shims or by leaving the misplaced copy "for backwards compatibility".

Carve-out (User mandate 2026-05-20). The 5 canonical tracker documents — `docs/Issues.md`, `docs/Issues_Summary.md`, `docs/Fixed.md`, `docs/Fixed_Summary.md`, `docs/CONTINUATION.md` — sit at `docs/` root by design. They are architectural constants of the project layout, analogous to AOSP's `Makefile`, `Android.bp`, `OWNERS` files at repo root. Their location is encoded as literal path strings in §11.4.12 + §11.4.15 + §11.4.16 + §11.4.19 + §11.4.44 + §11.4.53 propagation gates plus the helper-script constellation that regenerates them (`generate_issues_summary.sh`, `generate_fixed_summary.sh`, `sync_issues_docs.sh`, `commit_docs.sh`, `colorize_progress_html.py`, `export_progress_docs.sh`). Moving them would require coordinated amendment of those 6 sister anchors plus 5 pre-build gates plus ~20 helper scripts plus 42 consumer files (parent + 10 owned submodules + nested + HelixQA dependencies) in a single PWU. Per §11.4.66, that scope is operator-blocked until explicitly authorised. Audit-snapshot files (`docs/audit/anti_bluff_audit.md`, `docs/audit/PRE_SONOS_TAG_READINESS.md`, `docs/audit/D1_WIFI_FAIL_CLASSIFICATION.md`, plus any future audit snapshots) DO move under `docs/audit/` per the §11.4.11 general principle — they are not part of the tracker constellation and carry no propagation-gate path-string burden.

§11.4.12 — Auto-generated docs sync mandate

Every auto-generated document (Issues_Summary, API reference, build manifest, etc.) MUST be regenerated in the **same commit** as any edit to its source. Stale auto-generated docs are §11.4 PASS-bluffs in document form: an operator reading them gets a divergent view of project state.

All three file types MUST stay in sync at all times — Markdown source + HTML export + PDF export (or equivalent multi-format output the project ships). Enforced by a pre-build gate that checks mtime ordering and content hash agreement.

§11.4.13 — Out-of-band sink-side captured-evidence

Whenever a downstream consumer (HDMI sink, cloud service, monitoring target, downstream system) provides a network-accessible introspection API that reports what was actually received, the test suite MUST consume that report as captured-evidence for every test asserting end-to-end delivery.

The on-source-side view ALONE is insufficient — that is the exact "tests pass but the feature doesn't work" pattern §11.4 forbids. Sink-side reports MUST be: - Identity-verified (MAC, serial, UUID match) before consumed as evidence. - Topology-dispatched (§11.4.3) — sink-probe-unreachable → SKIP, never FAIL. - Cross-referenced with the on-source state at a matching wall-clock instant.

§11.4.14 — Test playback cleanup mandate

A test that completes (PASS, FAIL, or SKIP) MUST leave the target in a quiescent state — no orphan playback, no orphan recording, no orphan capture, no orphan background process continuing after the test's last assertion. Tests are transient probes; their side- effects on shared target state MUST NOT outlive the test.

Mandatory protections:

1. Every test that issues a playback or capture command MUST issue the matching cleanup at test end (force-stop the app, stop the recorder, close the file descriptor, terminate the helper).
2. Cleanup is mandatory on EVERY exit path — PASS, FAIL, SKIP, error, trap, signal, parent-killed. Use shell `trap '<cleanup>' EXIT` or `try/finally`.
3. Tests MUST verify the cleanup succeeded via positive evidence (§11.4.5).
4. The orchestrator MUST run a post-test sanity check between tests and FAIL the just-completed test if it left orphan state. This blames the leaker, not the next-test victim.
5. No grace period for "the next test will clean it up" — that is precisely the §11.4 PASS-bluff pattern. Cleanup is the test's responsibility, not the next test's, not the orchestrator's.

§11.4.15 — Item-status tracking mandate

Every active item tracked in the project's Issues file MUST carry a `**Status:**` line within five lines of its heading. The status value is drawn from a closed set:

State	Meaning
`Queued`	In backlog, no work has started. Sub-state qualifier `Queued — BLOCKED on <reason>` permitted.
`In progress`	Active investigation or coding underway.
`Ready for testing`	

State	Meaning
	Source-side fix landed (pre-build / meta-test gates green), waiting for the next deploy cycle.
`In testing`	A deploy + cycle is in flight that exercises the fix; live captured-evidence is being collected per §11.4.2 + §11.4.5.
`Reopened`	Item failed its runtime test cycle and is back to active work. Carries a reference to the failure-cycle artefact.
`Fixed (→ Fixed file)`	Item closed: captured-evidence per §11.4.5 collected, runtime test PASSes, paired meta-test mutation FAILs, item migrated to the Fixed file.

Status MUST be updated as the item progresses. Every commit that changes an item's lifecycle state MUST update its Status line in the same commit.

All three Issues / Issues_Summary / Fixed file types MUST be in sync at all times (Markdown + HTML + PDF).

§11.4.16 — Item-type tracking mandate

Every active item tracked in the project's Issues file MUST carry a `**Type:**` line within eight non-blank lines of its heading (the same window the §11.4.15 Status audit uses). The type value is drawn from a CLOSED set of three values:

Type	Meaning
`Bug`	Product defect / regression / user-visible broken behaviour. The product worked before (or was expected to) and now does NOT for the end user.
`Feature`	New capability not previously offered to end users — a new integration, new output, new probe, new bank, new architectural surface.
`Task`	Internal workstream — not user-visible. Refactor, infrastructure, documentation, audit, covenant clause, gate, mutation, propagation enforcement, host-session-safety hardening. The largest class by count; the lowest-stakes default when the classification is ambiguous.

The vocabulary is CLOSED — only `Bug`, `Feature`, `Task`. Any other value is a violation. When ambiguous, fall back to `Task`.

Type tells the operator WHAT KIND of work an item is — distinct from severity (HOW URGENT) and status (WHERE IN LIFECYCLE). The three axes together drive ownership routing, testing strategy, release-note framing, and changelog classification.

The Issues_Summary file MUST carry the Type column for every active item. All three file types (Markdown + HTML + PDF) MUST stay in sync under the same `CM-DOCS-EXPORT-SYNC` discipline as §11.4.12 + §11.4.15.

Pre-build gates `CM-ITEM-TYPE-TRACKING` (scans Issues file for `**Type:**` presence within the audit window) and `CM-COVENANT-114-16-PROPAGATION` (anchor presence in every project CLAUDE.md / AGENTS.md) enforce the mandate. Paired mutations in the project's meta-test prove neither gate is a bluff gate.

No escape hatch — items that genuinely don't fit MUST be re-classified as `INFORMATIONAL` and excluded from the active-item set rather than carry a fabricated Type.

§11.4.17 — Universal-vs-project classification of new rules (User mandate, 2026-05-14)

Forensic anchor — direct user mandate (verbatim, 2026-05-14):

> "Adding any new rules or mandatory constraints or anything relevant which should be added into Constitution, CLAUDE.MD, AGENTS.MD and relevant files MUST BE determined as reusable / universal VS project specific. If it is universal and reusable it MUST BE added into our root / main constitution Submodule, otherwise into our project level Constitution, AGENTS.MD and CLAUDE.MD (or Submodule if it is Submodule related)."

Before any new rule, mandatory constraint, covenant clause, gate declaration, or "MUST"-bearing statement is added to a project's Constitution / CLAUDE.md / AGENTS.md or to a child submodule's equivalents, the author MUST first classify it along this axis:

Classification	Definition	Destination
Universal (reusable)	The rule applies to ANY project — independent of language, framework, target platform, or domain. Examples: anti-bluff covenant, no-guessing mandate, credentials-handling, host-session safety, item-status tracking, multi-upstream push, file-layout discipline, deep-web-research-before-implementation.	This constitution submodule's `Constitution.md` + `CLAUDE.md` + `AGENTS.md`. Propagates to every consuming project via inheritance.
Project-specific		

Classification	Definition	Destination
	The rule references a specific hardware target, vendor, model, SoC, vendor-fork, project-internal package name, deployment topology, or company-internal asset. Examples (illustrative — these are intentionally NOT in this universal file): the SoC's specific power-management quirks; a particular Android/iOS/Linux SDK behaviour; a specific app or service bundled with the project.	The project's own `Constitution.md` / `CLAUDE.md` / `AGENTS.md` (top-level), OR the affected submodule's equivalents (when the rule scopes to that submodule).

A rule that mentions hardware part numbers, vendor names, specific package identifiers, geographic regions, or company-internal asset names is **project-specific by definition** — it cannot be lifted into the universal layer without genericising those references first. A universal rule MAY reference patterns (e.g., "USB-HID multitouch panels") but MUST NOT name specific vendors.

Anti-bluff: the universal-vs-project classification is itself an auditable artefact. Every new rule's commit message MUST include a one-line classification statement (e.g., `Classification: universal — applies to any project tracking items through an Issues/Fixed lifecycle`). A commit that adds a rule without classification justification is a §11.4.17 violation.

Authors who are uncertain SHOULD default to project-specific (narrower scope). Lifting a project-specific rule to universal later is cheap (one merge); generalising a universal rule that turned out to leak project-specific assumptions is expensive (every consumer must update).

Pre-build gate `CM-UNIVERSAL-VS-PROJECT-CLASSIFICATION` audits the last N commits for rule additions and asserts each one carries a classification statement. Paired mutation strips the classification literal and asserts the gate FAILs.

§11.4.20 — Subagent-driven-by-default mandate (User mandate, 2026-05-14)

Forensic anchor — direct user mandate (verbatim, 2026-05-14):

> "Make sure we ALWAYS WORK EVERYTHING subagents-driven if it is possible or applicable!"

When operating in a multi-agent runtime (Claude Code with subagents, Cursor with task-runners, Aider with sub-sessions, or any CLI tool that supports delegated execution), the primary agent MUST default to subagent delegation for any task that satisfies AT LEAST ONE of:

1. **Multi-step scope** — three or more discrete editing / file-creation / verification phases. Hand-off boundaries are where stalls happen; pre-planned subagent decomposition avoids them.
2. **Parallelisable work** — two or more independent tasks with no shared mutable state. Parallel subagents finish wall-clock-faster than sequential foreground work AND insulate the primary agent's context window from the volume of file reads.
3. **Long-running diagnostic loops** — repeated probe / re-test cycles, build-flash-cycle loops, soak tests. Subagents survive context compression boundaries that would otherwise truncate progress.
4. **Domain-specific or specialised workflows** — code review, security audit, infrastructure change review, performance triage, documentation propagation across N files. Specialised subagents (`code-reviewer`, `iac-reviewer`, `general-purpose`) bring pre-curated tool sets and disciplines the primary agent would re-derive from scratch.

The primary agent SHOULD only do foreground work when AT LEAST ONE of:

- The task is a single edit or single file read with no follow-up.
- The task requires conversational clarification with the operator mid-execution (subagents cannot ask the operator questions).
- The task is gating critical state (a commit, a push, a tag cascade) that must be sequenced before the next operator action.
- The task is so quick (under ~30 seconds of execution) that subagent spin-up overhead exceeds the work itself.

Anti-stall discipline. Subagents have watchdog timeouts in most runtimes (Claude Code: ~600 s of no-progress). When delegating, the parent agent MUST: (a) scope tightly (no "complete everything" prompts — break into 4-6 tightly-scoped tasks); (b) require **checkpoint commits** after each major task so partial progress is preserved even on stall; (c) explicitly prohibit destructive operations the subagent might attempt to "be helpful" (`git reset --hard`, `git push --force`, `--no-verify`); (d) provide explicit discipline pointers (§11.4.6 no-guessing, §11.4.10 credentials, §11.4.17 classification).

Anti-bluff applied. A subagent that returns "completed" without captured evidence in commits / outputs / artifacts is a §11.4 PASS-bluff at the multi-agent layer. The parent agent MUST verify subagent claims against repo state (`git log`, `git status`, post-completion gate runs) before treating the work as landed.

Coordination. When parallel subagents touch overlapping files, the parent agent MUST partition the work so subagents work on **non-overlapping files**. The parent's `commit_all.sh --auto-cascade` naturally bundles both subagents' dirty files into atomic commits via `git add -A` — exploit this for "forward-reference + provider" patterns where one subagent creates files another subagent references.

No escape hatch. Operating exclusively in the foreground when subagent delegation is feasible burns operator wall-clock time, risks context-window overflow on large tasks, and forfeits the parallelism that multi-agent runtimes were designed to provide. Pre-build gate `CM-SUBAGENT-DELEGATION-AUDIT` (when

implemented per consuming project) scans recent session transcripts for multi-step foreground work that should have been delegated; paired mutation enforces the gate is not a bluff.

§11.4.18 — Script documentation mandate (User mandate, 2026-05-14)

Forensic anchor — direct user mandate (verbatim, 2026-05-14):

> "Make sure that every single script inside the scripts dir (in all subdirs in depth) is fully documented and covered with full user manuals and user guides! … Whenever is some bash script modified we MUST document it fully and create for it complete user guide(s) and manual(s) - if already exists make sure all of it is in sync and fully updated! No documentation ever can be out of sync with its codebase!"

Every Bash / shell / POSIX-sh script ANYWHERE in a project (anywhere under `scripts/`, `bin/`, `tests/`, library directories, `Upstreams/`, deployment hooks, CI helpers, etc. — depth-N recursive) MUST carry:

1. **In-source documentation block** at the top of the file:
   - Purpose: one-sentence description of what the script does.
   - Usage: complete invocation syntax with all flags, env vars, positional arguments, examples.
   - Inputs: env vars, files read, command-line args, stdin.
   - Outputs: files written, stdout/stderr behaviour, exit codes.
   - Side-effects: anything that changes system state outside the script's own scratch directory.
   - Dependencies: required commands (and how to install them) + required system state (e.g., "must be run as root", "must run from project root").
   - Cross-references: companion guides under `docs/`.
2. **External user guide** under `docs/scripts/<script-name>.md` (Markdown). Required sections:
   - **Overview** — what the script does in plain English.
   - **Prerequisites** — environment + dependencies.
   - **Usage examples** — copy-paste recipes for the common cases.
   - **Edge cases** — unusual invocations, error conditions, how to diagnose them.
   - **Internal behaviour** — for advanced users / future maintainers: what the script does step-by-step.
   - **Related scripts** — companion scripts and how they compose.
   - **Last verified version / date** — when the doc was last reconciled against the script source.

Whenever the script source is modified, the in-source documentation block AND the external user guide MUST be updated in the SAME commit. A commit that touches a script but leaves its documentation unchanged (or worse, lets it drift) is a §11.4.18 violation.

Pre-build gate `CM-SCRIPT-DOCS-SYNC` walks every `*.sh` and `*.bash` under `scripts/` (or equivalent script directories), verifies a companion `docs/scripts/<name>.md` exists, AND verifies the doc was modified in the same commit as the script (by SHA cross-reference, or by mtime ≥ script mtime as a softer floor). Paired mutation strips the doc-sync invariant and asserts the gate FAILs.

No documentation ever can be out of sync with its codebase.

This mandate composes with §11.4.11 (file-layout discipline — docs live under `docs/`, scripts live under `scripts/`) and §11.4.12 (auto-generated docs sync — the on-disk Markdown + its rendered HTML/PDF must all stay synchronised).

§11.4.19 — Fixed-document column-alignment mandate (User mandate, 2026-05-14)

Forensic anchor — direct user mandate (verbatim, 2026-05-14):

> "Make sure that Fixed document (all 3 file formats) is consistent and has the same structure and columns as Issues and Issues_Summary documents! This has to be added as detail to Constitution, CLAUDE.MD and AGENTS.MD (constitution Submodule where these information shall exist)."

Every project that maintains an Issues / open-work tracker AND a Fixed / closed-archive tracker MUST keep the two structurally aligned along the same lifecycle axes — at minimum **Status** and **Type** — so an operator reading either gets a consistent view across the entire lifecycle (open → in progress → ready for testing → in testing → reopened → fixed).

Specifically:

1. **Per-entry annotation.** Every heading in the Fixed archive document MUST carry, within 8 non-blank lines of the heading, a `**Status:**` line and a `**Type:**` line — exactly as §11.4.15 + §11.4.16 require for the Issues tracker. For Fixed entries the Status value is drawn from the **closure-set**:

   - `Fixed (→ Fixed.md)` — fully verified on device with captured evidence per §11.4.5.
   - `Fixed — pending device verification` — source-side fix landed, regression-protection gate wired, awaiting the next firmware build + flash + cycle for captured evidence.
   - `Fixed — RECLASSIFIED` — item was reclassified during investigation (test-side defect, environment-only, etc.).

   Type values follow §11.4.16: `Bug` | `Feature` | `Task`. Default on missing-annotation: `Task`.

2. **Auto-generated companion summary.** A `Fixed_Summary.md` MUST exist alongside `Fixed.md`, regenerated by an automation script (e.g. `generate_fixed_summary.sh`), and MUST mirror the column structure of `Issues_Summary.md` exactly:

   ```
   | # | Level | Status | Type | One-line description |
   ```

   The `Level` (severity) column uses the **original severity** the item had when it was open — so an operator can correlate a closed item with its original priority bucket. The same deterministic classification rules apply (REAL PRODUCT DEFECT / PARTIAL-or-WORKSTREAM-or-DEEP-DIVE / everything-else mapping to C / M / L).

3. **Three-format sync.** All three file types (`.md`, `.html`, `.pdf`) for BOTH the Fixed archive AND the Fixed_Summary MUST stay in sync at all times — same `.html` + `.pdf` export pipeline that §11.4.12 requires for Issues + Issues_Summary. The single-shot sync wrapper (e.g. `sync_issues_docs.sh`) MUST invoke the Fixed_Summary generator AND export all five doc variants (Issues, Issues_Summary, Fixed, Fixed_Summary, CONTINUATION) in one operation so they all carry the same mtime.

4. **Cross-lifecycle consistency.** A status line in Issues.md that reads `Fixed (→ Fixed.md)` is a **migration marker**, not a self-describing closure: when an Issues.md entry resolves, it MUST be moved to Fixed.md in the same commit, then disappear from Issues_Summary (because Issues_Summary only enumerates OPEN items) and appear in Fixed_Summary (because Fixed_Summary enumerates CLOSED items). Per §11.4.4 fix-closure protocol + §11.4.5 captured-evidence requirement, the migrated entry in Fixed.md MUST carry: closure cycle, closure commit SHA, captured evidence pointer, regression-protection gate name + paired mutation reference.

Pre-build gate `CM-FIXED-COLUMN-ALIGNMENT` (5+ invariants): - `docs/Fixed_Summary.md` exists. - `docs/Fixed_Summary.md` table header line contains both `Status` and `Type` columns (column-alignment with Issues_Summary). - `mtime(Fixed_Summary.md) >= mtime(Fixed.md)` (regenerated after Fixed.md edits). - Generator script exists (`generate_fixed_summary.sh` or equivalent integrated branch in `generate_issues_summary.sh`). - Sync wrapper invokes the Fixed_Summary generator (grep for the script name in `sync_issues_docs.sh` or equivalent). - HTML + PDF exports for both Fixed and Fixed_Summary exist (`docs/Fixed.html`, `docs/Fixed.pdf`, `docs/Fixed_Summary.html`, `docs/Fixed_Summary.pdf`).

Paired mutation: strip the Status column from `Fixed_Summary.md` table header (or rename the generator) → gate FAILs. Enforced by `meta_test_false_positive_proof.sh`.

Propagation. This anchor is a §11.4.17-classified **universal** rule — it composes naturally with §11.4.12 (auto-generated docs sync), §11.4.15 (Status tracking), §11.4.16 (Type tracking) and is mandatory across every consuming project's CLAUDE.md / AGENTS.md. The propagation gate `CM-COVENANT-114-19-PROPAGATION` (when implemented per consuming project) enforces the anchor's presence in every covenant file.

No escape hatch. A Fixed archive that lacks Status/Type columns or whose Summary companion drifts out of sync is the exact "different lifecycles report different facts" hazard §11.4 forbids — an operator reading Issues_Summary can no longer answer "where is item X now?" by cross-referencing Fixed_Summary because the schemas diverge. Non-compliance is a release blocker regardless of context.

§11.4.21 — Operator-blocked status + self-resolution exhaustion mandate (User mandate, 2026-05-14)

Forensic anchor — direct user mandate (verbatim, 2026-05-14):

> "Add into Issues and Issues_Summary new status: Operator blocked! We need another column with details in Issues document with exact details on how exactly is operator blocked issue! Before issue becomes operator blocked we MUST investigate in-depth all ways on how it can be resolved without operator's involvement - by you, or by activating any relevant mechanism for the resolution!"

Every project that maintains an Issues / open-work tracker MUST treat operator dependency as a **last-resort classification**, earned only after documented exhaustion of self-resolution paths. Routing an item to the operator without proving the agent + tooling cannot unblock it themselves is the §11.4 PASS-bluff pattern transplanted into the planning layer — the work stalls indefinitely while the agent claims "nothing more I can do."

1. Status vocabulary extension. §11.4.15's closed-set of Status values is extended with a seventh value, `Operator-blocked`. The full closed-set becomes:

#	Status value	Meaning
1	`Queued`	Awaiting work to start.
2	`In progress`	Active agent / developer work.
3	`Ready for testing`	Source-side fix landed, awaiting verification.
4	`In testing`	Captured-evidence cycle running.
5	`Reopened`	Previously closed item resurfaced (regression).
6	`Operator-blocked`	**NEW.** Cannot proceed without an operator action that the agent + available tooling cannot perform.
7	`Fixed (→ Fixed.md)`	Closure migration marker per §11.4.15 + §11.4.19.

2. Self-resolution exhaustion mandate. BEFORE classifying any item as `Operator-blocked`, the agent MUST verifiably exhaust every self-resolution path applicable to the project:

- **(a) CLI / ADB / SSH / API access** — can the work be done via any access the agent already has? (e.g. `adb shell pm clear`, `gh api`, `aws ssm`, `kubectl exec`, `psql`).
- **(b) Subagent delegation** — can a specialised subagent (per §11.4.20) reach further than the primary agent? (e.g. `general-purpose` for multi-step probing, `code-reviewer` for architectural blockers, `iac-reviewer` for infra access).
- **(c) Existing tooling** — is there a script / helper / library already in the repo that handles this case? (e.g. `scripts/testing/<helper>.sh`, project-specific automation).
- **(d) Captured fallback** — can the blocker be sidestepped with a synthetic event, test asset substitution, mock, or topology-aware SKIP per §11.4.3?

(e.g. simulate the input, substitute a stand-in fixture, downgrade to a pre-build gate).
- **(e) Documentation + research** — per §11.4.8, has the agent consulted external sources (official docs, vendor guides, open-source codebases, issue trackers) for a self-resolution pattern the community has already solved?

Only AFTER each applicable path is **verifiably attempted and documented as exhausted** is `Operator-blocked` the correct classification. "I didn't try X because I assumed it wouldn't work" is a §11.4.6 no-guessing violation AND a §11.4.21 self-resolution violation.

3. Operator-Block-Details mandate. Every `Status: Operator-blocked` item MUST carry an `**Operator-Block-Details:**` line within 8 non-blank lines of its heading (mirroring the §11.4.15 Status placement pattern). The content MUST state ALL of:

- **WHAT** — the specific concrete action the operator must perform (verb + object + target system). Generic "the operator must investigate" is forbidden — name the action.
- **WHY** — the alternatives that were exhausted, enumerated. Each exhausted path from §11.4.21.2 above gets a one-line statement of what was attempted and why it could not unblock the item. "Subagent delegation: tried general-purpose with X scope — blocked because Y" is the bar.
- **UNBLOCK CONDITION** — the observable signal that the operator has completed the action (e.g. "operator confirms hardware rev-B installed", "operator pastes new API key into `scripts/testing/secrets/.foo.env`", "operator force-pushes PR #123 merge to upstream"). Without this, the agent cannot automatically detect when to re-evaluate.
- **WHO** — the handle / contact / pointer to the document where operator-side details live if questions arise (e.g. "operator: @username", "see `docs/operator/<topic>.md`", "vendor: support-id 12345"). Operator-blocked without WHO is operationally untrackable.

4. Issues_Summary inclusion as sortable axis. The auto-generated `Issues_Summary.md` (per §11.4.12 + §11.4.15) MUST include `Operator-blocked` as a first-class Status value in the table — the column header is unchanged but operators can sort / filter on it deterministically. The summary generator MUST also emit a count of `Operator-blocked` items in any rollup line / header.

5. Periodic re-evaluation requirement. Items in `Operator-blocked` status MUST be re-evaluated every Nth tag-cycle (project-defined, recommended every 3rd tag cycle). Operator dependencies change over time: CI may have been added, hardware may have been swapped, automation may have matured, vendor APIs may have unlocked. An item that was correctly Operator-blocked at tag T may be self-resolvable at tag T+3. Re-evaluation MUST follow the same §11.4.21.2 exhaustion checklist and document each re-attempt.

6. Anti-bluff layer. A fake `Operator-blocked` (classification applied without exhausting §11.4.21.2 alternatives) is a §11.4 covenant violation at the planning layer — equivalent severity to a PASS-bluff at the testing layer. The agent's commit message introducing or maintaining an `Operator-blocked` classification MUST contain evidence of self-resolution attempts (the explicit "Attempted: a — exhausted because X; b — exhausted because Y; c — exhausted because Z" form).

7. Pre-build gates (recommended, per consuming project):

- **`CM-ITEM-OPERATOR-BLOCKED-DETAILS`** — every heading whose Status line equals `Operator-blocked` has an `**Operator-Block-Details:**` line within 8 non-blank lines. Paired mutation strips the details line → gate FAILs.
- **`CM-OPERATOR-BLOCKED-SELF-RESOLUTION-AUDIT`** — every NEW `Operator-blocked` item introduced in a commit carries an "Attempted: …" audit trail (either in the Issues.md entry body OR in the commit message). Paired mutation introduces an `Operator-blocked` item without the audit trail → gate FAILs.

Propagation. This anchor is a §11.4.17-classified **universal** rule — it composes with §11.4.15 (Status tracking), §11.4.16 (Type tracking), §11.4.19 (Fixed-document column alignment), §11.4.12 (auto-generated docs sync), §11.4.20 (subagent delegation as a self-resolution path), §11.4.8 (research-before-implementation as a self-resolution path), §11.4.6 (no guessing about what the operator "would have to do"). Propagation gate `CM-COVENANT-114-21-PROPAGATION` (when implemented per consuming project) enforces the anchor's presence in every CLAUDE.md / AGENTS.md across the parent + every owned submodule + every dependency.

No escape hatch. Items that legitimately need operator intervention are real and unavoidable — hardware swaps, vendor contracts, physical-world inputs, account credentials, irreversible business decisions. But the classification must be **earned** via documented exhaustion. An `Operator-blocked` heading without an audit trail of self-resolution attempts is a §11.4 release blocker regardless of context.

---

§11.4.22 — Document-sync commit discipline (User mandate, 2026-05-14)

Forensic anchor — direct user mandate (verbatim, 2026-05-14):

> "For Issues, Issues_Summary and Fixed docs (and all its exports) we MUST commit and push (only them) as soon as they are updated (synced). Continuation doc and other similar / relevant documentation MUST be committed and pushed too! Like this we will always have up to date status of working items without need to do full commit and push of everything if that is not possible to do!"

Classification: §11.4.17-classified **universal** — every project that tracks work items through an Issues / Fixed lifecycle has the same problem, so the discipline lives in the Constitution and every consuming project implements its own wrapper.

The defect this anchor closes. Until this anchor existed, the only way to push a status update was the full-tree commit wrapper. When the working tree had unrelated heavy churn (large submodule diffs, in-flight rebase, partial-network conditions, ongoing build), operators had two bad options: (a) wait until the heavy work finishes — which can be hours — leaving the doc-status stale to every other agent; (b) skip the doc-status update entirely — which is a §11.4.4 documentation-drift violation. Neither option is acceptable.

The mandate. Every project under this Constitution that ships a status-tracking doc set MUST provide a **lightweight commit path** distinct from the full-repo commit wrapper. The lightweight path stages, commits, and pushes **only** the status-tracking doc set:

1. The active issue tracker (`docs/Issues.md` in ATMOSphere's layout; the project's equivalent file elsewhere) and its HTML + PDF exports.
2. The auto-generated issues summary (`docs/Issues_Summary.md`) and its HTML + PDF exports.
3. The fixed-bug archive (`docs/Fixed.md`) and its HTML + PDF exports.
4. The auto-generated fixed summary (`docs/Fixed_Summary.md`) and its HTML + PDF exports.
5. The cross-session continuation document (`docs/CONTINUATION.md`, per §12.10) and its HTML + PDF exports.
6. Any auto-generated audit artifact (`docs/audit/anti_bluff_audit.md` or equivalent) that pairs with the above.

The lightweight wrapper MUST:

1. **Auto-invoke the project's export-regeneration pipeline first** — so Markdown + HTML + PDF are all synchronised before push. (Skipped only when an explicit `--no-sync` flag is passed AND the operator just ran the pipeline manually.)
2. **Stage ONLY the doc-set files** — `git add` of an explicit file list, NEVER `git add -A` (which would catch unrelated WIP churn).
3. **Use a separate flock** disjoint from the full-tree commit wrapper's lock — so the two can run in parallel without contention on disjoint file sets.
4. **Push to every parent-repo remote** (reusing the project's push-all driver where available).
5. **Exit code semantics** — `0` success, `1` validation failure, `2` lock contention, `3` nothing-to-commit (informational, not an error — so the wrapper can be wired into automation that tolerates no-op outcomes).

The lightweight wrapper MUST be invocable either standalone OR via a flag on the full-tree wrapper (e.g. `commit_all.sh --docs-only`) so operators have a single mental model.

§9 preflight inheritance. The lightweight wrapper inherits the parent project's §9 preflight discipline — if `meta_test_*` is mid- mutation or `*.mut.bak` files exist, the wrapper refuses to run, same as the full-tree wrapper.

Pre-build gates (recommended, per consuming project):

- **`CM-COMMIT-DOCS-EXISTS`** — verifies the lightweight wrapper exists + is executable, its external user guide (per §11.4.18) exists, the full-tree wrapper advertises the delegation flag, and the wrapper's doc-set enumeration lists ≥ N entries (N depends on project — ATMOSphere's set is 15). Paired mutation strips the doc-set array → gate FAILs.

Propagation. Composes with §11.4.12 (export-sync invariant — HTML + PDF in sync with Markdown after every edit), §11.4.15 (item status tracking — Status lines must be visible quickly), §11.4.18 (every script ships with an in-source doc block + an external user guide), §12.10 (CONTINUATION document maintenance — the lightweight wrapper is one of the mechanisms that keeps CONTINUATION current).

No escape hatch. The discipline exists because doc-status drift is itself a §11.4 PASS-bluff pattern at the documentation layer: a green-looking `Issues_Summary.html` that doesn't reflect Markdown reality is the same class of defect as a passing test that doesn't reflect runtime reality. Operators who can't run the lightweight wrapper for some reason MUST run the full-tree wrapper and accept the heavier cost — the doc-status drift is non-negotiable.

---

§11.4.23 — Visual-cue & grouping mandate for Issues docs (User mandate, 2026-05-14)

Forensic anchor — direct user mandate (verbatim, 2026-05-14):

> "We MUST make sure that Issues, Issues_Summary and Fixed docs and its exported files (PDFs and HTMLs) have one small improvement: we MUST introduce some background coloring for cells of item type and status! Also, we MUST group items by the status! … Base colors for types would be: bug - background pale red, task - background pale blue, feature - background pale yellow. However, changing the status affects the color of the cell of the type and the status cell background color: queued - no color, no effect, fixed - both cells pale green, in progress - only status cell is pale green, reopened - status cell is pale red, blocker - status cell is red (not pale, however MUST BE still readable), anything else please follow the same logic!"

Classification: §11.4.17-classified **mixed** — the discipline (visual cues + status grouping for tracked-item docs) is universal across every project that maintains an Issues / Fixed lifecycle. The implementation (CSS classes, HTML post-processor, weasyprint integration) is project-specific because each project's export toolchain differs (pandoc, asciidoctor, sphinx, etc.).

The defect this anchor closes. A long, multi-column tracked-item table where every row looks identical at first glance is a §11.4 operational-readability defect — operators have to *read* every Status column to identify what is queued vs in-progress vs fixed. With dozens to hundreds of items, this is slow enough that operators inevitably skim, and the doc effectively becomes write-only. Color cues + status grouping make at-a-glance triage possible.

The mandate. Every project under this Constitution that ships tracked-item docs (per §11.4.15) MUST apply visual-cue coloring AND status grouping to those docs' HTML + PDF exports. The Markdown source stays free of color noise (Markdown is the canonical text), but the HTML + PDF exports MUST be enriched in a post-processing stage:

Type cell base colors:

Type	Background
Bug	pale red (`#FFE5E5` or equivalent — ~5% red saturation)
Task	pale blue (`#E5F0FF`)
Feature	pale yellow (`#FFF8DC`)

Status cell colors (override the Status column; Fixed also overrides Type cell):

Status	Status cell	Effect on Type cell
Queued	no color (transparent)	keeps base type color
In progress	pale green (`#D4F1D4`)	keeps base type color
Ready for testing	pale green-blue (`#C6EAEF`)	keeps base type color
In testing	pale green-yellow (`#E7F4D6`)	keeps base type color
Reopened	pale red (`#FFCCCC`)	keeps base type color
Fixed	pale green (`#A8E6A8`)	**also** pale green
Operator-blocked / Blocker	vibrant red (`#FF4444`) with white text for readability	keeps base type color

Status grouping: items in the rendered table MUST be grouped by Status, emitting an H3-class heading for each Status group with the item count. Group order (most-actionable first; closed last):

1. Operator-blocked / Blocker
2. In testing
3. Ready for testing
4. In progress
5. Reopened
6. Queued
7. Fixed

This ordering surfaces the items requiring operator attention at the top of the doc; closed/archived items sink to the bottom.

Print-fidelity requirement. The CSS MUST declare `print-color-adjust: exact` (or the project's renderer's equivalent) so PDF exports preserve the cell backgrounds. A weasyprint default-rendered PDF that strips colors is a §11.4 export-drift defect — HTML and PDF MUST be visually equivalent.

Pre-build gates (recommended, per consuming project):

- **`CM-DOC-COLOR-GROUPING-DISCIPLINE`** — verifies the project's CSS carries the type+status class set, the post-processor exists and is executable, the sync wrapper invokes it, AND after sync the rendered HTML carries the grouping-heading + per-cell color classes (the captured-evidence proof — without this last invariant a stub post-processor could silently pass the static gate). Paired mutation strips a Status class literal from CSS → gate FAILs.

Propagation. Composes with §11.4.12 (export-sync invariant — HTML + PDF in sync with Markdown after every edit), §11.4.15 (item status tracking — every active item carries a Status line from the closed-set vocabulary), §11.4.19 (Fixed_Summary column-aligned companion), §11.4.22 (lightweight commit path for doc-set updates).

No escape hatch. Color cues are not optional polish — they are the operational-readability seam that converts a long tracked-item table from a write-only Markdown blob into a self-triaging document.

---

§11.4.24 — Build-resource stats tracking mandate (User mandate, 2026-05-14)

Forensic anchor — direct user mandate (verbatim, 2026-05-14):

> "We need to incorporate through the Containers Submodule we use to run our System build Container the following mechanism safely: during its working lifetime we MUST track in proper Markdown document exported into PDF and HTML stats on System resources use! What was the peak and the minimum EVER the Container / System building process used! … In top of the document we MUST have this ever values. … After this we MUST present properly sorted divided by version tags names all build processes we were running with notes if the process completed with success or not, and resources usage during the process! Besides min and max values ever and per build iteration we MUST have for each resource we have average values — average memory usage, average CPU usage, and all other parameters IO and other resources!"

Classification: §11.4.17-classified **mixed** — the discipline of tracking host-side build-resource usage with min / max / mean / p95 per-build PLUS ever-values across all builds in a versioned Markdown + HTML + PDF triple is universal across every long-build project (AOSP, kernel, large monorepo, ML model training, multi-hour data pipelines). The implementation (registry path, monitor script name, exporter wiring) is project-specific.

The defect this anchor closes. Long builds (multi-hour AOSP builds, ML training runs, large monorepo CI) consume highly variable resource profiles over their lifetime — soong_build's 33 GB peak coexists with quiescent linker-only stretches at <2 GB. Without per-build sampling + ever-values, operators have NO empirical basis to (a) size containers correctly, (b) detect resource regressions across versions, (c) prove which build iteration was the OOM-killer's trigger, (d) reason about parallelism caps (cf. §12.7 -j2 hard cap whose forensic basis would have been one full Stats.md generation earlier had this discipline been in place). Memory pressure debugging without time-series data is the bluff this anchor forbids.

The mandate. Every project under this Constitution with a build exceeding **1 minute wall-clock** MUST:

1. Run a host-side resource sampler for every build. The sampler runs as a sibling background process (not inside the build's own process tree), reads `/proc/meminfo` + `/proc/loadavg` + `/proc/stat` + `/proc/diskstats` (or platform equivalents on non-Linux) at a fixed interval (recommended 5 s default — fine-grained enough to catch seconds-long peaks; coarse enough to keep itself <5% CPU and <50 MB RSS). Samples written as TSV.

2. Compute per-build summaries on stop. Per metric (memory used, CPU%, load average, disk read/write): **min, max, mean, p95**. p95 is the 95th-percentile sample value — required because mean alone hides extended-peak behaviour (a 2 h build with one 33 GB soong_build peak has mean ≈ 18 GB but p95 ≈ 32 GB; the latter is what operators must size for).

3. Append per-build summaries to a registry. One TSV row per build (build_id, status SUCCESS/FAIL/UNKNOWN, start/end timestamps, per-metric min/max/mean/p95, total disk read/write). The registry is the single source of truth — the Markdown report is derived.

4. Maintain ever-values across ALL builds. Min / max / mean across every tracked build, computed on every Markdown regeneration from the registry. Surface at the **top** of the report per User mandate verbatim.

5. Markdown + HTML + PDF triple. Stats.md (auto-generated), exported to Stats.html + Stats.pdf through the project's normal export pipeline. Triple stays in sync per §11.4.12. Committed via the project's lightweight doc-sync wrapper per §11.4.22.

6. Sort per-build entries by recency (most recent first) AND, where applicable, **group by version tag.** Operators reading the report want the latest build's profile at the top and the historical sweep underneath.

7. Track status per build. SUCCESS / FAIL / UNKNOWN — and if FAIL, capture the reason (exit code, OOM, ANR, etc.). A FAIL build's resource profile is the most operationally interesting one in the report — never silently drop FAIL entries.

8. Performance budget for the sampler itself. <50 MB RSS, <5% CPU. The monitor MUST NOT itself contribute to the resource pressure it is measuring (Heisenberg-class observer effect at scale). Pure /proc reads + awk arithmetic is the reference implementation; vmstat / pidstat acceptable; psrecord / heavy-Python-import samplers are forbidden.

9. Survive build failures. The stop hook MUST be called from both the success AND failure paths of the build wrapper so FAIL builds still produce a row. Hooking only the success path is a PASS-bluff at the telemetry layer.

Pre-build gate (recommended, per consuming project):

- **`CM-BUILD-RESOURCE-STATS-TRACKER`** — verifies (a) the monitor script exists + executable, (b) the Stats.md target file exists,
    1. the build wrapper invokes start AND stop, (d) the doc-sync wrapper enumerates the Stats.{md,html,pdf} triple, (e) the exporter advertises the build-stats slug. Paired mutation hides the monitor script aside → gate FAILs.

Propagation. Composes with §11.4.12 (export-sync invariant — HTML + PDF in sync with Markdown), §11.4.18 (script-documentation discipline — the monitor ships with an external user guide), §11.4.22 (lightweight doc-sync commit wrapper enumerates the Stats.{md,html,pdf} triple), §12.6 / §12.7 / §12.9 (host-safety + containerized-build envelope whose forensic anchors are the empirical motivation for this telemetry).

No escape hatch. Build-resource tracking is not optional polish — it is the operational-evidence seam that converts post-mortem "the build OOM'd, no idea why" into "build X at git-SHA Y peaked at Z GB at minute N, which is W% above the rolling p95". The discipline pays for itself the first time it diagnoses a build-resource regression.

§11.4.25 — Full-Automation-Coverage Mandate (User mandate, 2026-05-15)

Forensic anchor — verbatim user mandate (2026-05-15):

> "Make sure that every feature, every functionality, every flow, every use case, every edge case, every service or application, on every platform we support is covered with full automation tests which will confirm anti-bluff policy and provide the proof of fully working capabilities, working implementation as expected, no issues, no bugs, fully documented, tests covered! Nothing less than this does not give us a chance to deliver stable product! This is mandatory constraint which MUST BE respected without ignoring, skipping, slacking or forgetting it!"

Operative rule. For every consuming project under this Constitution, no feature, functionality, flow, use case, edge case, service, or application on any supported platform may be considered **deliverable** until it is covered by automation tests that collectively prove six invariants:

1. **Anti-bluff posture (per §7.1 + §11.4):** every assertion carries captured runtime evidence; metadata-only / configuration-only / grep-based / absence-of-error PASSes are forbidden.
2. **Proof of working capability:** the user-visible behaviour is exercised end-to-end on the target platform topology (per §11.4.3), not in a mock or in-memory facsimile.
3. **Working implementation as expected:** assertions match the product's documented promise (in user manual, README, specs), not an implementation-detail back-door.
4. **No issues, no bugs:** the test suite is the canonical seam for surfacing defects — passing without producing defect signals means defects do not exist or have been previously surfaced, tracked (per §11.4.15 / §11.4.16), and closed.
5. **Fully documented:** the feature has a user-facing doc entry (per §11.4.18 for scripts, and project-level user-manual coverage for everything else); the doc is kept in sync with the tests (per §11.4.12).
6. **Tests-covered (the four-layer floor per §1):** pre-build presence-of-change gate, post-build artifact-shipped gate, runtime / integration / on-device gate, AND meta-test paired mutation that proves the runtime gate catches the break.

Cross-cutting reach. This mandate is **universal** — it applies to every project consuming this Constitution, every feature surface they expose (HTTP endpoints, CLI commands, slash commands, IPC channels, plugins, hooks, MCP servers, agents, providers, integrations, GUI flows, mobile flows, scheduled jobs), and every supported platform (Linux, macOS, Windows, iOS, Android, AuroraOS, HarmonyOS, embedded, containers, headless servers, kiosk, etc.). A project that ships a feature without satisfying all six invariants is **not delivering a stable product**, irrespective of how green its summary line looks.

Coverage audit. Consuming projects MUST publish a coverage ledger (matrix of: feature × platform × invariant-1..6 × status) that is regenerated as part of the release-gate sweep. The ledger itself is documented (per §11.4.18 / §11.4.12) and committed via the §11.4.22 lightweight doc-sync wrapper. Gaps in the ledger (`UNCONFIRMED:` / `PENDING:` / `BLOCKED:` cells) MUST cite a tracked work item per §11.4.15 + §11.4.16; rows that quietly omit a platform are §11.4.25 violations.

Composition. §11.4.25 explicitly stacks on top of §1 (four-layer test-coverage floor), §1.1 (false-positive immunity), §7.1 (positive-evidence-only validation), §11.4.1 (FAIL-bluffs forbidden), §11.4.2 (recorded-evidence requirement), §11.4.3 (per-environment-topology dispatch), §11.4.6 (no-guessing), §11.4.15 / §11.4.16 (status + type tracking), §11.4.17 (universal-vs-project classification — this rule is

universal), §11.4.18 (script documentation), §11.4.20 (subagent delegation when the work is multi-step), §11.4.22 (lightweight doc-sync). It does NOT supersede them; it forecloses the loophole "the feature works locally for me, ship it".

Classification: universal (per §11.4.17). No escape hatch. Severity-equivalent to a §11.4 PASS-bluff at the release-gate layer — a project claiming "Done" without honoring §11.4.25 is making a false claim regardless of how the work-item tracker labels the row.

§11.4.26 — Constitution-Submodule Update Workflow Mandate (User mandate, 2026-05-15)

Forensic anchor — verbatim user mandate (2026-05-15):

> "Every time we add something into our root (constitution Submodule) Constitution, CLAUDE.MD and AGENTS.MD we MUST FIRST fetch and pull all new changes / work from constitution Submodule first! All changes we apply MUST BE commited and pushed to all constitution Submodule upstreams! In case of conflict, IT MUST BE carefully resolved! Nothing can be broken, made faulty, corrupted or unusable! After merging full validation and verification MUST BE done!"

Operative rule. Before ANY agent or operator modifies the `constitution/Constitution.md`, `constitution/CLAUDE.md`, or `constitution/AGENTS.md` files of a project that consumes this Constitution as a submodule, the agent or operator MUST execute the following pipeline in order, with NO step skipped:

1. **Fetch + pull first.** From inside the `constitution/` submodule worktree, run `git fetch <every-configured-remote>` followed by `git pull --ff-only origin <branch>` (or `--rebase` if non-FF-mergeable; **never** `--strategy=ours` / `--allow-unrelated-histories` without explicit operator authorization). The submodule MUST be at upstream tip BEFORE any local edit is applied.
2. **Apply the change.** Edit the relevant file(s). The edit MUST classify itself per §11.4.17 (universal vs project-specific) — only universal additions belong in the constitution submodule; project-specific clauses belong in the consuming project's own governance files. The edit MUST include the verbatim user mandate (if it originated from one) as a forensic anchor.
3. **Validate before commit.** Run the constitution submodule's `meta_test_inheritance.sh` (or equivalent) to confirm the inheritance chain still resolves. Verify no governance file was left with merge-conflict markers (`<<<<<<<`, `=======`, `>>>>>>>`). Verify all three governance files (Constitution + CLAUDE + AGENTS) cross-reference the new clause consistently.
4. **Commit + push to ALL upstreams.** Stage only the governance files (NEVER `git add -A` inside the constitution submodule — stray local artefacts MUST NOT enter governance). Commit with a message that cites the user mandate (verbatim quote) + the classification line per §11.4.17. Push to **every** configured remote of the constitution submodule. A commit that lives on one upstream but not others is a §11.4.26 violation equivalent to a §2.1 multi-upstream-push violation.
5. **Conflict resolution.** If `pull --ff-only` reports non-fast-forward, the merge MUST be performed carefully: inspect both sides, preserve the union of governance content (no clause silently dropped), re-classify per §11.4.17, validate per step 3. Force-push to "make conflicts go away" is FORBIDDEN

(§9.2). Nothing about the constitution may be broken, made faulty, corrupted, or rendered unusable by the merge.
6. **Post-merge validation + verification.** After the push lands, re-clone (or `git submodule update --remote --init`) in a throwaway worktree and re-run the consumer project's inheritance-cascade verifier (e.g. `scripts/verify-governance- cascade.sh`) to confirm the new clause reaches every owned submodule per CONST-047. Any cascade gap MUST be closed in the same change-window.
7. **Update the consuming project's pointer.** The consuming project's `.gitmodules`-tracked submodule pointer MUST be bumped to the new constitution HEAD in the SAME commit as any downstream cascade work; out-of-sync submodule pointers are a §11.4.26 violation.

Operational scope. This workflow applies regardless of who initiates the change (operator, primary agent, subagent per §11.4.20, automated lint suggestion). The workflow CANNOT be shortcut by "I'll fetch later" or "I'll push to the other upstreams in the next commit". The constitution is the **single source of truth** for every project that imports it; allowing it to fragment across upstreams is the structural equivalent of a §11.4 PASS-bluff at the governance layer.

Cross-cutting reach. §11.4.26 composes with: §2 (single- entrypoint commit wrapper), §2.1 (multi-upstream push), §3 (submodule changes propagate through submodule commits first), §9.1 / §9.2 (data-safety + force-push authorization), §11.4.17 (universal-vs-project classification), §11.4.22 (lightweight doc-sync), §11.4.25 (full-automation coverage — the post-merge validation step is itself a form of automation coverage). It does NOT supersede them.

Classification: universal (per §11.4.17). No escape hatch. A constitution-submodule change that violates §11.4.26 is a release blocker for every consuming project, equivalent in severity to a force-push without §9.2 authorization.

---

§11.4.31 — Submodule-Dependency-Manifest Mandate (User mandate, 2026-05-15)

Forensic anchor — verbatim user mandate (2026-05-15):

> "We MUST HAVE mechanism for each Submodule to determine / know what are its Submodule dependencies so new projects or palces we are incorporate them can add these Submodules to the project root and make them available! Suggested idea is configuration file with expected Submodules Git ssh urls perhaps? New project can read it, and recursively add each Submodule to the root of the project and install / expose it to veryone. This MUST be analyzed and applied. We MUST apply the best strategy for this which can be easily executed just by following our root Constitution, AGENTS.MD and CLAUDE.MD! Process this, extend out Constitution, AGENTS.MD and CLAUDE.MD with mandatory instructions and then process project root and all Submodules deep recursively so proper configuration Submodules dependency files are created! Document EVERYTHING and cover with all supported test types! Any kind of bluff is strictyl forbidden! All wotk MUST come with mechanism for validation and verification by creating proper proofs for all critical points!"

Operative rule. Every owned-by-us submodule MUST ship a machine-readable, version-controlled **dependency manifest** at the canonical path `helix-deps.yaml` (or `helix-deps.json` / `helix-deps.toml` if the submodule's ecosystem strongly prefers a different serialisation — but only one canonical file per submodule, and its name MUST appear in the submodule's `README.md` + governance trio).

Manifest schema (CONST-051(C)-aligned):

```yaml
# helix-deps.yaml
# Declares this submodule's own-org dependencies. Consuming
        projects
# read this manifest recursively and add each declared dependency
        to
# their root at <root>/<name>/ or <root>/submodules/<name>/ per
# CONST-051(C). Nested own-org submodule chains are FORBIDDEN —
        this
# manifest is the bridge.

schema_version: 1
deps:
  - name: Challenges                                    # canonical
        name (will become snake_case per §11.4.29)
    ssh_url: git@github.com:vasic-digital/Challenges.git
    ref: main                                           # branch or
        pinned tag
    why: "Cross-cutting Challenges + Panoptic browser harness"
    layout: flat                                        # 'flat' =
        <root>/<name>/; 'grouped' = <root>/submodules/<name>/
  - name: HelixQA
    ssh_url: git@github.com:HelixDevelopment/HelixQA.git
    ref: main
    why: "Anti-bluff QA orchestration + autonomous-session driver"
    layout: flat

transitive_handling:
  # Each declared dep itself ships a helix-deps.yaml. The
        incorporator
  # tooling MUST recurse — top-level project gets the union of all
  # transitively-declared deps, flattened to root.
  recursive: true

  # When two submodules declare the same dependency at different
  # refs, CONFLICT — operator MUST resolve before incorporation
  # proceeds. The incorporator aborts on conflict (never silently
  # picks one).
  conflict_resolution: operator-required

language_specific_subtree: false      # set true for Android/
        Kotlin/Apple
                                       # roots (per §11.4.29
        exception);
                                       # excludes inner subtree
        from
                                       # snake_case enforcement.
```

Tooling contract. A consuming project MUST be able to bootstrap its entire own-org dependency graph by:

1. Running `incorporate-submodule <ssh-url>` (canonical name; lives in the constitution submodule's `scripts/` or in a parent project's bin path).
2. The script:
   - Adds the supplied submodule at its declared canonical path (per CONST-051(C) `flat` vs `grouped` layout).
   - Reads the newly-added submodule's `helix-deps.yaml`.
   - For each declared dep, checks if it already exists at the consuming project's root; if missing, recurses (incorporate- submodule on the dep's ssh_url).
   - On conflict (same name declared at different ref by two submodules), aborts with a directed error pointing the operator at the conflicting declarations.
   - Emits a final manifest-of-manifests file at `<root>/.helix-manifest.yaml` listing every submodule + its declared deps, for audit + reproducibility.

Anti-bluff guarantee. Every manifest MUST be paired with a **verification proof**: a Challenge script (per §11.4.27 + CONST-050(B)) that:

- Bootstraps a throwaway consuming project from scratch in a temp directory.
- Runs `incorporate-submodule` against the manifest under test.
- Verifies the produced submodule layout matches the manifest's declarations (every dep present at its declared layout path; no extras; no missing).
- Runs the submodule's own test suite against the bootstrapped layout; asserts pass.
- Captures wire evidence (per §11.4.2) of every step.

A manifest without this verification proof is a §11.4.31 violation of equal severity to a §11.4 PASS-bluff at the dependency-graph layer.

Cascade requirement. Every owned-by-us submodule MUST ship `helix-deps.yaml` at its root, recursively (sub-submodules of submodules ship their own). The constitution submodule itself ships a manifest declaring its own deps (currently empty for the universal Constitution; project consumers may have project-specific extensions). The verifier (`scripts/verify-governance-cascade.sh` or its successor) MUST check every owned submodule for manifest presence.

Composition. §11.4.31 directly enables §11.4.28 / CONST-051(C) flat-layout enforcement: nested own-org submodule chains can be mechanically flattened because each submodule declares what it needs, and the incorporator places those deps at the root. Composes with §1 (manifest schema is itself tested for parse + validation), §3 (submodule-first commit discipline applies to manifest changes too), §11.4.12 (manifest changes regenerate any derived docs/diagrams), §11.4.17 (universal — no project-specific assumptions in the manifest format), §11.4.18 (manifest documented in script-doc + external user guide), §11.4.20 (subagent delegation for cross-submodule manifest authoring), §11.4.25 (manifest presence in coverage ledger), §11.4.26 (constitution-update workflow when extending manifest schema), §11.4.27 (manifest test matrix), §11.4.28 (this rule is its operational complement), §11.4.29 (manifests use snake_case names + canonical paths), §11.4.30 (manifest is tracked source — not a build artefact), CONST-047 (manifests cascade recursively).

Classification: universal (per §11.4.17). No escape hatch. Severity-equivalent to a §11.4 PASS-bluff at the dependency-graph layer.

§11.4.32 — Post-Constitution-Pull Validation Mandate (User mandate, 2026-05-15)

Forensic anchor — verbatim user mandate (2026-05-15):

> "Every time we fetch and pull new changes on constitution Submodule we MUST process the whole project and all Submodule (deep recursively) for validation and verification taht every single rule or mandatory constraint is followed and respected! If it is not, IT MUST BE!"

Operative rule. Whenever a consuming project's constitution submodule is fetched + pulled with **any** content change (rule addition, rule revision, gate addition, anchor reference, version bump), the consuming project MUST execute a full-project + recursive-submodule validation sweep BEFORE the new constitution HEAD is treated as canonical for any other work.

Validation sweep contract. The sweep is implemented as `scripts/verify-all-constitution-rules.sh` (canonical name) which:

1. Re-runs the existing governance-cascade verifier (`scripts/ verify-governance-cascade.sh`) covering every §11.9 + CONST-* anchor across every owned submodule (recursive per CONST-047).
2. For each rule whose enforcement gate is implementable programmatically (e.g., CONST-053 `.gitignore`-pattern audit; CONST-051(C) nested-own-org-chain audit; CONST-052 case-conformance audit; CONST-050(A) mock-path-from-production- code audit; CONST-035 anti-bluff smoke-scan), the sweep runs the corresponding gate against the post-pull tree.
3. Any failure produces a directed FAIL entry naming the rule (e.g., `FAIL: CONST-053 — *.log tracked at HelixCode/foo.log`)
    - the canonical fix.
4. Failures populate the project's Issues tracker per §11.4.15 with Status: `Reopened` (since a previously-passing audit now fails) and Type: `Bug` (since real codebase state violates the constitution).
5. The agent or operator MUST resolve every FAIL before treating the new constitution HEAD as canonical — closure of each reopened item per the existing §11.4 anti-bluff covenant (positive-evidence-only, captured wire evidence).

Pull-time invocation. `git submodule update --remote constitution` MUST trigger the sweep automatically (post-update hook OR the operator's commit wrapper invokes it as part of any commit that advances the constitution submodule pointer). Operator-explicit manual invocation MUST also be available (`./scripts/verify-all-constitution-rules.sh`).

Anti-bluff guarantee. A sweep that exits PASS without actually running every implementable gate is a §11.4.32 violation. The sweep's own meta-test (paired mutation, §1.1) MUST plant a known violation of each enforced gate and assert the sweep reports FAIL for the planted gate.

Composition. §11.4.32 is the **enforcement engine** for every other §11.4.x and CONST-NNN rule. Without it, new rules cascade as anchors but never get enforced in the codebase. Composes with every rule that has a programmatic gate: §1, §1.1, §11.4.10, §11.4.12, §11.4.15, §11.4.16, §11.4.18, §11.4.20, §11.4.22, §11.4.24, §11.4.25, §11.4.26, §11.4.27, §11.4.28, §11.4.29, §11.4.30, §11.4.31, CONST-035,

CONST-038, CONST-042, CONST-043, CONST-044, CONST-045, CONST-046, CONST-047, CONST-048, CONST-049, CONST-050, CONST-051, CONST-052, CONST-053, CONST-054.

Classification: universal (per §11.4.17). No escape hatch. Severity-equivalent to a §11.4 PASS-bluff at the constitutional- enforcement layer — without §11.4.32, every other rule is a decorative anchor rather than an enforced gate.

§11.4.30 — .gitignore + No-Versioned-Build-Artifacts Mandate (User mandate, 2026-05-15)

Forensic anchor — verbatim user mandate (2026-05-15):

> "every project module, every Submodule, every servcie and apolication MUST HAVE proper .gitignore file! We MUST NOT git version build artifacts, cache files, tmp files, main .env file(s) or any files containing sensitive data, API keys or token! Any build derivate which we can recreate by executing proper mechanism for generating MUST NOT be versioned! We MUST pay attention what is going to be commited every time we are preparing to execute commit! If any violetion is detected it MUST be fixed before commit is executed!"

Operative rule. Every project module, owned-by-us submodule, service, and application under this Constitution MUST ship a proper `.gitignore` covering its full set of forbidden-from-version- control patterns. The following file/directory classes MUST NEVER appear in version control:

1. **Build artefacts** — anything produced by a build / compile / package step that can be regenerated from sources:
   - Binaries (`/bin/`, `/build/`, `/dist/`, `/out/`, `target/`, `*.exe`, `*.dll`, `*.so`, `*.dylib`, `*.a`, `*.o`, `*.class`, `*.pyc`).
   - Generated source files where the generator is committed (e.g. protobuf `.pb.go` may be checked in OR ignored — project decides — but never both).
   - Bundled assets where the bundler is committed.
2. **Cache files** — anything a tool regenerates on demand:
   - `__pycache__/`, `.pytest_cache/`, `.mypy_cache/`, `.ruff_cache/`, `node_modules/`, `.next/`, `.nuxt/`, `.cache/`, `.gradle/`, `.idea/cache/`, `.vscode-test/`, `target/`, `.terraform/`, language-server caches.
3. **Temp files** — `*.tmp`, `*.swp`, `*.swo`, `*~`, `.DS_Store`, `Thumbs.db`, IDE backup files, `*.orig`, `*.rej`.
4. **Sensitive-data files** — anything containing credentials, tokens, keys, or personal data:
   - `.env`, `.env.*` (allow `.env.example` / `.env.sample` / `.env.template` with placeholder values only — never real secrets even as examples).
   - `*.pem`, `*.key`, `*.crt`, `*.p12`, `*.pfx`, `id_rsa*`, `id_ed25519*`, `*.kdbx`, `.netrc`, `secrets/` directory trees.

    - `api_keys.sh`, any file containing `BEGIN PRIVATE KEY`, credential tokens, or session cookies.

5. **Generated reports / logs** — `*.log`, `coverage.out`, `*.coverage`, `htmlcov/`, `*.gcda`, `*.gcno`, screenshots/ recordings that aren't reference assets.

6. **OS / IDE / personal-state files** — `.idea/`, `.vscode/` (except shareable `.vscode/settings.json` etc. if explicitly project-shared), `.history/`, `.svn/`, editor-state files.

7. **Test playback artefacts** (§11.4.14) — every test's runtime capture goes under an ignored evidence directory unless the capture IS the reference asset.

Anti-bluff invariant. Per CONST-035 / §11.4 the presence of a `.gitignore` line alone is not sufficient — the project MUST also verify that no file matching the forbidden patterns is currently tracked. A `.gitignore` that lists `*.log` while `app.log` sits tracked in the repo is a §11.4.30 violation of equal severity to no `.gitignore` at all.

Pre-commit attention. Every commit author (human OR agent) MUST inspect `git diff --staged` and `git status` BEFORE executing the commit. If a staged path matches any forbidden class, the commit MUST be aborted and the issue fixed (un-stage the path, add to `.gitignore`, scrub if already-tracked). Gate `CM-GITIGNORE-PRECOMMIT-AUDIT`: pre-commit hook (or commit wrapper in §2) inspects every staged path against the forbidden-pattern matrix; hits abort the commit with a directed error pointing the operator at the specific offending path and the canonical fix. Paired mutation (§1.1): stage a `*.log` → gate FAILs.

Cascade reach. This rule applies recursively to every owned-by- us submodule per CONST-047 and to every owned-submodule's nested non-owned trees within their working copy. The `.gitignore` files themselves are project-specific in their concrete patterns but universally bound to the rule's normative force.

Secret-leak intersection. §11.4.30 composes tightly with §11.4.10 (Credentials-handling mandate / CONST-042) and §12.1. A `.env` leak is BOTH a §11.4.30 violation (build/sensitive file versioned) AND a §11.4.10 violation (credential leak). The combined severity is **release-blocker requiring rotation + post-mortem** per §11.4.10.

Coverage of "recreatable" content. When in doubt about whether something is a build derivative: if there exists a documented, scripted, or automated mechanism that recreates the file from sources, the file is a build derivative and MUST be ignored. The committed sources MUST include the generator (Makefile target, npm script, codegen invocation), so any downstream consumer regenerates the artefact on demand.

Classification: universal (per §11.4.17). No escape hatch beyond the explicit exceptions enumerated above (e.g., `.env.example` placeholder files). Severity-equivalent to a §11.4 PASS-bluff at the repository-hygiene layer — a tracked build artefact silently drifts vs. its source over time, producing the same class of "works-on-my-machine" surprise that the §11.4 anti-bluff covenant forbids at the test layer.

Composition. §11.4.30 composes with §1 (test coverage — ignored files don't get spuriously included in coverage stats), §2 (commit-wrapper enforces the pre-commit gate), §9.1 (destructive-operation safeguards — never `git clean` an operator's

working tree to “fix” a violation), §11.4.10 (credentials-handling), §11.4.12 (auto-generated docs sync — generators committed, outputs ignored only if recreatable on demand from the same generator), §11.4.17 (universal — applies to every consuming project), §11.4.18 (script docs — generator scripts documented), §11.4.20 (subagent delegation for cross-submodule .gitignore audit sweeps), §11.4.25 (coverage ledger lists `.gitignore` presence per submodule), §11.4.26 (constitution-update workflow — the constitution submodule’s own `.gitignore` complies), §11.4.27 (no-fakes: test fixtures that ARE the reference asset are tracked; runtime captures are ignored), §11.4.28 (submodules-as-equal-codebase: each submodule’s `.gitignore` audited on equal basis), §11.4.29 (the renamed paths in the lowercase migration MUST also be properly ignored where applicable), CONST-047 (recursive governance reach).

§11.4.29 — Lowercase-Snake_Case-Naming Mandate (User mandate, 2026-05-15)

Forensic anchor — verbatim user mandate (2026-05-15):

> “naming convention for Submodules and directories (applied deep into hierarchy recursively) - all directories and Submodules MSUT HAVE lowercase names with space separator between the words of ’_’ character (snake-case)! All existing Submodules and directories which are not following this rule MUST BE renamed! However, since this will most likely break some of the functionalities renaming we do MUST BE applied to all references to particular Submodule or directory! Everywhere where particular Submodule directory are referenced proper updates MUST BE applied - all configuration files, documentation and relevant materials, links to Submodules and directories, source code that points to them, etc. There MUST BE reasonable exceptions for this rules - source code for programming languages or Submodules which apply different naming convention - Android, Java, Kotlin and others. Root directory for such applications, services or Submdoules can follow OUR convention, but EVERYTHING inside still MUST follow language / technology specific rules! We apply this rules per common sense basis and it MUST NOT be the cause of bigger issues such as technology breaking! Upstreams directory which all of our projects and Submodules have MUST BE renamed to the lowercase letters too, however root project containing the install_upstreams system command (it is exported in out paths in our .bashrc or .zshrc) MUST BE updated to fully work with both Upstreams and upstreams directory. That change if it is not already applied MUST BE done, commited and pushed! … NOTE: Rules lowercase / snake-case do apply to all project files as well and references to it and from them! Every change done MUST BE covered with all supported test types, full automation tests for validation and verififcation and fully applied and followed anti-bluff policy and all anti-bluff rules!”

Operative rule. Every directory, submodule, and file under the parent project’s working tree MUST use a **lowercase, snake_case** name (ASCII letters / digits / underscores, words separated by _). Existing names that violate the rule (`HelixCode/`, `Challenges/`, `Containers/`, `HelixAgent/`, `HelixQA/`, `Security/`, `Github-Pages-Website/`, `Upstreams/`, `Dependencies/`, etc.) MUST be renamed as part of the migration window opened by this clause. Every reference in the codebase MUST be updated atomically with the rename: configuration files,

documentation, user manuals, diagrams, scripts, source-code imports, links, governance files. **Reference drift after a rename is a §11.4.29 violation** of equal severity to the rename itself.

Exceptions (common-sense scope). The rule MUST NOT break language-/ technology-specific conventions:

- **Programming-language source roots** that mandate a specific case (Java / Kotlin package paths, Android resource folders, Apple framework directories, C# / Swift project layouts) keep their language-mandated names. The submodule's root directory follows our convention; the language-specific subtree inside follows its own.
- **Vendor / upstream submodules** (third-party orgs not in our owned set) keep their upstream-mandated names — we MUST NOT rename a third party's repo.
- **Build-tooling artefacts** (`node_modules/`, `__pycache__/`, `.git/`, `target/`, `build/`, `bin/`) keep their tool-mandated names.

When in doubt, the test "does renaming break the technology?" trumps the snake_case rule. The §11.4.29 spirit is operator- ergonomics + reference-discoverability, not pedantic uniformity at the cost of technology compatibility.

`Upstreams/` → `upstreams/` transition. The constitution submodule's installer (`install_upstreams.sh`) — exported on operator paths via `.bashrc` / `.zshrc` — MUST support **both** `Upstreams/` and `upstreams/` directory layouts during the migration window, so existing checkouts keep working while consuming projects rename at their own pace. The installer reads whichever directory exists; if both exist the lowercase wins. After every project under this Constitution has migrated, the uppercase fallback MAY be retired by a deliberate amendment, but the migration window remains open as long as ANY owned project still ships the uppercase form.

Project-Toolkit Upstreamable submodule synchronisation. The Upstreamable / Project-Toolkit machinery that propagates governance into every consuming project MUST be fetched + pulled before any rename batch, and MUST itself comply with this rule. Any Upstreamable submodule lacking BOTH-directory support is a release blocker for the rename program.

Test coverage of renames. Every batch of renames MUST ship with: (i) a regression test that verifies every reference to the renamed entity now resolves to the new name (no stale references left); (ii) a full CONST-050(B) test-type matrix run against the post-rename tree; (iii) anti-bluff (CONST-035) wire-evidence captured during the runtime verification. A rename batch without all three is a §11.4.29 violation.

Cascade reach. This rule applies recursively through every owned-by-us submodule layer (per CONST-047) and applies equally to the consuming project and its owned submodules (per CONST-051). Per CONST-051(C) — dependencies at the parent root — the renamed paths MUST be the only canonical location; old-name aliases remain only as transitional symlinks if absolutely required, and those symlinks MUST be removed on next-N-cycle review (project- configurable, recommended ≥3 release cycles).

Phased execution. Because the rename touches every reference, the execution MUST be planned as fine-grained phases per the operator's explicit instruction: comprehensive brainstorming, phase-divided plan, fine-grained tasks/subtasks with

enormous detail, every change covered by every applicable test type. The phases run in parallel with mainstream work (§11.4.20 subagent delegation is the natural fit for cross-submodule rename sweeps).

Classification: universal (per §11.4.17). No escape hatch beyond the explicit common-sense exceptions enumerated above. Severity-equivalent to a §11.4 PASS-bluff at the reference-integrity layer — a half-completed rename that leaves broken references is worse than no rename at all because it silently breaks consumers.

Composition. §11.4.29 composes with §1 (four-layer floor for every rename batch), §1.1 (paired mutation: rename without updating a reference → gate FAILs), §11.4.12 (regenerate auto- generated docs on rename), §11.4.17 (universal — no project- specific assumptions), §11.4.18 (script-doc-sync after renamed script paths), §11.4.20 (subagent delegation for cross-cutting rename sweeps), §11.4.25 (every renamed path appears in coverage ledger), §11.4.26 (constitution-submodule rename pipeline), §11.4.27 (rename-touch test types apply across the matrix), §11.4.28 (owned submodule renames cascade per CONST-047), CONST-047 (recursive governance reach).

§11.4.28 — Submodules-As-Equal-Codebase + Decoupling + Dependency-Layout Mandate (User mandate, 2026-05-15)

Forensic anchor — verbatim user mandate (2026-05-15):

> "All existing Submodules in the project that we are controlling and belong to some our organizations (vasic-digital, HelixDevelopment, red-elf, ATMOSphere1234321, Bear-Suite, BoatOS123456, Helix-Flow, Helix-Track, Server-Factory — we can ALWAYS check dynamically using GitHub and GitLab CLIs) are equal parts of the project's codebase! We MUST work on that code as much as we do with main project's codebase! All on equal basis! Equally important! We MUST take it into the account, analyze it, extend it, create missing tests, do full testing of it, fill the gaps (if any), fix any issues that we discover or they pop-up, write and extend the documentation, user guides, manulas, diagrams, graphs, SQL definitions, Website(s) and all other relevant materials! We MUST NEVER modify Submodules to bring into them any project specific context since they all MUST BE ALWAYS fully decoupled, project not-aware, fully reusable and modular (by any other project(s)), completely testable! All Submodule dependencies that are used by Submodule MUST BE acessed from the root of the project! We MUST NOT have nested Submodule dependencies but accessing each from proper location from the root of the project — directly from project's root project_name/ submodule_name or some more proper structure project_name/ submodules/submodule_name! This MUST BE heavily enforced and respected so no chaos and mess is created with various dependencies we may have!"

Operative rule. Three cooperating invariants govern every consuming project's relationship with its owned-by-us submodules (those whose upstream `origin` lives under one of the operator- listed orgs — `vasic-digital`, `HelixDevelopment`, `red-elf`, `ATMOSphere1234321`, `Bear-Suite`, `BoatOS123456`, `Helix-Flow`, `Helix-Track`, `Server-Factory` — or any additional org the operator subsequently authorises, the canonical list discoverable at any time via `gh org list` / `glab` / the orgs' public APIs):

(A) Equal-codebase invariant. Every owned-by-us submodule is an **equal part** of the consuming project's codebase. The consuming project's engineering practice — analysis, extension, test creation, gap-filling, bug-fix, documentation (user manual, guides, diagrams, graphs, SQL definitions, website pages, any other authoring surface) — applies to each owned submodule on equal basis. A round of work that improves only the main project while leaving an owned-submodule deficiency unaddressed is a §11.4.28 violation, severity-equivalent to a §11.4 PASS-bluff at the project-scope layer. Coverage ledgers (§11.4.25) MUST list every owned submodule as an in-scope target. CONST-047 (recursive cascade) governs the propagation of governance changes; §11.4.28 is the engineering-content counterpart that mandates the same attention to non-governance content.

(B) Decoupling / reusability invariant. Owned submodules MUST remain **fully decoupled** from any specific consuming project. No project-specific context, hardcoded paths, hostnames, asset names, naming schemes, or runtime assumptions may be introduced into an owned submodule's source tree. Every owned submodule MUST be:

- **Project-not-aware** — its code, tests, and docs make no reference to which parent project consumes it.
- **Fully reusable** — any future Helix-family or third-party project must be able to consume the submodule unmodified.
- **Modular** — its public surface is the only documented integration contract; internal layout may evolve without breaking consumers.
- **Completely testable** — every public surface has standalone tests (per the §11.4.27 100%-test-type matrix) that pass when the submodule is checked out as a standalone repo, without any parent-project rigging.

A commit that adds `project_name/...` strings, hostnames belonging to a specific deployment, or other parent-project context to a submodule's source tree is a §11.4.28 violation. The honest path when a submodule needs information from the parent project is configuration injection (env var, config file, constructor parameter) — never a hardcoded reach into the parent's tree.

(C) Dependency-layout invariant. Every dependency that an owned submodule itself consumes MUST be accessible **from the root of the parent project** at one of two canonical paths:

```
<project_root>/<submodule_name>/          # flat layout
<project_root>/submodules/<submodule_name>/   # grouped layout
```

Nested-submodule chains are FORBIDDEN. A submodule MUST NOT have its own `.gitmodules` entries that pull in further owned- by-us repos (transitive own-org submodule recursion). Every dependency required by submodule X MUST be added to the parent project at the canonical path above; X reaches it via documented import / SDK path / runtime resolver — never via its own nested submodule pointer.

Rationale: nested own-org submodule chains cause version-drift chaos (two consumers of `LLMsVerifier` end up at different SHAs because each parent picked a different transitive path). The flat / grouped layout makes the consuming project's submodule graph a tree-of-depth-1, which any developer can audit at a glance via `git submodule status` from the project root.

Third-party submodules (not under our orgs) are exempt — they MAY appear at any depth as the upstream's structure dictates. The invariant applies only to our owned set.

Audit + enforcement.

- Gate `CM-OWNED-SUBMODULE-EQUAL-ENGINEERING` (project-side): every release-gate sweep verifies each owned submodule has current test runs, coverage entries (§11.4.25), and documentation freshness on par with the main project. Stale submodules surface as §11.4.28 violations (Status: Operator-blocked or In progress per §11.4.21 — never "ignored").
- Gate `CM-OWNED-SUBMODULE-DECOUPLING` (submodule-side): every owned submodule's pre-commit hook (or equivalent) greps the staged diff for parent-project names / hostnames / asset names. Hits abort the commit until refactored to configuration injection.
- Gate `CM-OWNED-SUBMODULE-LAYOUT` (project-side): the parent project's pre-merge sweep verifies (i) every owned submodule sits at `<root>/<name>/` or `<root>/submodules/<name>/`,
    1. no owned submodule contains a nested `.gitmodules` entry whose upstream is in our org list, and (iii) every dependency declared by an owned submodule has a corresponding parent-project submodule entry at the canonical path.
- Paired mutations (§1.1) for all three gates: plant the forbidden pattern → gate FAILs; restore → gate PASSes.

Workflow integration. Honoring §11.4.28 in practice:

- Engineering rounds plan in submodule-aware tranches: "improve feature X in main; in same round, audit + improve the X- related surface in submodule Y; cascade governance per CONST-047 as usual".
- The §11.4.25 coverage ledger row format is extended with a `submodule` column so coverage rolls up across the whole owned set.
- Cross-submodule refactors that touch shared types or interfaces ship as a single change-window: parent + every affected owned submodule advanced in lockstep (§3 submodule- first-commit discipline + §2.1 multi-upstream push).

Classification: universal (per §11.4.17). No escape hatch. Composes with: §1 (four-layer test floor reaches submodules too), §1.1 (false-positive immunity), §3 (submodule changes propagate through submodule commits first), §11.4.17 (universal-vs-project classification), §11.4.20 (subagent delegation for the cross-submodule audit sweep), §11.4.25 (full-automation coverage ledger expanded to submodules), §11.4.26 (constitution-update workflow's submodule-pointer-bump step), §11.4.27 (100%-test- type coverage applies to every submodule's standalone surface), CONST-047 (recursive governance cascade). A round of work that violates §11.4.28 is a release blocker for the consuming project, severity-equivalent to a §11.4 PASS-bluff at the codebase-completeness layer.

§11.4.27 — No-Fakes-Beyond-Unit-Tests + 100%-Test-Type-Coverage Mandate (User mandate, 2026-05-15)

Forensic anchor — verbatim user mandate (2026-05-15):

> "Mocks, stubs, placeholders, TODOs or FIXMEs are allowed to exist ONLY in Unit tests! All other test types MUST interract with real fully implemented System! No fakes, empty implementations or bluffing is allowed of any kind! All codebase of the project MUST BE 100% covered with every supported test type: unit tests, integration tests, e2e tests, full automation tests, security tests, ddos tests, scaling tests, chaos tests, stress tests, performance tests, benchmarking tests, ui tests, ux tests, Challenges (fully incorporating our Challenges Submodule — https://github.com/vasic-digital/Challenges). EVERYTHING MUST BE tested using HelixQA (fully incorporating HelixQA Submodule — https://github.com/HelixDevelopment/HelixQA). HelixQA MUST BE used with all possible written tests suites (test banks) for every applications, service, platform, etc and execution of the full HelixQA QA autonomous sessions! All required dependency Submodules MUST BE added into the project as well (fully recursive!!!)."

Operative rule. Two cooperating invariants:

(A) No-fakes-beyond-unit-tests. Mocks, stubs, fakes, placeholders, in-memory facsimile implementations, `TODO`, `FIXME`, "for now", "in production this would", or any empty-implementation pattern are PERMITTED only inside unit-test sources (e.g., `*_test.go` files invoked WITHOUT the integration build tag; `tests/unit/`; equivalent per-language conventions). Every other test type — integration, end-to-end, full automation, security, DDoS, scaling, chaos, stress, performance, benchmarking, UI, UX, Challenges, HelixQA suites — MUST exercise the **real, fully implemented system** against real infrastructure (real databases, real HTTP endpoints, real containers, real downstream services, real captured devices). A non-unit test that imports mocks, in-memory repositories, fabricated provider responses, or placeholder structs is a §11.4.27 violation regardless of how green its summary line looks — severity-equivalent to a §11.4 PASS-bluff. The same prohibition extends to **production code**: no mock import path may be reachable from any code path that runs in production binaries. Pre-build gate `CM-NO-FAKES-BEYOND-UNIT-TESTS` scans the non-unit test trees and production trees for the forbidden patterns; paired mutation plants a fake → gate FAILs.

(B) 100% test-type coverage with every supported type. Every project under this Constitution MUST cover its entire codebase with **every supported test type** the project's domain warrants:

1. **Unit** — fast, isolated, mocks permitted per (A).
2. **Integration** — multi-component, no mocks, real backing services.
3. **End-to-end (E2E)** — full user-flow exercise on target topology.
4. **Full automation** — orchestrated suites exercising every feature × platform combination (§11.4.25 coverage ledger).
5. **Security** — authn/authz boundaries, secret-leak scans (§11.4.10), input-fuzzing, dependency-CVE scanning, threat- model verification.
6. **DDoS** — request-flood resilience at advertised throughput tier, with rate-limit + back-pressure assertions.
7. **Scaling** — horizontal + vertical scale behaviour under linear load growth, including replica add/remove transitions.
8. **Chaos** — controlled failure injection (network partition, process kill, disk full, clock skew) verifying graceful degradation paths.
9. **Stress** — sustained load above advertised tier, asserting bounded resource exhaustion + clean recovery.
10. **Performance** — latency / throughput / tail-latency invariants vs SLO baselines.

11. **Benchmarking** — micro + macro benchmark suites with historical p95-drift detection (§11.4.24 build-resource composition).
12. **UI** — visual-regression + DOM-state + interaction-flow coverage on every target platform's UI surface.
13. **UX** — flow-correctness + accessibility + i18n + visual- cue ordering (§11.4.23 composition).
14. **Challenges** — `vasic-digital/Challenges` submodule fully incorporated; per-feature Challenge scripts covering real user use-cases with captured runtime evidence.
15. **HelixQA** — `HelixDevelopment/HelixQA` submodule fully incorporated; ALL written test banks executed; full autonomous QA sessions run as part of release gates.

Per-type 100% coverage means: for every feature × platform cell in the §11.4.25 coverage ledger, the cell carries a verified PASS evidence pointer for **each supported test type the cell warrants** (a CLI-only feature warrants unit + integration + E2E + full-automation + security + chaos + stress + performance + Challenges + HelixQA; a UI feature additionally warrants UI + UX; a network service additionally warrants DDoS + scaling + benchmarking). Gaps are tracked per §11.4.15 with explicit `UNCONFIRMED:` / `PENDING_FORENSICS:` / `OPERATOR-BLOCKED:` reasons.

Required submodule incorporation (recursive). Every project consuming this Constitution MUST add the following as Git submodules (or vendored equivalents authorised by the operator), fully recursively per CONST-047 / §11.4.x cascade:

- `Challenges` — `git@github.com:vasic-digital/Challenges.git`
- `HelixQA` — `git@github.com:HelixDevelopment/HelixQA.git`
- Any additional functionality submodules under `vasic-digital/*` or `HelixDevelopment/*` orgs that the consuming project depends on (do not duplicate work the organisations already maintain).

Submodule pointers MUST be bumped to upstream HEAD in the SAME commit as any dependent cascade work (§11.4.26 step 7). Pointer drift = §11.4.27 violation.

HelixQA autonomous sessions. "Full HelixQA QA autonomous sessions" means: HelixQA's orchestrator drives the consuming project's running surface end-to-end, executing every test bank the project has registered with it, capturing wire evidence per check (status + body bytes + body-head + duration + screenshots for UI), and producing a session report that is itself committed via the §11.4.22 lightweight doc-sync wrapper. A green autonomous-session report without captured wire evidence is a §11.4 PASS-bluff at the QA-orchestration layer.

Composition. §11.4.27 composes with: §1 (four-layer floor), §7.1 (positive-evidence-only), §11.4.1 (FAIL-bluffs forbidden), §11.4.2 (recorded-evidence), §11.4.3 (per-topology dispatch), §11.4.6 (no-guessing), §11.4.10 (credentials-handling — security test category), §11.4.15 / §11.4.16 (status + type tracking), §11.4.17 (universal-vs-project classification), §11.4.20 (subagent delegation for orchestrating the test-type matrix), §11.4.22 (lightweight doc-sync for coverage ledgers and session reports), §11.4.25 (full-automation-coverage — §11.4.27 is its strict expansion into per-type-of-test territory). It does NOT supersede them. CONST-047 (recursive submodule application) governs the cascade reach.

Classification: universal (per §11.4.17). No escape hatch. A project shipping with mocks reachable from non-unit-test paths OR missing required test-type coverage is **not delivering a stable product** — release blocker for every consuming project, severity-equivalent to a §11.4 PASS-bluff at the release-gate layer.

---

§11.4.33 — Type-aware closure-status vocabulary (User mandate, 2026-05-15)

Forensic anchor — direct user mandate (verbatim, 2026-05-15):

> "make sure we use proper wording for the workable item completion status in Issues, Issues_Summary and Fixed docs: - Fixed -> into the proper wording depending on the workable item type (task, feature, and so on) - for example: Fixed, Implemented, Completed. Add this important detail in our root Constitution, CLAUDE.MD and AGENTS.MD so it is ALWAYS respected and followed!"

Classification: §11.4.17-classified **universal** — naming discipline applies to every project that tracks work items by type. Bug closures aren't "implemented", feature deliveries aren't "fixed", and infrastructure refactors aren't either. Type-mismatched closure vocabulary is a quiet but persistent semantic-drift bluff at the documentation layer — it makes greppable releases lie about what shipped.

The mandate. §11.4.15 defined the lifecycle Status closed-set including the closure terminal value `Fixed (→ Fixed.md)`. §11.4.16 defined the Type closed-set `{Bug | Feature | Task}`. §11.4.33 binds the two: the closure terminal value MUST agree with the item Type, drawn from this 3-element closed map:

Item `**Type:**`	Closure `**Status:**` value
`Bug`	`Fixed (→ Fixed.md)`
`Feature`	`Implemented (→ Fixed.md)`
`Task`	`Completed (→ Fixed.md)`

The `(→ Fixed.md)` suffix is preserved across all three so the existing migration-discipline tooling (Issues.md → Fixed.md atomic move per §11.4.19) continues to work without per-Type branching. Generators (`generate_issues_summary.sh`, `generate_fixed_summary.sh`, status-counter helpers, the §11.4.23 colorizer) MUST treat the three terminal values as semantically equivalent (all map to "closed, positive evidence captured") while preserving the literal in the emitted document.

No escape hatch. Closing a `Feature` with `Fixed (→ Fixed.md)` or a `Task` with `Implemented (→ Fixed.md)` is a §11.4.33 violation. Pre-build gate (recommended, per consuming project) `CM-CLOSURE-VOCAB-TYPE-AWARE` walks every Fixed.md heading + every Issues.md heading whose `**Status:**` is one of the three terminal values, and asserts the Status value matches the item's `**Type:**` per the table. Paired mutation flips a Bug entry's status from `Fixed (→ Fixed.md)` to `Implemented (→ Fixed.md)` → gate FAILs.

Propagation. Composes with §11.4.15 (item-status tracking), §11.4.16 (item-type tracking), §11.4.19 (Fixed-document column alignment), §11.4.23 (colorisation — the closed-state palette applies regardless of which of the three terminal words is used).

§11.4.34 — Reopened-source attribution mandate (User mandate, 2026-05-15)

Forensic anchor — direct user mandate (verbatim, 2026-05-15):

> "when we reopen some workable item (bug, task or feature) we MUST HAVE details on who did reopened this and why (- Reopened status needs: by who, AI or User + details in main Issues doc.). For example, reopened by AI or reopened by real User, why - test failed, manual testing detected problem, etc. Adapt our docs for this - Issues, Isssues_SUmmary and Fixed."

Classification: §11.4.17-classified **universal** — every project that uses §11.4.15's `Reopened` lifecycle value benefits from knowing the reopen source + cause. Without this, reopen-thrash patterns (the same item bouncing between Fixed and Reopened across cycles) cannot be diagnosed, and the §11.4.7 demotion-evidence rule loses its forensic counterparty (you cannot evaluate whether a reopen was operator-side observation or agent-side over-restoration without provenance).

The mandate. Every Issues.md (or equivalent project tracker) heading whose `**Status:**` value is `Reopened` MUST carry, within 8 non-blank lines of the heading, a `**Reopened-Details:**` line that captures four sub-facts:

- **By:** `AI` or `User` (the source-of-truth observer who flipped the status). `AI` covers in-loop reopens (test failure, gate regression, captured-evidence retrospect). `User` covers operator- side observations (manual testing, end-user report, design reconsideration).
- **On:** ISO date (`YYYY-MM-DD`).
- **Reason:** one-line cause classification — chosen from a closed vocabulary `{ test-failed | manual-testing-detected | captured-evidence-contradicts | end-user-report | cycle-re-discovered | design-reconsidered }`. Other values are permitted with explicit `Reason: <free text>` annotation but the closed list MUST be tried first.
- **Evidence:** path to or short description of the captured artefact that justifies the reopen — log file, recording, gate failure ID, operator quote, etc. Reopens without evidence are §11.4.6 / §11.4.7 violations: the reopen IS a demotion-from-Fixed classification change, and demotion requires positive evidence captured under the conditions that re-exposed the defect.

The Issues_Summary.md (or equivalent) Status column MUST distinguish the four `Reopened` sub-states by source so a sweep query for "reopens by AI in the last 30 days" is mechanically possible. Suggested column rendering: `Reopened (AI: test-failed)` vs `Reopened (User: manual-testing)`.

No escape hatch. A `Reopened` entry without `**Reopened-Details:**` is a §11.4.34 violation. Pre-build gate (recommended, per consuming project) `CM-ITEM-REOPENED-DETAILS` mirrors `CM-ITEM-OPERATOR-BLOCKED-DETAILS` (Phase 39.AT pattern): walks every actionable heading, scans 8 non-blank lines for a `**Status:** Reopened` line, then continues scanning for `**Reopened-Details:**`. Missing details line emits WARN initially, hardens to FAIL once backlog is fully populated.

Propagation. Composes with §11.4.6 (no-guessing — the Reason must be drawn from the closed vocabulary or explicitly annotated; no `likely/probably/maybe` causes), §11.4.7 (demotion-evidence — reopen IS a demotion from Fixed), §11.4.15 (item-status tracking — extends the `Reopened` value's discipline), §11.4.21 (Operator- blocked discipline — same audit-line pattern with the same gate shape).

---

§11.4.35 — Canonical-root inheritance clarity (User mandate, 2026-05-15)

Forensic anchor — direct user mandate (verbatim, 2026-05-15):

> "Parent AGENTS.MD or CLAUDE.MD are located under constitution directory (Submodule) containing these parent (root) which files we are inheriting inside the project! Pay attention to this and make sure you ALWAYS follow this rule! Same applies to the root Constitution file!"
>
> "If needed (and we think it is needed it seems) add into the root Constitution, AGENTS.MD and CLAUDE.MD located in constitution directory (Submodule) which we are inheriting that the root (parent) Constitution, AGENTS.MD and CLAUDE.MD are not the ones in root of the project but inside the constitution directory (Submodule). We are inheriting it and if project specific rules have to be added or project specific constraints which are not universal and reusable, then they go into the Constitution, CLAUDE.MD and AGENTS.MD of the project directory itself!"

Classification: §11.4.17-classified **universal** — every project that consumes this constitution as a Git submodule MUST unambiguously distinguish the **canonical root** (the constitution submodule's three files) from the **consumer extensions** (the project's own three files).

The defect this anchor closes. Loose terminology — "parent CLAUDE.md", "root CLAUDE.md", "main constitution" — used without a file-path anchor causes a quiet but persistent confusion between the inheriting copy and the source-of-truth. AI agents have already been observed editing the consumer-side `CLAUDE.md` when the User's intent was to extend the canonical layer (or vice-versa), causing universal rules to leak into project-specific copies and project-specific rules to be falsely promoted as universal. The defect compounds: once a rule is misfiled, future propagation gates treat the misfile as canonical and silently spread it.

The mandate. The three files in **this constitution submodule** (`constitution/Constitution.md`, `constitution/CLAUDE.md`, `constitution/AGENTS.md`) are the **canonical root** — also called the **parent** files. They contain only universal rules per §11.4.17 — rules that any project consuming this submodule benefits from.

The three files in **the consuming project's repository root** (`<project-root>/CLAUDE.md`, `<project-root>/AGENTS.md`, optionally `<project-root>/Constitution.md` or equivalent) are the **consumer extensions**. They MUST start with the inheritance pointer per the existing constitution/CLAUDE.md "How inheritance works" section. They contain only project-specific rules per §11.4.17 — rules that reference particular hardware, vendor names, regulatory regions, internal asset names, or project-private conventions.

Operative invariants that follow:

1. **When in doubt about which file to edit:** if the rule is reusable across any project, edit the constitution submodule's file. If the rule references project-private specifics (a hardware revision, a vendor SDK version, a regional constraint, a particular service), edit the consumer's file. Default to consumer-side when uncertain (per §11.4.17 — narrower scope is cheap to widen later, the reverse is expensive).

2. **Terminology anchor.** When prose in any file in this constitutional family references "the parent CLAUDE.md" or "the root Constitution," the referent is the constitution-submodule file at `constitution/<filename>`, never the consumer's file. When it references "the project CLAUDE.md" or "this project's AGENTS.md," the referent is the consumer-side file at `<project-root>/<filename>`. AI agents reading this constitution MUST resolve ambiguous pronouns ("the CLAUDE.md", "the constitution") via this rule.

3. **Universal rules added to the consumer side are misfiled.** Per §11.4.17 commit-time check, before adding a `MUST` to the consumer file, the author assesses whether it would benefit other projects that consume this constitution. If yes, lift it to the constitution submodule first.

4. **Project-specific rules added to the constitution submodule are misfiled.** Per §11.4.17, before adding a `MUST` to the constitution submodule, the author assesses whether it references project-private specifics. If yes, demote it to the consumer file (or genericise it to a universal form first).

5. **Propagation gates target both layers but their contents differ.** The existing `CM-COVENANT-114-N-PROPAGATION` gate family verifies that anchor TEXT for §11.4.N exists in EVERY `CLAUDE.md` and `AGENTS.md` across the project. The constitution-submodule files carry the **canonical** text; the consumer files carry **compact propagation pointers** that reference the canonical authority by file path. Both forms count as valid presence; the gate is satisfied when any form is found.

6. **`@constitution/CLAUDE.md` import (Claude-Code-style) and the pointer-block fallback (Aider/Codex/Gemini-style) are equivalent.** The consumer's CLAUDE.md MUST start with one of:

   - The native import: `@constitution/CLAUDE.md`
   - The portable pointer-block per the `## INHERITED FROM constitution/CLAUDE.md` heading defined in `constitution/CLAUDE.md` "How inheritance works".

7. **No silent demotion or silent promotion.** Moving a rule between layers MUST be a visible commit — `git mv` of a section if it's a clean clone, or an explicit "Lifted from to constitution per §11.4.35" / "Demoted from constitution to per §11.4.35" line in the commit message. AI agents MUST NOT silently re-author a §11.4.X anchor in the wrong layer and call it propagation.

Pre-build gate (recommended, per consuming project):

- **`CM-CANONICAL-ROOT-CLARITY`** — verifies (a) consumer's `CLAUDE.md` opens with the inheritance pointer (either `@import` or `## INHERITED FROM constitution/CLAUDE.md` heading), (b) the constitution submodule's three

files are present at the expected path, (c) no `## INHERITED FROM` block in the constitution submodule's own files (those ARE the source-of-truth, not consumers).

Propagation. Composes with §11.4.17 (universal-vs-project classification — §11.4.35 defines the file-layer split that §11.4.17 classifies INTO), §11.4.18 (script documentation discipline — same canonical-vs-consumer file-layer rule applies to docs/scripts/). Reading order: this anchor first, then §11.4.17 for the classification rule.

No escape hatch. Misfiling a rule between layers IS a §11.4.35 violation regardless of intent. The fix is `git mv` + commit per invariant 7, not "leave both copies in place to be safe" — duplicates silently diverge over time.

§11.4.36 — Mandatory install_upstreams on clone/add Mandate (User mandate, 2026-05-15)

Forensic anchor — verbatim user mandate (2026-05-15):

> "Every Submodule or Git repository we add or clone MUST BE upstreams installed using Upstreamable utility which MUST BE available through exported paths of the host system (in .bashrc or .zhrc) using install_upstreams command executed from the root of the cloned (added) repository - only if in it is Upstreams or upstreams directory present with bash script files (recipes) for all repository's upstreams!"

Operative rule. Every time an operator or agent adds or clones a Git repository (`git clone`, `git submodule add`, `incorporate-submodule` per §11.4.31, etc.) under any consuming project of this Constitution, the post-clone procedure MUST be:

1. `cd` into the newly-cloned (or newly-added submodule's) working tree.
2. If the working tree contains an `upstreams/` directory (or legacy `Upstreams/` per the §11.4.29 transition) populated with one or more `*.sh` recipe files declaring upstream Git SSH URLs (the canonical declaration format defined by this constitution submodule's `Upstreams/*.sh` shape), the operator or agent MUST invoke `install_upstreams` from that directory.
3. `install_upstreams` is a host-system utility installed on the operator's `PATH` via `.bashrc` or `.zshrc` export. Implementation lives in the constitution submodule (`install_upstreams.sh`); operators alias / symlink it onto `PATH` once per host setup. The utility reads the recipe files, configures every declared upstream as a named git remote, and fans out `origin` push URLs across all declared upstreams.
4. If no `upstreams/` directory is present, no `install_upstreams` invocation is required.
5. If `upstreams/` is present but `install_upstreams` is not on `PATH`, the operator MUST install it (per the constitution submodule's setup docs) BEFORE making the clone usable.
6. Skipping step 2 when an `upstreams/` directory IS present is a §11.4.36 violation — the next push from that working tree will land on only one upstream, breaking §2.1 (Multi-upstream push is the norm).

Pre-commit attention. Before the first commit lands inside the newly-cloned working tree, the operator MUST verify that `install_upstreams` has been executed (when applicable). The quickest check: `git remote -v | grep -c push` reports the expected upstream count. If it reports `1` while `upstreams/*.sh` declares more, abort the commit and run `install_upstreams` first.

Automation. The constitution submodule's tooling (the future `incorporate-submodule` per §11.4.31, the existing `scripts/init-submodules.sh` patterns in consuming projects) MUST auto-invoke `install_upstreams` as part of any clone / add operation when the target tree has an `upstreams/` directory. Operator-explicit manual invocation remains supported.

Gate `CM-INSTALL-UPSTREAMS-ON-CLONE`. Pre-merge gate inspects every owned-by-us submodule pointer commit and verifies that: (a) if `upstreams/` (or legacy `Upstreams/`) is present in the submodule's tree, (b) the submodule's local checkout has at least as many configured push URLs as the declared recipe count. Paired mutation (§1.1): remove one upstream from local `origin --push` config → gate FAILs.

Composition. §11.4.36 composes with §2 (single-entrypoint commit/push wrapper), §2.1 (multi-upstream push is the norm — this rule is its setup-time complement), §3 (submodule changes propagate through submodule commits first — requires multi-upstream parity), §9.2 / CONST-043 (no force-push — the multi-upstream wrap-up makes unforced parity easy), §11.4.17 (universal — every consuming project's clone procedure), §11.4.20 (subagent delegation when batch-cloning), §11.4.28 / CONST-051 (owned-submodule discipline), §11.4.29 / CONST-052 (`Upstreams/` → `upstreams/` transition is exactly this rule's scope), §11.4.30 / CONST-053 (`.gitignore` ignores `upstreams/` only if the project explicitly decides not to track recipes, which is unusual — recipes ARE source of truth and SHOULD be tracked), §11.4.31 / CONST-054 (submodule-dependency- manifest works alongside upstreams recipes: helix-deps.yaml lists WHAT to add at parent root, upstreams/ recipes list WHERE to push the resulting clones).

Classification: universal (per §11.4.17). No escape hatch. Severity-equivalent to §2.1 multi-upstream-push-violation at the clone-time setup layer — without the post-clone install, the working tree silently has a single-upstream blind spot.

---

§11.4.37 — Fetch-before-edit mandate (User mandate, 2026-05-15)

Forensic anchor — verbatim user mandate (2026-05-15):

> "Make sure that feedback_fetch_before_edit memory rule is part of our constitution Submodule - the root Consitution, AGENTS.MD and CLAUDE.MD. Validate and verify that Proejct-Toolkit and all Submodules do inherit all of them! Follow the constitution Submodule documentation for details."

Background. In multi-agent / multi-upstream codebases — where parallel Claude Code, Cursor, Aider, Codex, Gemini CLI, or human operator sessions may operate on overlapping scope — the local working tree's state can lag behind the canonical upstream state by the time any given agent receives a task. Acting on stale local state produces three failure modes documented in the originating session (2026-05-15):

1. **Redundant work** — the agent re-does what a parallel session already finished, wasting tokens and operator review time. (In the originating incident, "Gitee removal" had already been committed upstream by a parallel agent 25 minutes earlier; the second agent's `git remote remove gitee` was a no-op echo of work already done.)
2. **False confidence** — the agent reports completion of work that was already done by someone else, with no mechanical signal that their commit duplicates upstream history.
3. **Divergent history** — if the agent commits a parallel "completion" of already-done work, the result is two siblings of the same change, doubling the conflict surface for the next push attempt to multi-upstream remotes (§2.1).

The mandate. The FIRST git-touching action of any session, on any consuming project that participates in this constitution, MUST be:

```
git fetch --all --prune
git log --oneline HEAD..@{u}              # parent
git submodule foreach --recursive 'git fetch --all --prune --
        quiet'
```

If `HEAD..@{u}` is non-empty, the agent MUST integrate (ff-merge, rebase, or — if non-fast-forward — surface to operator per §11.4.4) BEFORE issuing any local edit, scanner run, or test cycle. The fetch step is non-negotiable even when the operator's directive is phrased as "do X immediately" — the 30-second check prevents hour-long conflict reconciliation later.

Scope. Applies to:

1. The consuming project root.
2. Every owned submodule (per §11.4.28) — recursive.
3. The constitution submodule itself (§11.4.26 step 1 makes this explicit for constitution-side edits; §11.4.37 generalises it to ANY edit on the consuming project, not only constitution edits).
4. Any dependency cloned via `incorporate-submodule` (§11.4.31) or `git submodule add` (§11.4.36).

Anti-bluff invariant. The fetch+log check MUST produce captured evidence — the actual `HEAD..@{u}` output, even if empty. Skipping the check on the basis of "I just fetched" or "nothing could have changed in the last N minutes" is a §11.4.6 (no-guessing) violation: the remote state is not knowable without a fetch.

Composition. Composes with §2.1 (multi-upstream push — without the fetch, the agent can't know which upstream has the canonical ref), §11.4.4 (test-interrupt-on-discovery — newly-fetched commits that contradict the planned work are exactly the "freshly discovered defect" that triggers cycle interruption), §11.4.6 (no-guessing — remote state requires fetch, not assumption), §11.4.20 (subagent delegation — parallel subagents MUST coordinate through fetched state, never assumed-local state), §11.4.26 (constitution update workflow — this rule generalises

§11.4.26 step 1 to ALL edits, not only constitution-file edits), §11.4.32 (post-constitution-pull validation — fetch-before-edit happens BEFORE the constitution-pull sweep §11.4.32 mandates).

Gates. Pre-build gate `CM-FETCH-BEFORE-EDIT-AUDIT` (when implemented in the consuming project) audits the most-recent commit range against the upstream HEAD at the commit's parent — if the parent ref was not the upstream HEAD at the time the commit was authored, FAIL. Paired mutation (§1.1): synthetic commit whose parent is N commits behind the then-current upstream HEAD — gate must FAIL.

Classification: universal (per §11.4.17). The rule has no project-specific assumptions — it applies to any multi-upstream / multi-agent codebase. No escape hatch. Severity-equivalent to a §11.4 PASS-bluff at the planning layer — acting on stale local state is the operational analog of asserting truth from unverified premises.

§11.4.38 — Installable-Asset Evidence Mandate (User mandate, 2026-05-17)

Forensic anchor — verbatim operator report (2026-05-17):

> "app does not have a launcher icon anymore — the latest published app tester build(s). how come app passed anti-bluff checks?"

For any user-distributable build artifact (package, bundle, installer, or container image produced by the build pipeline and distributed to end users), tests and challenges MUST open the artifact and verify each user-visible asset is **present** and **non-degenerate**.

"User-visible asset" includes (non-exhaustive):

- Application icons for every density / resolution tier declared in the artifact metadata, including any platform-specific icon format (multi-resolution container, adaptive XML, symbol set, etc.) that the target OS uses when the minimum supported OS version requires it.
- Splash screens declared in the install metadata.
- Application name strings as declared in the install metadata.
- Any other asset whose absence causes the user to be unable to identify, launch, or interact with the installed application from the OS launcher / home screen / app drawer.

A PASS without opening the artifact and verifying the asset chain end-to-end is a §11.4 PASS-bluff, regardless of whether source files exist and tests otherwise pass. The specific failure mode this rule targets is: source file exists → build pipeline packages it → post-build checks pass at the source layer → artifact ACTUALLY produced with the asset stripped or misconfigured, and no gate ever opens the artifact to verify.

Required evidence: the anti-bluff challenge for a user-distributable artifact MUST produce per-asset PASS/FAIL lines showing: (a) the asset entry exists in the artifact package listing, (b) the asset is non-empty / non-degenerate (size ≥ platform-defined minimum OR format-validated), (c) where the OS's asset-resolution path involves indirection (alias, XML reference chain, density-qualifier override), the full chain is traced to the final rendered resource.

Consuming-project implementation: each consuming project ships one challenge script per artifact type that opens the produced artifact and verifies every declared user-visible asset. The challenge MUST run as part of the project's standard QA gate (equivalent of `make qa-all`).

Root cause of §11.4.38 introduction: a consuming project shipped multiple releases with the application icon absent on the target OS because: (1) the icon XML used a resource type that the OS resolves differently above a specific API level, (2) all pre-ship challenges verified source files only, none opened the packaged artifact. This is a pure §11.4 PASS-bluff — every test and challenge reported green while the end user saw no icon.

Classification: universal (per §11.4.17). No escape hatch. Severity- equivalent to a §11.4 PASS-bluff at the artifact-packaging layer. Composes with §11.4.1–§11.4.5 (evidence requirements), §11.4.25 (full automation coverage), §11.4.27 (no fakes beyond unit tests — source-layer checks of a distributable artifact are the distributable- layer analog of unit-test-only coverage). See Constitution §11.4.38 for the full mandate.

§11.4.39 — Per-Feature On-Device End-User Validation Mandate (iter-76, 2026-05-17)

Every user-facing feature in a consuming project MUST have at least one HelixQA scenario that exercises the feature from cold-launch with **positive runtime evidence**:

- A screenshot or screen recording captured at a named checkpoint.
- At least one assertion of type `screenshot`, `accessibility_count`, `accessibility_node`, `ocr_text`, `pixel_histogram`, or `editor_state`.
- Evidence files written to a discoverable output directory so every PASS is cross-verifiable after the fact.

Scenarios MUST be RE-EXECUTED on every release candidate to catch cross-iteration regression. A scenario authored but never re-run on subsequent iterations degrades to a metadata-only gate — a §11.4 PASS-bluff at the regression layer.

Authoring rules (consuming project sets project-specific paths):

1. Scenarios are stored in a canonical bank directory designated by the consuming project (e.g. `<banks-root>/feature-coverage/`).
2. Each scenario YAML MUST include: `name`, `version`, `metadata`, `platforms`, `test_cases`, and at least one step with `evidence_required: true` and a specific `evidence_type`.
3. A **coverage matrix** document MUST exist listing feature × iteration × scenario. Matrix rules: (a) new iteration MUST add ≥ 1 scenario per new user-facing feature; (b) prior scenarios MUST NOT be deleted (mark as `status: retired` if feature is removed); (c) all non-retired scenarios MUST PASS for ship-ready.
4. A **static gate** (challenge script) MUST assert the scenario count ≥ N (where N is the number of user-facing features registered in the coverage matrix) and that each YAML contains a `evidence_type:` assertion.
5. **Portable host tooling**: evidence-validation scripts MUST use POSIX-portable file-size helpers (e.g. `wc -c < file`) instead of OS-specific commands (e.g. GNU `stat -c%s`) to avoid silent 0-byte reports on BSD/macOS hosts. A regression challenge MUST verify the portable helper on the actual host.

iOS / platform-deferred pattern: scenarios written platform-agnostically with the target platform listed in a comment deferral (e.g. `# iOS: deferred — tracker #X`) are valid; add the platform to the `platforms:` list when the automation toolchain becomes available — no structural change to the scenario is needed.

Classification: universal (§11.4.17). Composes with §7.1, §11.4 (anti-bluff), §11.4.25 (full automation coverage), §11.4.27 (no fakes), §11.4.38 (artifact evidence). No escape hatch — a feature without an on-device scenario is NOT covered per §11.4.25 invariant 2. See Constitution §11.4.39 for the full mandate.

§11.4.40 — Full-suite retest before release tag mandate (User mandate, 2026-05-17)

Forensic anchor — verbatim user mandate (2026-05-17):

> "Please, do complete retests with all existing tests (multi-hours efforts) when all new workable items are done, fixed, polished and verified. Time is essential! We should already have this in our (root) Constitution, CLAUDE.MD and AGENTS.MD."

Operative rule. A release tag (any `vX.Y.Z` / `X.Y.Z-suffix` identifier MUST NOT be created until a **COMPLETE retest with ALL existing tests** has been executed on a clean baseline AFTER every workable item in the batch is done, fixed, polished, and individually verified. A "spot-check" retest that runs only the tests directly touched by the batch is FORBIDDEN — it misses interaction defects between the batch's fixes and previously- stable code paths.

The complete retest comprises:

1. **Pre-build verification full sweep** — every gate in the project's pre-build script runs; 0 FAIL required.
2. **Post-build verification full sweep** — every gate in the project's post-build script runs against the freshly-assembled image; 0 FAIL required.
3. **On-device 4-phase cycle** (e.g. `test_all_fixes.sh` or project equivalent) on **EVERY owned device** (the full topology at that point in time). Each phase (Immediate Fresh Flash / After Reboot / After Factory Reset / After Final Reboot) MUST complete; no phase may be skipped. Phase summaries captured.
4. **Meta-test full mutation sweep** — every paired mutation in `meta_test_false_positive_proof.sh` (or project equivalent) executed; every gate proven non-bluff.
5. **Test bank full sweep** — if a Challenge-driven test bank exists, every Challenge in the bank covered by a full QA session (not a sub-set).
6. **Issues.md / Fixed.md state audit** — no item silently demoted; every Reopened item has §11.4.34 `Reopened-Details`; every closure has captured-evidence per §11.4.5; no `Status:` value outside the §11.4.15 closed-set.
7. **CONTINUATION.md sync check** — per §12.10, document reflects current state at the moment of tagging.

Time is essential. A complete retest is typically a **12–48 hour elapsed effort** depending on parallelism, device count, and meta-test mutation count. This is NOT optional and NOT abbreviated. Operators should plan release cadence with this duration in mind, NOT skip the retest to ship faster. Skipping the retest is the exact "tests passed but feature broken" failure mode §11.4 specifically prohibits.

Composition with §11.4.4. Per-fix retest (`test_all_fixes.sh` run after each individual fix lands) is STILL mandatory per §11.4.4. §11.4.40 is the **additional final integrity check** that runs after the BATCH is complete — catching interaction defects that individual per-fix retests cannot see in isolation.

Composition with §11.4.7. The complete retest is the authoritative captured-evidence baseline for any closure of items present in the batch. If a §11.4.7-promoted item PASSed its per-fix retest but FAILs the full-suite retest, the closure MUST be reverted and the item moved back to `In progress` — captured-evidence-contradicts under same-conditions per §11.4.7.

Composition with §11.4.39. Per-feature on-device end-user validation runs as part of step 3 (on-device cycle) — every feature's wrapper-test fires during the full-suite retest. The two mandates are complementary: §11.4.39 covers individual feature validation breadth; §11.4.40 covers release-time integration depth.

Gate `CM-FULL-SUITE-RETEST-MANDATE`. Pre-tag gate inspects the release-candidate state and verifies that within the last 72 hours of the candidate commit, evidence exists of (a) full pre-build + post-build run, (b) on-device 4-phase cycle on every owned device, (c) meta-test full sweep, (d) Issues.md/Fixed.md audit pass. Evidence captured in `docs/changelogs/<tag>.md` per §11.4.4(c) requirements. Paired mutation (§1.1): strip `Full-suite retest evidence` block from a changelog → gate FAILs.

Classification: universal (per §11.4.17) — every consuming project's release procedure. No escape hatch — there is no `--skip-full-retest` flag, no `--quick-release` mode. Operators who feel time-pressured to skip the full retest should instead delay the release until the retest completes; shipping unverified code is worse than delayed shipping.

§11.4.41 — Pre-Force-Push Merge-First Mandate (User mandate, 2026-05-17)

Forensic anchor — verbatim user mandate (2026-05-17):

> "make sure we bring everything from branches to our side before forc push is done! Afer everything is safely and fully merged and all potential conflicts (if any) resolved, then do force push! make sure nothing isnlost, broken or corrupted on bith sides! add these rules in our root Constitution, CLAUDE.MD, AGENTS.MD (constitution Submodule) if itnis not added already! Extremely important rules and mandatory constraints we MUST HAVE and fully respect!"

Operative rule. Any force-push (`git push --force`, `git push --force-with-lease`, `git push +<ref>`, equivalent history-rewriting operation on any remote) authorised under §9.2 / CONST-043 MUST be preceded by a mechanical 4-step merge- first pipeline that brings every remote-side commit into the local tree, resolves every conflict carefully, and verifies nothing is lost or corrupted on EITHER side BEFORE the overwriting push is executed.

The 4-step pipeline (mandatory, in order):

1. **Fetch every remote-side reference.** Run `git fetch --all --prune --tags` against every configured remote (origin + every upstream). Capture the output.

2. **Integrate every divergent commit locally.** Compute `git log --oneline HEAD..<remote>/<branch>` for every remote. For every non-empty range, integrate the remote commits into the local tree via the appropriate strategy:
   - **`git rebase <remote>/<branch>`** when the local divergent work is a strict superset that should land on top, OR
   - **`git merge <remote>/<branch>`** when the local + remote histories are independent additions that both deserve preservation, OR
   - **operator-confirmed cherry-pick** when remote contains a subset of commits already present locally in different form.
3. **Audit the integrated tree.** Verify (a) no conflict markers anywhere in the working tree (`grep -rn '^<<<<<<< \|^=======$\|^>>>>>>> ' .` returns empty across all governance + source + test files — third-party docs containing the markers as illustrative content excepted), (b) no file silently dropped (`git diff --stat HEAD@{1} HEAD` shows only expected additions), (c) every previously-passing test still passes on the integrated tree (per §11.4.4 + §11.4.40 baseline), (d) every captured- evidence artifact from the pre-merge state still validates.
4. **Execute force-push.** Only after steps 1-3 produce captured evidence of clean integration: `git push --force-with-lease <remote> <ref>` (NEVER `--force` without `--with-lease` unless explicitly authorised per §9.2 sub-clause 6 for a specific remote where lease semantics are unavailable). One force-push event per CONST-043 authorisation — no batch authorisation across multiple force-pushes.

Three failure modes prevented:

1. **Remote-side content loss.** Force-push without prior fetch silently overwrites commits a parallel session, sibling agent, or co-developer landed on the remote. The lost work is recoverable from the remote's reflog only within its TTL window — typically 30-90 days — but the audit trail (PR comments, CI evidence, governance signatures) is lost permanently.

2. **Stale-state act on remote.** Force-push from a divergent local state that was branched from a remote tip N hours ago treats the remote-side N hours of work as nonexistent. The operator's `--force-with-lease` safety check catches the literal SHA mismatch but ONLY when invoked AFTER a fetch that updated the lease reference; without step 1, the lease check is reading stale local refs.

3. **Conflict-driven corruption.** A merge that completes with unresolved markers in the tree (one half kept, other dropped silently because the marker was inside a comment block, OR markers committed verbatim as in CONST-049-prerequisite incidents observed 2026-05-17 on helix_qa + containers) lands broken governance / source on the force-push target. Step 3's explicit conflict-marker grep is the mechanical guard.

Two-gate composition with CONST-043. §11.4.41 does NOT relax CONST-043's operator-approval requirement — it adds a SECOND gate on top:

- **Gate A (CONST-043).** Operator types explicit per-operation force-push authorisation in the conversation.
- **Gate B (§11.4.41).** Agent executes the 4-step merge-first pipeline, captures evidence of clean integration, presents evidence to operator BEFORE the force-push.

Both gates required. CONST-043 alone authorises a force-push that loses remote work; §11.4.41 alone risks force-pushing without operator awareness. The two together produce a force-push that is BOTH operator-authorised AND remote-safe.

Verification artefact. Every §11.4.41-governed force-push emits a `docs/changelogs/<tag>.md` "Force-push merge-first audit" section containing: (i) `git fetch` output, (ii) per- remote `HEAD..<remote>/<branch>` log before integration, (iii) integration strategy chosen per remote with rationale, (iv) post-integration conflict-marker scan output (must be empty), (v) post-integration test suite delta (must show only expected changes), (vi) the `--force-with-lease` push output with lease SHA evidence, (vii) CONST-043 authorisation quote from the conversation.

Cascade requirement (per CONST-047). This anchor (verbatim or by `§11.4.41` / `CONST-061` ID reference) MUST appear in every owned submodule's `CONSTITUTION.md`, `CLAUDE.md`, and `AGENTS.md`. Severity-equivalent to a §11.4 PASS-bluff at the remote-data-integrity layer. Gate `CM-FORCE-PUSH-MERGE-FIRST` walks `docs/changelogs/<tag>.md` "Force-push" entries for the 7 audit elements; paired mutation strips any element and asserts gate FAILs.

Classification: universal (per §11.4.17) — every owned project executes force-pushes against its remotes; the merge- first discipline is reusable verbatim. No escape hatch — the operator-pressure escape ("just force-push, we'll fix it later") is the exact failure mode this anchor closes.

Composes with: §9.2 (data-safety hardlinked backup), §11.4.4 (test-interrupt-on-discovery — broken integration triggers rollback), §11.4.6 (no-guessing — every step's outcome captured, not assumed), §11.4.26 (constitution-submodule update pipeline — that mandate is the per-submodule specialisation of §11.4.41 for governance-files-only commits), §11.4.32 (post-pull validation — the audit step's mechanical companion), §11.4.37 (fetch-before- edit — step 1 enforces it for the force-push specifically), §11.4.40 (full-suite retest — step 3's test-evidence prerequisite), CONST-043 (per-operation operator approval), CONST-047 (recursive cascade). ### §11.4.42 — Iteration-discipline mandate (User mandate, 2026-05-18)

Forensic anchor — verbatim user mandate (2026-05-18):

> "Are we clear about working iterations and sorting priorites for testing and fixing? We first fix top and middle critical priorites → We flash and test all of these → If nothing is broken and no new issues are reported by users or found we run full system testing (many hours for everything) → if anything is still broken or new issues are reported we get back to fixing → Cycle repeats. Make sure this is absolutely clear and mandatory foundation of our root (constitution Submodule) Constitution, AGENTS.MD and CLAUDE.MD! We MUST WORK in this manner until otherwise has been told."

Operative rule. Project work proceeds in priority-ordered iteration cycles. Each cycle is a closed loop of five steps that MUST run in order; advancing past any step before its acceptance condition is met is a §11.4 bluff equivalent to skipping the step entirely.

The five-step cycle:

1. **Priority-ordered fix selection.** Open the project's issue tracker, filter to items whose `**Status:**` is one of `Queued | In progress | Reopened | Operator-blocked` (per §11.4.15 closed-set), sort by severity DESC then intra-group criticality DESC (per §11.4.12 sort order), select ONLY items classified `Top critical` or `Middle critical` for the current batch. `Low` items are explicitly DEFERRED until the critical batch is closed.
2. **Batch implementation.** Land source-side fixes for the selected batch per §11.4.9 (batch-source-fixes-before-rebuild) — every non-runtime-dependent fix lands BEFORE the next rebuild, every fix gets the §11.4.4 four-layer coverage (pre-build gate + post-build gate + on-device test + paired mutation).
3. **Smoke-test gate.** After flash, run the SMOKE bundle:
    1. anti-bluff baseline (`meta_test_false_positive_proof.sh` for gates touched by this batch), (b) every `test_*.sh` whose name matches a fix in this batch, (c) the critical-path regression probe (boot + launcher + WiFi + audio + secondary-display routing core), (d) the §11.4.39 per-feature on-device validation for any feature touched. Estimated runtime: <30 minutes on the parallel device fleet. Smoke MUST emit zero FAIL and zero unclassified WARN.
4. **Full-system-test gate.** ONLY if (a) smoke is GREEN AND (b) no new operator/user report has surfaced during the batch's validation window, run the §11.4.40 complete retest (all 7 steps including 12–48 h on-device 4-phase cycle on every owned device). If smoke FAILed OR a new report arrived, the full-system test is FORBIDDEN; loop back to step 1 with the new evidence added to the priority queue.
5. **Release-readiness or loop.** If full-system test is GREEN AND no new report arrived during it, the batch is release-ready (the §11.4.40 tagging procedure may proceed pending operator authorization). Otherwise loop back to step 1.

Priority taxonomy (binds existing severity convention). `Top critical` = severity `C` (Critical per Issues.md convention) AND intra-group criticality `5`. `Middle critical` = severity `C` with intra-group `1–4` OR severity `M` (Major) with any intra-group score. `Low` = severity `L` per the existing `[C/M/L]` taxonomy documented in §11.4.12. Severity `WARN` items are treated as `Low` for iteration-discipline purposes.

Cycle repeats. The five-step loop continues, fed by the priority queue, until the operator explicitly authorizes a release. There is no upper bound on iteration count; "we've cycled enough" is not a Constitution-recognised stopping condition. Time is essential per §11.4.40 BUT the cycle MUST NOT be truncated to ship faster — that is exactly the bluff §11.4 forbids.

Composition. §11.4.4 (per-fix retest inside step 2) + §11.4.7 (closures require same-conditions positive evidence; step 4 is authoritative baseline) + §11.4.9 (source-side batching inside step 2) + §11.4.34 (Reopened items attribute By: AI / User) + §11.4.40 (the multi-hour full-suite retest IS step 4 — §11.4.42 is the meta-loop conductor binding the instruments).

Anti-bluff coupling. Per §11.4.2 + §11.4.5, every smoke-test and full-system-test PASS MUST carry positive captured evidence of user-visible behaviour. Tests AND HelixQA Challenges bound equally — a Challenge that scores PASS without applicable analysis is a §11.4 PASS-bluff.

No escape hatch. No `--skip-priority-batch`, `--skip-smoke`, `--full-suite-only`, or `--release-without-loop` flag exists. Subagents performing autonomous work default to the §11.4.42 path. Operators who feel time-pressured to short-circuit the loop should add capacity (more devices in parallel, more agent slots) rather than skip steps. Shipping under-validated code is worse than delayed shipping.

Gate `CM-COVENANT-114-42-PROPAGATION` mirrors `CM-COVENANT-114-40-PROPAGATION` 1:1 — every CLAUDE.md / AGENTS.md in the covenant file set carries the §11.4.42 anchor. Paired mutation strips the anchor literal from one consumer file → gate FAILs.

Classification: universal (per §11.4.17) — every consuming project's iteration workflow.

§11.4.43 — TDD-Fix-Discipline Mandate (User mandate, 2026-05-18)

Forensic anchor — verbatim user mandate (2026-05-18):

> "Make sure you do validate and verify every fix! Make sure you first do the fix on live device using ADB if that is possible, once fix is confirmed, make sure that it is fully applied into our codebase … It is important to note that we shall start by creating test which confirms the issue (fails as expected), and ones the fix is produced it becomes positive - it does pass with success! No bluff policy is mandatory and nothing can be bluffed! Every step of this process MUST BE 100% bluff-free and bluff-proofed!"

Operative rule. Every fix MUST follow the 5-step TDD-fix workflow:

1. **RED — Failing test FIRST.** Before any code change, author a test (or extend an existing one) that exercises the user-visible path of the reported defect and FAILs for that defect specifically. Per §11.4.1 the FAIL MUST be a real product defect (not a script-bug induced FAIL). Per §11.4.2 the FAIL run MUST produce captured evidence (logcat / dumpsys / screenrecord / dmesg / `/sys` snapshot / sink-side probe). The test identifier MUST cite the §-letter or Fix-# under repair. A RED step that "happens to pass because the underlying probe lifecycle isn't wired" is itself a §11.4 PASS-bluff.

2. **LIVE-ADB-PROBE — try the fix on the running device first (when feasible).** For mutable surfaces — `setprop persist.*`, `settings put`, `pm clear`, `am force-stop`, `am start`, `cmd wifi`, `cmd media_session`, ad-hoc `/sys` writes, boot- script edits pushed to `/vendor/bin/`, test-fixture replacement — the operator MUST attempt the fix on the running device via `adb shell` first. The fast adb-loop (~5 min per attempt) replaces the build-flash-test loop (~6 h per attempt) for confirmation of the hypothesis. The live-probe IS NOT the fix; it is the EXPERIMENT that proves the hypothesis is correct before the source-side investment.

   Live-probe is INFEASIBLE (exception, must rebuild) for: kernel changes, AOSP framework (frameworks/base/), hardware HAL changes, vendor/ partition contents, init.rc service definitions, sepolicy, ro.* build properties (immutable post-boot), AOSP-build-time XML / resource overlays, Android.bp/

Android.mk changes, signed-system-app contents (LOCAL_CERTIFICATE := platform). Each exception MUST be cited in the commit message as `LIVE_PROBE_INFEASIBLE: <reason>`.

3. **GREEN — apply the fix to source code.** The source patch MUST achieve the same effect on the device as the live probe. Per §11.4.9 the source change is batched with other source-side fixes where appropriate. Build via the project's containerized build (§12.9). Flash via the project's flash script. The bytes on the assembled image MUST land where the post-build gate expects (per §11.4.4(b) four-layer requirement).

4. **VERIFY — re-run the RED test, must now PASS.** Per §11.4.7 demotion-evidence: the PASS MUST come from the SAME conditions that initially FAILed (same device, same firmware NOW carrying the fix, same load profile, same cycle position). Per §11.4.5 the PASS MUST carry positive captured evidence (presence + correctness — RMS amplitude, ffprobe channel count, frame count, OCR text, sink codec state, etc.). Per §11.4.2 video tests MUST cross-check via the recording-analyzer. Per §11.4.42 a reliability check at the iteration count (typically 10 iterations) MUST PASS all iterations — a single intermittent FAIL keeps the item in `In progress` per §11.4.7.

5. **DOCUMENT — update every relevant doc in the SAME commit.** Per §11.4.4(c): `docs/Issues.md` → `docs/Fixed.md` migration with type-aware closure vocabulary per §11.4.33 (Bug → `Fixed`, Feature → `Implemented`, Task → `Completed`); the project's CLAUDE.md Applied Fixes Reference row; per-version `docs/changelogs/<tag>.md` entry; affected user-facing guides under `docs/guides/`; HelixQA bank entry per §11.4.4(b); `docs/CONTINUATION.md` per §12.10; project memory file if non-obvious.

Composition. §11.4.43 is the workflow that ENACTS the existing covenant. Composes with: §11.4.1 (the RED FAIL is a real defect), §11.4.2 (captured evidence at both RED and VERIFY), §11.4.3 (topology dispatch in the RED test), §11.4.4 (four-layer coverage at GREEN), §11.4.5 (quality analysis at VERIFY), §11.4.6 (no guessing during the LIVE-PROBE hypothesis statement — `UNCONFIRMED:` until probe proves it), §11.4.7 (demotion-evidence at VERIFY — same conditions), §11.4.8 (research citation precedes step 1), §11.4.9 (GREEN is batched), §11.4.40 (full-suite retest is the release-tag-time additional check), §11.4.42 (10-iteration reliability loop IS step 4's inner cycle).

Gate `CM-COVENANT-114-43-PROPAGATION`. Pre-build gate verifies the §11.4.43 anchor is present in every CLAUDE.md / AGENTS.md across the covenant file set. Paired mutation strips the anchor literal from one consumer file → gate FAILs.

Classification: universal (per §11.4.17) — every consuming project's fix workflow. **No escape hatch.** No `--skip-red-test`, `--no-live-probe`, `--skip-verify`, `--skip-document` flag exists. "I'll add the test after the fix" is the exact PASS-bluff pattern §11.4 forbids because the test authored after the fix demonstrates only that the test agrees with the fix, not that the test catches the bug.

§11.4.44 — Document Revision Header Mandate (User mandate, 2026-05-18)

Forensic anchor — verbatim user mandate (2026-05-18):

> “Add this change / improvement as background work in parallel with our mainstream work. All documents (Issues, Issues_Summary, Fixed, Continuation doc and all others MUST HAVE revision number! Revision number with information such as: last date and time when document was modified MUST GO at the top of the document below the main H1 title. Add all relevant information about the document as well. However, last date and time document has been modified and revision number are MANDATORY! Revision number shall be the whole number 1, 2, 3, etc. Document all this in our root (constitution Submodule) Constitution, AGENTS.MD and CLAUDE.MD as mandatory rule(s) and constraints we MUST HAVE! No bluff is allowed of any kind!”

The mandate. Every IN-scope tracked Markdown document MUST carry a header block directly below the H1 title containing two MANDATORY fields:

- `**Revision:** N` where N is a monotonic positive integer (1, 2, 3, …). Never decremented. Never reset on rewrite. A freshly- created IN-scope document starts at Revision 1.
- `**Last modified:** YYYY-MM-DDTHH:MM:SSZ` (ISO 8601 UTC).

Optional encouraged fields: `**Description:**`, `**Authority:**`, `**Maintainer:**`, `**Scope:**`, `**Auto-generated-from:**` (auto-generated docs only), `**Status:**` (plan docs only).

Header format (canonical):

```
# Document Title

**Revision:** 47
**Last modified:** 2026-05-18T13:42:00Z
**Description:** Open / in-flight bug + work tracker
**Authority:** Constitution §11.4 covenant
**Maintainer:** Operator + AI loop per §11.4.42

<content...>
```

YAML front-matter and HTML-comment headers are FORBIDDEN — they are invisible to humans scrolling the raw Markdown and break the “all relevant information about the document” visibility clause.

IN scope (revision header MANDATORY): `docs/Issues.md`, `docs/Issues_Summary.md`, `docs/Fixed.md`, `docs/Fixed_Summary.md`, `docs/CONTINUATION.md`, `docs/guides/**/*.md`, `docs/research/**/*.md`, `docs/scripts/**/*.md`, `docs/changelogs/**/*.md`, `docs/superpowers/plans/**/*.md`, `docs/hardware/**/*.md`, and every other tracked Markdown under `docs/`.

OUT scope (revision header NOT required): `CLAUDE.md` / `AGENTS.md` (every layer — already version-tracked via VERSION file + inheritance pointer per §11.4.35; adding a per-file revision would duplicate authority and obscure the canonical-root contract); `README.md` / `CONTRIBUTING.md` / `LICENSE` / `NOTICE` / `VERSION` / `OWNERS` (standard metadata); rendered HTML/PDF artifacts (revision inherited from source Markdown); auto- generated docs that already carry an embedded generation-time stamp.

Auto-bump tooling: - `scripts/doc_revision_bump.sh <file>` — manual entry point, idempotent (no-op if Last-modified within 5 s of `date -u`). - `.git/hooks/pre-commit` — automatic entry point; walks staged IN-scope docs, calls the bump for each, re-stages. - `scripts/testing/sync_issues_docs.sh` — composes the bump for Issues_Summary + Fixed_Summary after regeneration. - `scripts/commit_docs.sh` — calls the bump before staging so operators who skipped the hook installation are still covered.

Composition with §12.10. CONTINUATION.md already carries `Last updated:` per §12.10. §11.4.44 adds `**Revision:** N` directly under H1 + reuses the existing `Last updated:` line as the §11.4.44 `Last modified:` line. One source of truth, two pointers — no duplication.

Composition with §11.4.12. Issues_Summary.md / Fixed_Summary.md inherit the revision number of their source-of-truth Markdown at generation time; they are NOT incremented independently.

Composition with §11.4.23. HTML colorizer preserves the revision header in the colorized HTML — pandoc renders the bold lines as styled `<p><strong>...</strong></p>` blocks which the colorizer leaves alone (only Issues-table rows are mutated).

Gates: `CM-DOC-REVISION-HEADER-PRESENT` walks IN-scope docs and asserts both mandatory fields appear within 15 lines below the H1 title; `CM-COVENANT-114-44-PROPAGATION` asserts the anchor literal is present in every covenant file. Paired mutations strip the `**Revision:**` line OR the anchor literal → gates FAIL.

Classification: universal (per §11.4.17). **No escape hatch** — no `--skip-revision-bump`, `--no-header`, `--allow-missing-revision` flag exists. Operators who feel pressured to skip the bump should land smaller commits, not bypass the discipline.

§11.4.45 — Integration-Status-Doc Maintenance Mandate (User mandate, 2026-05-18)

Forensic anchor — verbatim user mandate (2026-05-18):

> "Make sure this Markdown document is regularly updated to reflect current up to date state of full integration … Whole integrations MUST be regularly updated and retested! Keep the Markdown document ALWAYS up to date and ALWAYS exported into PDF and HTML! Make sure document is ALWAYS in sync! Add all this into our Constitution, AGENTS.MD and CLAUDE.MD."

Generalisation note. §11.4.45 is the generic form of §12.10 (CONTINUATION.md) applied to every domain integration. §12.10 binds the canonical handoff document specifically; §11.4.45 generalises the sync-and-evidence pattern to every non-trivial integration (Dolby/Arvus, NanoKVM, Firebase, Widevine L1/L3, Play Protect, Netflix, Cuttlefish, Chromecast, WiFi chip, ES8388 codec, Sonos, AC-3, Google Stack, HiFi audio, TV apps, etc.). Without this generalisation each new integration re-invents the same sync wrapper, revision-header discipline, captured-evidence requirement, and operator-blocked surface — duplicating effort and risking drift.

Operative rule (10 sub-points — every Status.md MUST hold ALL).

Every integration-status document at `docs/<domain>/<integration>/Status.md`:

1. MUST exist when a domain integration is non-trivial (more than one fix / test / gate landing for that integration).
2. MUST carry the §11.4.44 revision header directly below the H1 title (Revision + Last modified + Description + Authority + Maintainer + Scope).
3. MUST be auto-synced (HTML + PDF) on every related-test-cycle completion AND every fix touching the integration. "Auto-synced" means the wrapper from sub-point 5 is invoked; manual regeneration is forbidden as the sole path because operators will forget.
4. MUST be auto-colorized per §11.4.23 — Status column cells colored by lifecycle state, Type cells colored by Type, operator-blocked rows visually distinct.
5. MUST have a sync wrapper invocable as `bash scripts/testing/sync_integration_status.sh <Status.md>` OR a per-integration thin shell that delegates to the generic wrapper. The thin shell exists so operators can type a short command per integration without remembering paths.
6. MUST live under `docs/<domain>/<integration>/` directory structure — consistent layout, discovery by glob remains O(1).
7. MUST include a captured-evidence-driven status table per §11.4.5 — every status claim cites the test log path, recording file path, or sink-probe report that backs it. Status claims without evidence are §11.4 PASS-bluffs.
8. MUST distinguish status values per §11.4.6 / §11.4.7: PASS / FAIL / SKIP / PENDING_FORENSICS / OPERATOR-BLOCKED — closed vocabulary, mechanically distinguishable, no implicit values.
9. MUST list operator-blocked items at the top of the document (just below the revision header) so an operator scanning the Status.md finds action items in O(1) — not buried 200 lines deep in a chronological evidence log.
10. MUST be referenced from `docs/CONTINUATION.md` §3 (Active work) when any item in the Status.md is non-terminal (Queued / In progress / Reopened / Operator-blocked) — composes with §12.10 so the canonical handoff document points at the integration-status doc rather than re-stating its contents.

Gates:

- `CM-COVENANT-114-45-PROPAGATION` — anchor text propagated to every CLAUDE.md / AGENTS.md across the covenant file set.
- `CM-AF-INTEGRATION-STATUS-DOCS` — discovers all `docs/**/Status.md` via glob, verifies each carries the §11.4.44 header, has a sync wrapper, HTML+PDF exports are mtime-current vs the source, colorization applied (cell-status class present in the colorized HTML).

Paired meta-test mutations (§1.1 compliance): propagation strip → propagation gate FAILs; delete `**Revision:**` from a Status.md → status-docs gate FAILs; touch Status.md without re-syncing → mtime mismatch FAILs; inject `likely` outside `UNCONFIRMED:` block → existing `CM-NO-GUESSING-MANDATE` catches it (composition catch).

Composition. §11.4.5 (captured evidence in every row), §11.4.6 / §11.4.7 (status vocabulary closed-set), §11.4.12 (sync wrapper pattern reused for HTML+PDF re-export), §11.4.13 (sink-side captured evidence is a specific instance of the generic

rule), §11.4.15 (status values), §11.4.22 (commit_docs.sh wrapper), §11.4.23 (colorizer extends to Status.md HTML), §11.4.44 (every Status.md carries the revision header), §12.10 (CONTINUATION.md references Status.md paths in §3).

No escape hatch — no `--skip-status-sync`, `--no-revision-bump-on-status`, `--allow-stale-html` flag. Operators who feel pressured to skip the sync should land smaller, more frequent integration commits rather than batching status updates into one giant push.

Classification: universal (per §11.4.17).

§11.4.46 — Validate-recent-work-before-post-flash-tests mandate (User mandate, 2026-05-18)

Forensic anchor — verbatim user mandate (2026-05-18):

> "We see that on newly flashed devices we execute all post flash tests. Make sure we do execute them all after we validate and verify all recent work from the Issues and Continuation docs first! Once all new / latest work is fully validated and verified, no new issues found, nothing is discovered broken or faulty, then we run all post-flash tests! If validation and verification of recent work fails for any reason, and we must go back to fix anything that popped up we do not have to run post-flash tests! We MUST save time! All post-flash tests can be executed only after recent work on features and issues is fully validated and confirmed with complete anti-bluff policy enforced! Add this so it is absolutely clear into our root (constitution Submodule) Constitution, AGENTS.MD and CLAUDE.MD! Start following all these rules IMMIDIATELY! Iff required abort all post-flash testing if it is in progress!"

Operative rule. After every device flash, the orchestrator MUST first run a recent-work validation pass before any full post-flash suite (`test_all_fixes.sh` or project equivalent). The validation pass enumerates recent work from three docs (Issues.md / Fixed.md / CONTINUATION.md §3), maps each recent item to its on-device test binding, runs ONLY those tests, classifies each result per §11.4.6, and demands 100% green before the full suite is permitted.

Definition of "recent work" (closed-set):

- `docs/Issues.md` actionable headings with `**Status:**` ∈ `{In progress, Ready for testing, Reopened}` (per §11.4.15 closed-set).
- `docs/Fixed.md` headings closed within `--max-age-days N` (default 7).
- `docs/CONTINUATION.md` §3 "Active work" sub-headings listed as IN PROGRESS or BLOCKED.

Recent-work test binding. Each recent item's on-device test is discovered via §-letter or Fix # match against the project's test_*.sh filenames OR an explicit `Test:` line in the item heading. Items without an on-device test are §11.4.4 four-layer-coverage violations (a fix without an on-device test is forbidden) — flagged as VALIDATION-PASS-BLOCKED until the test lands.

Authorization sequence (machine-enforced):

1. `scripts/testing/recent_work_validate.sh --device <serial>` runs; writes `/data/local/tmp/.recent_work_validated` on the device IFF all targeted tests return PASS with captured evidence.
2. The full suite (`test_all_fixes.sh`) checks for the marker file at entry; absent or stale (older than the last flash boot epoch) → refuses to proceed with exit code 11.
3. Marker is invalidated automatically on reboot — the orchestrator stores the device boot epoch in the marker and re-validates.

Anti-bluff during validation. Each recent-work item that landed a fix in this batch MUST have a paired §11.4.43 RED test (captured before the fix) AND the same test now GREEN (captured after the fix). A GREEN with no prior RED is itself the bluff §11.4 forbids — the gate FAILs.

Honest classification of validation FAILs. Per §11.4.6: PRODUCT defect (real regression — block full-suite, surface to operator), TEST-INFRA defect (test-script bug — fix per §11.4.1 at source layer, re-run), BLUFFY-THRESHOLD (e.g. SLO floor wrong — re-baseline), or UNCONFIRMED (not reproducible in isolation — tag per §11.4.7 PENDING_CYCLE_RETEST, NEVER silently demote).

Composition. §11.4.4 (STOP-on-discovery) + §11.4.6 (no- guessing) + §11.4.7 (demotion-evidence) + §11.4.40 (full-suite gate) + §11.4.42 (iteration discipline — smoke-test ≡ recent-work- validation) + §11.4.43 (RED test for each recent-work item IS the validation test) + §11.4.44 (revision header determines recency) + §12.10 (CONTINUATION.md source-of-truth for §3 Active work).

Gates:

- `CM-COVENANT-114-46-PROPAGATION` — anchor literal present in every CLAUDE.md / AGENTS.md across the covenant file set.
- `CM-AF-RECENT-WORK-VALIDATION-GATE` — asserts `scripts/testing/recent_work_validate.sh` exists + executable + sources the anti-bluff library + the full-suite orchestrator contains the entry-gate check.
- `CM-AF-VALIDATION-ARTIFACT-FILE` — asserts the marker file path `/data/local/tmp/.recent_work_validated` is literal-identical across helper + orchestrator + propagation-block + this Constitution section.

Paired mutations strip the anchor / remove the entry-gate / flip the marker path → respective gate FAILs.

No escape hatch. No `--skip-validation`, `--full-suite-always`, `--ignore-recent-work` flag is permitted. The full-suite-without- validation pattern is the precise "tests passed but feature broken" failure mode §11.4 specifically prohibits. The 60–90 min validation pass is ALWAYS faster than 4–6 h of full-suite chasing a defect.

Classification: universal (per §11.4.17).

§11.4.47 — Firebase Data Review Mandate (User mandate, 2026-05-18)

Forensic anchor — verbatim user mandate (2026-05-18):

> "We MUST regularly before every bigger working round check Firebase information gathered so far: Crashlytics (all fatals, non-fatals and ANRs), Analytics data and Performance data. For anything problematic depending on severity proper workable items (issues) MUST be created (Issues, Issues_Summary docs) with full references (links) to original data on Firebase. We MUST make sure we do not create for same issues multiple entries, so we MUST distinguish between same issue manifested in different forms (stacktraces)! Everything MUST BE comprehensive and in-depth level of details so any changes we do (or fixes) do solve the original root problems!"

The mandate. Before every "bigger working round" (pre-build, pre-flash, pre-tag, daily, post-deployment burn-in) the operator/ loop MUST execute the Firebase review pass via `scripts/firebase/review_round.sh`. The pass queries Crashlytics (fatals + non-fatals + ANRs) + Analytics + Performance, classifies each finding by the §11.4.47 severity table, dedup-maps to existing Issues.md entries via the three-tier algorithm, and drafts new Issue entries for unrecognised findings. Skipping the pass is a §11.4 PASS-bluff: Firebase IS the captured evidence from real end-user devices; ignoring it is the precise "tests pass, feature broken in the wild" failure mode §11.4 prohibits.

Five mandatory elements (ALL must hold):

1. **Trigger cadence (5 trigger types).** Pre-build (blocking, <24 h freshness), pre-flash (blocking, <24 h), pre-tag (blocking, <6 h), daily 09:00 UTC default (non-blocking sweep), post-deployment burn-in T+24 h (non-blocking).
2. **Three-source query.** Crashlytics + Analytics + Performance — ALL three. Skipping one source is a §11.4 PASS-bluff (Firebase reports a regression on the skipped axis and we never see it).
3. **Issues.md output.** Every "problematic" finding MUST map to either (a) a new Issues.md entry, OR (b) a recognised existing entry via the dedup algorithm. Both paths produce a full Firebase Console URL in the entry's metadata so any future agent can re-fetch the raw data without rebuilding context.
4. **Three-tier deduplication.** Tier 1 (exact Firebase Issue-ID match) → Tier 2 (stacktrace-similarity cluster hash: top-3 non-generic frames normalised + crash class → SHA-256 first 8 hex chars) → Tier 3 (monthly operator merge review). Multiple Issues.md entries pointing at the same underlying root cause is a §11.4 violation (operator chases ghosts; root cause stays unfixed).
5. **Comprehensive root-cause analysis.** Every Firebase-sourced Issue carries the §11.4.4(a) systematic-debugging output — Phase 1 evidence, Phase 2 pattern, Phase 3 root-cause hypothesis (UNCONFIRMED until §11.4.43 RED test reproduces), Phase 4 fix direction. Comment-only entries ("crash in App X") are PASS-bluff stubs and FAIL the Issue-xref gate.

Severity classification: Crashlytics FATAL impactedUsers > 10 last 7d → Critical; 1–10 → Major; 0 but historic → Low. NON-FATAL > 100/day → Major; 10–100 → Low; < 10 → Informational. ANR > 1% sessions → Major; 0.1–1% → Low. Performance KPI regression > 20% → Major; 10–20% → Low. Analytics funnel-drop > 50% → Major; 25–50% → Low; feature_used regression > 75% → Major. Composes with §11.4.16 Type assignment.

Gates:

- `CM-COVENANT-114-47-PROPAGATION` — anchor present across every CLAUDE.md / AGENTS.md in the covenant file set.

- `CM-AF-FIREBASE-REVIEW-CADENCE` — `scripts/firebase/review_round.sh` exists, executable, contains the 7-stage pipeline literals + dedup algorithm + severity table.
- `CM-AF-FIREBASE-ISSUE-XREF` — every Issues.md entry whose `**Source:**` is `Firebase Crashlytics` / `Firebase Analytics` / `Firebase Performance` carries `**Firebase Issue IDs:**` + `**Firebase URL:**` + (`**Stacktrace Cluster Hash:**` for Crashlytics OR `**KPI:**` for Performance OR `**Funnel:**` for Analytics).

Paired mutations strip the anchor / rename the review_round function / remove the Firebase-URL field from the template → respective gate FAILs.

Composition. §11.4.4 (STOP-on-discovery), §11.4.4(a) (systematic-debugging), §11.4.6 (no-guessing — Firebase data IS captured evidence), §11.4.7 (demotion-evidence — a previously- Fixed Issue resurfacing in Firebase IS positive captured evidence the fix didn't hold; reopen attribution = "AI: captured-evidence-contradicts" per §11.4.34), §11.4.10 (credentials — never log the bearer token), §11.4.12 (Issues_Summary sync), §11.4.14 (cleanup of `/tmp/firebase_review_*` work-dirs), §11.4.15 (Status: Queued), §11.4.16 (Type per severity), §11.4.34 (Reopened-Details on Firebase-resurface), §11.4.42 (implicit step 2 inclusion), §11.4.43 (RED test per stacktrace before fix lands), §11.4.44 (revision header on every Firebase-sourced Issue), §11.4.45 (Firebase Status.md at `docs/firebase/status/Status.md`), §11.4.46 (validation pass consults latest Firebase delta).

No escape hatch. No `--skip-firebase-review`, `--no-issue-from- firebase`, `--firebase-review-not-applicable` flag is permitted. The operator MAY filter by minimum severity (`--severity-min major`) to reduce noise but the pass itself MUST execute.

Classification: universal (per §11.4.17) — every consuming project that ships to real end-user devices benefits from the same review cadence and the same dedup algorithm.

§11.4.48 — UI-Driven Video Testing Mandate (User mandate, 2026-05-18)

Forensic anchor — verbatim user mandate (2026-05-18):

> "We MUST make sure that all video playback testing with 2nd display is performed by fully automatically using the application's UI / UX, with full navigation, choice of video and clicks to real UI buttons to play or switch / stop videos being played! Maybe direct execution of Intents or Broadcasts does not have reported issues! We MUST test ALL WAYS users or Systems may / could play some video contents or streams! For all applications, all video and audio stream types! All supported codecs! EVERYTHING verified with 2nd display on D3 device and every other device with connected 2nd display and Avrus for proper codec(s) used in its Web Interface / Dashboard! Write as much as possible new tests for all supported test types!"

Why the existing video-test set is insufficient. The legacy `test_video_routing.sh` (v33), `test_video_secondary_display.sh`, `test_video_playback_automation.sh` (v1) plus the 53 per_app_video wrappers all dispatch playback via `am start -a android.intent.action .VIEW` or via MediaSession `cmd media_session play` — surfaces that skip the app's own

content-selection UI, player-surface allocation, MediaSession lifecycle (start/pause/stop), and back-button cleanup. A defect like §CL routing-race or §CM frozen-frame mid-back-press cannot reproduce via Intent dispatch because Intent dispatch re-launches the app cold each time and `am force-stop` is a clean cleanup path that masks the leak.

Operative rule. Every video-playback test that asserts secondary-display routing MUST traverse the user-equivalent UI path. Mandatory 5-surface coverage per video app:

1. **Surface A — Launch via launcher icon.** `am start -n <launcher-pkg>/<launcher-activity>` then `uiautomator dump` → locate app icon tile → `input tap X Y`. NOT `am start <video-pkg>/<MainActivity>` (skips launcher).
2. **Surface B — Navigate to content list.** Per-app driver script knows the app's home-tab → content-tab swipe / tap sequence. Asserts at least one content tile is visible via `uiautomator dump` content-desc / text match.
3. **Surface C — Select + play.** Tap a specific content tile (driver records the picked tile's content-desc as captured evidence). Wait for player surface visible.
4. **Surface D — In-app control interaction.** Pause via in-app pause button (not `input keyevent KEYCODE_MEDIA_PAUSE`), resume via in-app play button, in-app rapid switch to a second video (covers §CL).
5. **Surface E — Stop via back button.** `input keyevent KEYCODE_BACK` to exit player. Assert §11.4.14 cleanup: VOM `activeDecoder == null`, secondary surface cleared, no orphan MediaSession.

App-coverage matrix: every video-capable app in PRODUCT_PACKAGES MUST have a UI-driven driver script set (Layer 2 per Section 7 design) AND at least one scenario that uses it. Today: 15 apps covered fully in the initial batch (VK Video, MPV, Lampa as fully-implemented; remaining 12 as templated stubs per Section 8). Expansion to all 53 video-app entries is queued as follow-up work under §O Issues.md per-app matrix.

Stream-type matrix: progressive HTTP / HLS / DASH / RTMP / file- local / DRM-protected. Per app, the driver script tags which stream types its picked content exercises. Aggregator emits a coverage matrix per cycle so gaps are visible.

Codec matrix: H.264 / H.265 / VP9 / AV1 / MPEG-2 / MP4V (video) + AC-3 / E-AC-3 / TrueHD / DTS / DTS-HD / MLP / Opus / AAC (audio). Per scenario, the codec assertion is made via `dumpsys media .metrics` (decoder name in event log) AND via Arvus codec-state probe (audio side) per §11.4.13.

Secondary-display verification. On any device with HDMI-A-1 attached (detected via `dumpsys display | grep "Display 2"`): - Captured-evidence: dual_display_record.sh ON for both displays per §11.4.5 - Assertion: `ffprobe -count_frames` on secondary mp4 reports > N frames within 3 s of play-action timestamp - Anti-assertion: `ffprobe -count_frames` on primary mp4 reports ONLY launcher/home pixels during playback window (no video-decoder output) - VOM cross-check: `dumpsys video_output_manager` shows `activeDecoder != null` during playback and `== null` after stop within 3 s

When secondary absent (D2 default topology): SKIP per §11.4.3 with explicit reason "topology: no secondary display attached". NEVER FAIL on missing topology.

Arvus codec-state mandate. For every test asserting an audio codec (AC-3, E-AC-3, TrueHD, DTS, etc.) — composed with §11.4.13 + §CG: - During playback window, call `arvus_probe_codec_state` ≥ 3 times at 1 s intervals (transients fade by sample 2-3) - Call `arvus_screenshot_capture` per §CG to attach visual evidence of the dashboard - Assert reported codec matches expected via `arvus_assert_codec_format <regex>` - ARVUS_HOST unreachable (ABK4 not joined / sink off) → SKIP per §11.4.3, never FAIL

Pre-build gates: - `CM-COVENANT-114-48-PROPAGATION` — asserts anchor literal present in every CLAUDE.md / AGENTS.md across parent + 10 owned submodules + HelixQA dependencies (42-file scan). - `CM-AF-UI-DRIVEN-VIDEO-COVERAGE` — asserts the directory `device/rockchip/rk3588/tests/ui_driven/` exists, contains `lib/ui_driver.sh` reference + scenarios/ subdir + per-app driver subdirs for at least the 3 reference apps (vk_video, mpv, lampa), and that `scripts/testing/run_ui_driven_video_suite.sh` is executable. - Paired mutations: deleting `ui_driver.sh` → CM-AF-UI-DRIVEN- VIDEO-COVERAGE FAILs; stripping the anchor literal → CM-COVENANT-114-48-PROPAGATION FAILs.

No escape hatch. No `--use-intent-shortcut` / `--skip-ui-traverse` / `--legacy-intent-mode` flag exists. The discipline exists because Intent-based dispatch is exactly the class of test that PASSes while users hit real defects (§11.4 forensic anchor). Operators who feel UI-driven tests are "too slow" should land scenario-level parallelism (multi-device fan-out) rather than bypass the discipline.

Composition. - §11.4.3 — topology dispatch (secondary present vs absent) - §11.4.5 — captured-evidence quality (every UI tap captured) - §11.4.13 — Arvus codec-state mandate - §11.4.14 — playback cleanup verified via UI back-button path - §11.4.43 — RED-test-first discipline (initial UI tests RED against §CL/§CM) - §11.4.44 — revision header on every plan/driver doc - §CG — Arvus dashboard screenshot

Classification: universal (per §11.4.17). Applies to every project that consumes the constitution submodule AND ships any video-capable Android app.

§11.4.49 — Dual-Approach Testing Mandate (User mandate, 2026-05-18)

Forensic anchor — verbatim user mandate (2026-05-18):

> "Kinopoisk playback of 5.1 audio supported movies MUST BE done via UI / UX full automation and via execution of Intents (or Broadcasts) directly both! We could have shared tests base and specialized part which will run / play the movie with properly chose audio track (5.1). Document everything up to the smallest details! Create as much as needed new tests for all supported test types and Challenges!"

Composition with §11.4.48. §11.4.48 mandated UI-driven traversal for every video routing test, eliminating the Intent-only PASS-bluffs. §11.4.49 REFINES rather than replaces: every feature test ships in BOTH variants — UI-driven AND Intent-driven — over a shared assertion base. Either alone is a §11.4 PASS-bluff for the OPPOSITE half of the stack. UI catches app-side bugs (content selection, player surface allocation, MediaSession lifecycle, in-app overlays, back-button cleanup).

Intent catches framework/system-server bugs (Intent extras parsing, MediaCodec.configure hook MIME routing, IVideoOutputManager bind timing, broadcast permission gating, headless cron automation paths).

Operative rule — 5 mandatory elements:

1. **Both variants required.** Every feature test exercising a user-visible behaviour MUST ship both `<feature>_ui.sh` (uiautomator-based, §11.4.48 surfaces A–E) AND `test_<feature>_ intent.sh` (`am start --es` / `am broadcast`-based). Either alone is forbidden.
2. **Shared assertion base.** Codec-state assertions, captured- evidence collection (screen-recording, ffprobe, RMS amplitude analysis), Arvus codec-state probe + dashboard screenshot, and §11.4.14 cleanup MUST be implemented in a single POSIX-sh library (`tests/lib/dual_approach_test_base.sh`) and reused by BOTH variants of every dual-approach test. Code duplication in the shared layer is forbidden.
3. **Specialised driver code.** UI variant uses §11.4.48 per-app driver scripts under `tests/ui_driven/<app>/`. Intent variant uses `am start --es content_id <id> --es audio_track_id <track>` / `am broadcast -a <action>` directly. Each variant's specialised layer is responsible ONLY for "how the playback gets started"; everything downstream of "playback is now happening" routes through the shared base.
4. **Comprehensive documentation per test.** Every dual-approach test set ships with: (a) §11.4.44 revision header in BOTH variant scripts, (b) per-feature contract under `docs/dual_approach/ <feature>.md` describing the captured-evidence pairing and operator-side verification, (c) Issues.md / Fixed.md entry cross-linking both variant paths.
5. **Kinopoisk 5.1 EAC3 is the canonical first implementation.** The shared base + Kinopoisk 5.1 UI variant + Kinopoisk 5.1 Intent variant land in the same batch as this mandate. Subsequent features (Netflix Dolby Atmos, MPV DTS-HD, Lampa+TorrServe HEVC, VLC FLAC) port to the same pattern. The §CN subagent's fix to the underlying decoder pipeline is the FIRST GREEN target of these tests; both variants are RED per §11.4.43 until §CN lands.

Captured-evidence directory contract. Both variants write to mirror-structured directories `qa-results/dual_approach/<F>/<run- ts>/{ui,intent}/`. Identical filenames; orchestrator diffs the two `result.json` files. Status mismatch (UI=PASS but Intent=FAIL or vice-versa) is itself a finding — it pinpoints which half of the stack contains the bug.

Pre-build gates: - `CM-COVENANT-114-49-PROPAGATION` — anchor literal in every CLAUDE.md / AGENTS.md across parent + 10 owned submodules + HelixQA dependencies (42-file scan). - `CM-AF-DUAL-APPROACH-COVERAGE` — `tests/lib/dual_approach_test_ base.sh` exists + sources anti_bluff / ui_driver / credentials + exports `dat_init` / `dat_start_capture` / `dat_assert_codec_state` / `dat_assert_video_frames` / `dat_assert_audio_channels` / `dat_arvus_dashboard_capture` / `dat_cleanup` / `dat_report_ finding`. - `CM-AF-KINOPOISK-5-1-DUAL-COVERAGE` — both `tests/ui_driven/kinopoisk/kinopoisk_5_1_play_movie_ui.sh` AND `tests/test_kinopoisk_5_1_play_movie_intent.sh` exist + both source the shared base + both reference EAC3 codec + both assert 5.1 / 6-channel. - Paired mutations (3): strip anchor → propagation gate FAILs; delete UI variant → coverage gate FAILs; delete Intent variant → coverage gate FAILs.

No escape hatch. No `--ui-only` / `--intent-only` / `--skip- dual` flag exists. The discipline exists because either-alone is the PASS-bluff pattern §11.4 specifically prohibits. Operators who feel dual variants are "too slow" should land scenario-level parallelism rather than bypass the discipline.

Composition. - §11.4.3 — topology dispatch (Arvus reachable, secondary present) - §11.4.5 — captured-evidence content-quality analysis - §11.4.6 — UNCONFIRMED tagging for un-verified Kinopoisk element IDs - §11.4.10 — credentials never echoed, per-device per-service loader - §11.4.13 — Arvus codec-state mandate (out-of-band evidence) - §11.4.14 — playback cleanup via EXIT trap in shared base - §11.4.17 — universal classification - §11.4.43 — RED-first TDD (both variants RED until §CN fix lands) - §11.4.44 — revision header on every variant script + design doc - §11.4.48 — UI-driven traversal (this MANDATE refines, not replaces) - §CG — Arvus dashboard screenshot via Presenter receiver - §CB — credentials loader

Classification: universal (per §11.4.17). Applies to every project that consumes the constitution submodule AND ships any Android app whose user-visible behaviour can be triggered through both UI and Intent/Broadcast paths.

§11.4.50 — Deterministic Consistency Mandate (User mandate, 2026-05-18)

Forensic anchor — verbatim user mandate (2026-05-18):

> "Our anti-bluff / bluff-proofed / proof-driven work MUST satisfy the following: not a single feature, System or application flow, use case, edge case or procedural action(s) MUST NOT partially work, or sometimes work and sometimes not! There is only one truth that MUST BE fulfilled - no matter how many times we repeat all these scenarios with variations in data, System state(s) and speed of execution, results MUST BE consistent and successful without exception! No false positives or bluffing of any kind is allowed! We MUST add as much full automation tests which will use real System and Applications UI and UX with all flows, use cases and edge cases as needed or as much as it is possible!"

Why this anchor exists. §11.4.7 already forbids the vocabulary `intermittent` / `transient` / `flake` in closure narratives, but that's a textual ban — operators could still let a test "PASS once, FAIL once, PASS again" and report only the first PASS. §11.4.50 closes that gap MECHANICALLY by requiring every PASS to come from N identical iterations rather than from a single observation.

Operative rule — 5 mandatory elements:

1. **N-iteration deterministic outcome.** Every test that PASSes MUST have been executed N times (default N=3, N=10 for cycle- validation suites) against the same firmware MD5 + same device
    - same topology and produced IDENTICAL PASS in every iteration. A test whose outcome diverges across iterations is auto-FAIL per this mandate — there is no "first PASSed therefore X was a flake" path. §11.4.7's expanded forbidden-vocabulary list is enforced mechanically here.

2. **Edge-case + flow + use-case coverage.** Every public API path (Activity / Service / Broadcast receiver / ContentProvider URI / IPC interface / JNI entry / sysprop write / sysfs node / init.rc trigger) MUST have ≥1 dedicated test that drives it. Untested paths surface in a feature-coverage-matrix audit and block release at the 99% threshold (ratchet sequence 70 → 85 → 95 → 99 mirrors §11.4.18 the existing project-side docs-coverage ratchet).

3. **Reliability check helper.** Project anti-bluff helper library ships `ab_run_n_times <test_name> <N> <fn> [args...]`:

   - Loops N times, captures exit code + evidence-hash per iter
   - Asserts all N exit codes identical AND all N evidence-hashes identical
   - On any divergence: `ab_fail` with full N-iteration report
   - Per §11.4.7 forbidden vocabulary: NO operator-facing escape path converts a divergent N-iter run into PASS

4. **Feature-coverage-matrix audit.** Project ships `scripts/testing/feature_coverage_audit.sh`:

   - Walks source layers (every public API surface)
   - Walks test layers (test_*.sh references)
   - Emits coverage report at `qa-results/feature_coverage/ <ISO-UTC>/coverage.tsv`
   - Pre-build gate enforces minimum threshold; threshold ratchets up each phase (70 → 85 → 95 → 99)

5. **Composes with every prior anti-bluff anchor.** §11.4.50 does NOT replace §11.4.43 / §11.4.48 / §11.4.49 — it adds the N-iter consistency dimension across all of them. RED-first TDD (§11.4.43) still applies: every new test starts RED and becomes GREEN after the fix lands. UI-driven (§11.4.48) and dual-approach (§11.4.49) still apply: every test ships in both variants AND each variant passes N iterations identically.

Pre-build gates:

- `CM-COVENANT-114-50-PROPAGATION` — anchor literal `§11.4.50` present in every CLAUDE.md / AGENTS.md across parent + 10 owned submodules + HelixQA dependencies (≥42-file scan; phase 1 uses 5-canonical-file scan matching §11.4.49 propagation pattern).
- `CM-AF-RELIABILITY-CHECK-WIRED` — `ab_run_n_times` function literal present in `device/rockchip/rk3588/tests/lib/ anti_bluff.sh` AND at least 3 on-device tests source-reference it. The 3-test floor ratchets upward each phase.
- `CM-AF-FEATURE-COVERAGE-MATRIX` — `scripts/testing/feature_coverage_audit.sh` exists + executable + has run within the last 7 days + most-recent coverage report meets the current threshold.

Paired mutations (3):

- Strip the `§11.4.50` anchor literal from `constitution/CLAUDE.md` via `sed -i 's|11\.4\.50|11.4.MUTATED|g'` → `CM-COVENANT-114-50-PROPAGATION` MUST FAIL.
- Rename `ab_run_n_times` to `ab_run_n_times_DISABLED` in `anti_bluff.sh` via targeted sed → `CM-AF-RELIABILITY-CHECK- WIRED` MUST FAIL.

- Move `scripts/testing/feature_coverage_audit.sh` aside via `mv` → `CM-AF-FEATURE-COVERAGE-MATRIX` MUST FAIL.

No escape hatch. No `--allow-flake`, `--first-pass-suffices`, `--skip-n-iter`, `--skip-coverage-audit` flag exists. The discipline exists because the user mandate is unambiguous: "results MUST BE consistent and successful without exception."

Composition. - §11.4.1 — FAIL-bluffs forbidden - §11.4.2 — captured-evidence (every iteration captures its own) - §11.4.5 — quality analysis (per-iteration content compared) - §11.4.6 — no guessing (UNCONFIRMED required for any iteration-budget assumption) - §11.4.7 — forbidden flake/intermittent vocabulary (enforced mechanically by this mandate) - §11.4.43 — RED-first TDD (every new test starts RED, N-iter applies across the GREEN transition) - §11.4.46 — validate-before-suite (N-iter applied at single-test scope first) - §11.4.48 — UI-driven traversal (each UI traversal MUST be N-iter-consistent) - §11.4.49 — dual-approach (both variants of every dual-approach test MUST pass N iterations identically)

Classification: universal (per §11.4.17). Applies to every project that consumes the constitution submodule.

---

§11.4.51 — Live-ADB-First Maximization Mandate (User mandate, 2026-05-18)

Forensic anchor — verbatim user mandate (2026-05-18):

> "Was it possible to max the fix and test it via ADB or it can be only done by making the fix and reflashing for validation and verification? If such option exists, we should always make the best of it! Making fix on live devices via ADB → Once done commit and push changes, rebuild System and reflash → Validate and verify everything (with the test we write as well). Make sure this idea(s) is (are) part of our root (constitution Submodule) Constitution, AGENTS.MD and CLAUDE.MD if they are not already! EVERY DETAIL IS IMPORTANT!!!! Not a single idea or idea's detail can be ignored, skipped, relativized or bluffed!"

§11.4.51 REFINES §11.4.43 step 2 ("LIVE-ADB-PROBE — try the fix on the running device first") with mechanical enforcement: a per-file- class decision matrix, a classifier helper, and a commit-message footer literal.

Operative rule (5 mandatory elements):

1. **Classify every fix** by rebuild-requirement using the project's per-file-class decision matrix. No guessing (§11.4.6). Every file in the diff cites the matrix row that classified it.

2. **If LIVE_ADB_TESTABLE,** the operator MUST apply the fix to the running device via the appropriate `adb` channel first (`adb push` for scripts, `setprop persist.*` for runtime properties, `mount -o remount,rw /vendor` for boot scripts, `pm install -r` for APKs built locally via gradle). Run the §11.4.43 RED test against the live-probed device, capture positive-evidence PASS,

THEN commit source-side + rebuild + reflash as belt-and-suspenders re-validation. Commit message footer MUST state `LIVE_ADB_VALIDATED: yes` with one-line description of the live-probe sequence.

3. **If REQUIRES_REBUILD,** the operator proceeds directly to source-side fix + rebuild + flash. Commit message footer MUST state `REQUIRES_REBUILD: <reason>` citing the matrix row that classified the file. Categories of REQUIRES_REBUILD: kernel, AOSP framework Java / AIDL, native C++ inside APEX, sepolicy, init.rc, `ro.*` build properties (immutable post-boot), `Android.bp` / `Android.mk` / `BoardConfig.mk`, XML resource overlays, codec XML inside APEX, ramdisk-bundled artifacts.

4. **If mixed batch,** commit footer states `LIVE_ADB_VALIDATED: partial` and enumerates per-file classification. Per §11.4.9 batching is preferred; live-probe the testable files first, then batch with the rebuild-required ones into a single rebuild.

5. **Helper script enforces classification mechanically.** The project provides `classify_fix_rebuild_requirement.sh` which walks `git diff --name-only`, looks up each file against the matrix, and emits the recommended commit-message footer. No operator-facing override flag converts an unclassified path to LIVE_ADB_TESTABLE by default — unmatched paths classify as `REQUIRES_REBUILD: unmatched-path` (safe default per §11.4.6).

Per-file-class decision matrix (canonical):

File class	Rebuild required?	Reason
`*test_*.sh` / `tests/lib/*.sh` (on-device)	NO	adb push to /data/local/tmp/tests/
Host-side test scripts	NO	host script — no device interaction
`atmosphere-*.sh` boot scripts	DEPENDS	adb push to /vendor/bin/ after remount
Markdown docs / Constitution / plans / Issues / Fixed / CONTINUATION	NO	host docs — no firmware impact
Forked-player Kotlin (Presenter, MPV, SmartTube, TorrServe, Lampa, VLC, etc.)	YES via gradle local build + `adb install -r`	APK re-link required but no full firmware rebuild
Framework Java (`frameworks/base/**/*.java`)	YES	system_server requires reboot at minimum
AIDL (`*.aidl`)	YES	stub generation requires recompile
Native C++ (`external/**/*.cpp`, `frameworks/native/**/*.cpp`)	YES	builds into APEX or system library
	YES	squashfs read-only by Mainline integrity

File class	Rebuild required?	Reason
APEX libraries (`vendor/**/lib/**/*.so` inside APEX path)		
Kernel sources (`kernel-5.10/**`)	YES	requires kernel rebuild + boot.img
sepolicy (`*.te`)	YES	requires policy build
`init.rc`	YES	parsed at boot only
`ro.*` build properties	YES	immutable post-boot
`persist.*` runtime properties	NO	`setprop persist.*` mutable
XML resource overlays	YES	RRO recompile
`media_codecs_*.xml`	YES	APEX-bundled
`display_settings.xml` (userdata)	DEPENDS	adb push possible if SELinux permits
`Android.bp` / `Android.mk` / `device.mk` / `BoardConfig.mk`	YES	rebuilt-into-image artifacts
Test fixture binary assets	NO	adb push to /data/local/tmp/

Pre-build gates: `CM-COVENANT-114-51-PROPAGATION` (anchor across canonical files) + `CM-AF-CLASSIFY-FIX-HELPER-EXISTS` (helper present + sentinel literals) + `CM-AF-LIVE-ADB-FIRST-COMMIT-MARKER` (advisory WARN that scans recent commits for the footer literal). Three paired meta-test mutations.

Composition. - §11.4.43 — TDD-fix workflow (§11.4.51 REFINES step 2 with the mechanical classifier). - §11.4.9 — batch-source-fixes-before-rebuild (compose: live-ADB- test individually, then batch-commit). - §11.4.6 — no guessing (each classification justified by matrix row; unmatched paths default to rebuild-required, never silent testable). - §11.4.46 — validate-recent-work-before-post-flash (live-probe IS implicit pre-validation). - §11.4.48 — UI-driven tests (live-probe CAN use uiautomator over adb shell). - §11.4.49 — dual-approach (live-probe applies to both UI and Intent variants). - §11.4.50 — deterministic consistency (live-probe runs N iterations on the live device before commit).

No escape hatch — no `--skip-classify`, `--assume-rebuild`, `--no-footer-required` flag exists. The discipline exists because the user mandate is unambiguous: “EVERY DETAIL IS IMPORTANT!!!! Not a single idea or idea’s detail can be ignored, skipped, relativized or bluffed!”

Classification: universal (per §11.4.17). Applies to every project that consumes the constitution submodule — the matrix itself MAY be project-specific (each consumer adapts the file-class list to its own source tree), but the mandate to classify-before-commit + the LIVE_ADB_VALIDATED / REQUIRES_REBUILD footer literals + the classify-helper-script + the three pre-build gates are universal.

§11.4.52 — Autonomous-Validation Mandate (User mandate, 2026-05-18)

Forensic anchor — verbatim user mandate (2026-05-18):

> "Make sure we have full automation tests which will do all this work in full automation! IMPORTANT: Make sure that all existing tests and Challenges do work in anti-bluff manner — they MUST confirm that all tested codebase really works as expected! We had been in position that all tests do execute with success and all Challenges as well, but in reality the most of the features does not work and can't be used! This MUST NOT be the case and execution of tests and Challenges MUST guarantee the quality, the completition and full usability by end users of the product! This MUST BE part of Constitution of our project, its CLAUDE.MD and AGENTS.MD if it is not there already, and to be applied to all Submodules's Constitution, CLAUDE.MD and AGENTS.MD as well."

Why this anchor exists. §11.4.25 (full-automation coverage) and §11.4.27 (no-fakes-beyond-unit-tests + 100%-test-type-coverage) and §11.4.39 (per-feature on-device end-user validation) collectively mandated that every user-facing feature has automation tests with captured runtime evidence. They did NOT explicitly forbid operator-attended-only validation paths — i.e., a feature whose ONLY proof of working state requires a human to physically present at the device, drive UI manually, and observe outcomes. Phase 39.§CN exposed the gap: §CN's Kinopoisk 5.1 EAC3 closure was structurally fixed (libavutil bundled in APEX, six decoders register) yet end-user-validation degraded to "operator drives remote control while test polls" because Kinopoisk's TV-Compose UI is not accessibility-instrumented and uiautomator returns near-empty hierarchy. That validation path is operator-attended, non-reproducible in CI, and cannot prove deterministic consistency (§11.4.50) across N iterations. The User mandate elevates "full automation" from "preferred" to "mandatory": every user-facing PASS MUST have at least one captured-evidence path that does NOT require operator presence.

Operative rule (4 mandatory elements):

1. **Every user-facing feature MUST have at least one autonomous validation path.** "Autonomous" means: the validation runs end-to-end via `adb shell` + scripted automation, produces captured runtime evidence per §11.4.5, and reaches a PASS / FAIL verdict WITHOUT a human present to drive UI, observe screen, or make decisions. Operator-attended tests are SUPPLEMENTARY, never PRIMARY. A feature whose ONLY validation path is operator-attended is a §11.4.52 violation regardless of how carefully the operator's captures match §11.4.5 evidence requirements — the path does not scale to CI, does not run on every commit, does not survive operator unavailability, and produces the exact "tests pass but feature doesn't work for users" failure mode §11.4 specifically forbids.

2. **Acceptable autonomous-validation paths** (any one or more — non-exhaustive):

    - **Programmatic instrumentation APK** — a small platform-signed app that exercises the under-test surface directly via SDK APIs (e.g., `MediaCodec.createDecoderByName`, `MediaCodec.configure`, `AudioTrack` writes) and writes a structured JSON result file to a

discoverable path. Closes UI-instrumentation gaps where the target app is not accessibility-instrumented.
    - **Headless intent dispatch + state poll** — `am start --es` / `am broadcast --es` to dispatch the under-test code path, then poll `dumpsys`, `/proc/<pid>/maps`, `media.metrics`, `/sys/...`, kernel sysfs, or the integration's network probe for the expected state transition.
    - **ADB-driven uiautomator** — applicable ONLY if the target app exposes accessibility nodes the dump can resolve. MUST verify via `uiautomator dump | grep -c clickable=true` ≥ 1 before claiming uiautomator coverage is real. A near-empty hierarchy is itself the captured-evidence proving UI-driven automation is INFEASIBLE for that target and the test MUST fall back to instrumentation-APK or headless-intent paths.
    - **Network-side sink probe** — for features that surface on a network-accessible peripheral (e.g., Arvus HDMI dashboard at `http://<sink>/`, Sonos REST API, AirPlay receiver), the sink's own report counts as autonomous out-of-band evidence per §11.4.13.
    - **HelixQA autonomous QA session** — for projects that have HelixQA fully incorporated per §11.4.27, the orchestrator's end-to-end session output counts as autonomous evidence.

3. **Per-feature classification + tracking.** Every entry in the project's coverage ledger (§11.4.25) MUST classify each feature's autonomous-validation path as one of: `AUTONOMOUS_VERIFIED` (path exists + last-run-green per §11.4.46), `AUTONOMOUS_DESIGNED` (path exists but RED per §11.4.43), `OPERATOR_ATTENDED_ONLY` (path requires human presence — release blocker until promoted to one of the other states), or `NOT_APPLICABLE` (e.g., a CLI tool that has no UI surface and the unit + integration tests already constitute autonomous proof). `OPERATOR_ATTENDED_ONLY` rows MUST cite a tracked work item per §11.4.15 + §11.4.16 that designs the autonomous-path migration. Closing the work item requires the migration to land, not just the operator capture.

4. **Anti-bluff for autonomous paths themselves.** An autonomous path that returns PASS without exercising the user-visible behaviour is a §11.4.52 violation just as severe as the original §11.4 PASS-bluff pattern. The autonomous path MUST produce positive captured evidence per §11.4.5 (audio: channel count + sample rate + glitch census; video: frame count + routing target

    - frame health + obstruction census). A grep on a metadata field that doesn't prove behaviour is not an autonomous validation — it is a metadata bluff dressed up as automation. Paired meta-test mutations (§1.1) MUST catch the autonomous path's degradation to metadata-only or grep-only.

Composition. - §11.4.25 — full-automation-coverage mandate. §11.4.52 is the strict expansion of invariant 1 + 2: "anti-bluff posture" and "proof of working capability" cannot be carried by operator-attended-only evidence — they MUST be carried by at least one autonomous path. - §11.4.27 — no-fakes-beyond-unit-tests + 100%-test-type-coverage. §11.4.52 is the operational layer that closes the gap between "the project has full-automation tests" and "every PASS has an autonomous evidence path". - §11.4.39 — per-feature on-device end-user validation. §11.4.52 refines: the on-device scenario MUST itself be autonomously drivable. - §11.4.43 — TDD-fix-discipline. The autonomous path MUST exist in the RED state BEFORE the fix lands, then go GREEN after. An autonomous path authored after the fix is a §11.4 PASS-bluff. - §11.4.48 — UI-driven video testing. §11.4.52 establishes that when uiautomator returns a near-empty hierarchy on the target app, UI-driven IS

infeasible and the test MUST fall back to instrumentation-APK or headless-intent. The fallback is REQUIRED — not optional. - §11.4.49 — dual-approach testing. The Intent variant in a dual-approach pair often IS the autonomous path when the UI variant requires operator assistance. - §11.4.50 — deterministic consistency. Autonomous paths run N iterations naturally; operator-attended paths cannot satisfy the N-iteration discipline at scale. - §11.4.51 — live-ADB-first maximization. The instrumentation-APK path is itself LIVE_ADB_TESTABLE (gradle build + `adb install -r` + run + read JSON result), so live-probe IS the design loop for the autonomous path.

Pre-build gates: - `CM-COVENANT-114-52-PROPAGATION` — anchor literal `§11.4.52` present in all 5 canonical files (`constitution/Constitution.md`, `constitution/CLAUDE.md`, `constitution/AGENTS.md`, plus the parent consumer's `CLAUDE.md` + `AGENTS.md`) PLUS the full per-consumer propagation (parent project + every owned submodule + nested submodules + HelixQA dependencies — same scan pattern as `CM-COVENANT-114-51-PROPAGATION`). - `CM-AF-AUTONOMOUS-PATH-PER-FEATURE` — every entry in the consuming project's coverage ledger (§11.4.25) has a non-empty autonomous-classification column with value in `{AUTONOMOUS_VERIFIED, AUTONOMOUS_DESIGNED, OPERATOR_ATTENDED_ONLY, NOT_APPLICABLE}`. `OPERATOR_ATTENDED_ONLY` rows MUST cite a tracked migration work item. (Project-side gate — consuming projects implement the ledger surface in their own pre-build test suite.)

Paired mutations (per §1.1): - Strip `§11.4.52` literal from `constitution/Constitution.md` → `CM-COVENANT-114-52-PROPAGATION` FAILs. - Inject a feature row with `OPERATOR_ATTENDED_ONLY` and NO tracked migration work item → `CM-AF-AUTONOMOUS-PATH-PER-FEATURE` FAILs.

No escape hatch. No `--allow-operator-attended-only`, `--skip-autonomous-path`, `--manual-validation-suffices` flag exists. The discipline exists because the user mandate is unambiguous: "execution of tests and Challenges MUST guarantee the quality, the completition and full usability by end users". Operator-attended-only validation cannot guarantee that property at the release-gate layer — only autonomous paths can.

Classification: universal (per §11.4.17). Applies to every project consuming the constitution submodule. Each consumer adapts the autonomous-validation path catalogue to its own technology surface (Android: instrumentation APK + uiautomator + headless intent; Web: Playwright headless + API probe; CLI: subprocess + stdout/stderr assertion; mobile native: device farm headless runner) but the mandate to provide at least one autonomous path per user-facing feature is universal.

---

§11.4.53 — Fixed_Summary parity mandate (User mandate, 2026-05-18)

Forensic anchor — verbatim user mandate (2026-05-18T17:55Z):

> "Note: Just like for Issues we have Issues_Summary, for Fixed we MUST HAVE Fixed_Summary - like all other docs: ALWAYS in sync and up to date and ALWAYS exported into the PDF and HTML! Add this mandatory rule / constraint into the root (constitution Submodule) Constitution, AGENTS.MD and CLAUDE.MD."

Why this anchor exists. §11.4.12 already established that `docs/Issues_Summary.md` is the canonical short-form summary of `docs/Issues.md` and MUST always be regenerated + re-exported (HTML + PDF) whenever the source changes. §11.4.19 added the column- alignment requirement so `Fixed_Summary.md` mirrors `Issues_Summary.md` shape. §11.4.53 closes the propagation gap: the parity discipline that §11.4.12 imposes on Issues / Issues_Summary MUST apply symmetrically to Fixed / Fixed_Summary. The mechanical behaviour already exists in `scripts/testing/sync_issues_docs.sh` (lines 98- 105 regenerate Fixed_Summary in stage 1b alongside Issues_Summary in stage 1a), but the mandate was never explicit in the Constitution — and an unstated rule is a §11.4 PASS-bluff risk: future refactors of `sync_issues_docs.sh` could drop the Fixed_Summary half without violating any canonical authority. §11.4.53 makes the existing behaviour explicit canonical authority so the gate set can defend it.

Operative rule. `docs/Fixed_Summary.md` is the symmetric short- form summary of `docs/Fixed.md`. It MUST be regenerated whenever `Fixed.md` changes. Its HTML + PDF exports MUST travel with the markdown (identical mtimes within sync_issues_docs.sh granularity). Stale exports are §11.4.53 violations regardless of whether the underlying `.md` is correct — an operator (or future agent) reading the HTML or PDF gets a divergent view of which items are closed, and the §12.10 CONTINUATION resumption guarantee silently breaks. Same discipline as §11.4.12 Issues_Summary applied to Fixed.md.

Generator. `scripts/testing/generate_fixed_summary.sh` is the canonical generator. It MUST exist + be executable + emit a markdown table whose header columns include `Status` and `Type` (per §11.4.19 column-alignment) so the `Fixed_Summary.md` shape mirrors `Issues_Summary.md` exactly.

Auto-sync wrapper. `scripts/testing/sync_issues_docs.sh` is the single operator-facing entry point. It already regenerates BOTH Issues_Summary AND Fixed_Summary in one shot (stages 1a + 1b), then exports HTML + PDF (stage 2), then colorizes per §11.4.23 (stage 3), then re-renders the PDFs from the colorized HTML (stage 3b). MUST be invoked after any edit to `Fixed.md` (just as §11.4.12 requires after any edit to `Issues.md`). Manual invocation of just the Issues half (`--issues-only`-style flag) is FORBIDDEN — the wrapper has no such flag and §11.4.53 prohibits adding one.

Sort order. Same pattern as §11.4.12 — by closure date DESC (most-recent-Fixed first), with §-letter / Fix-# secondary sort. Documented at the top of the generated file.

HTML + PDF travel-together. Per §11.4.12 + §11.4.44, the three file types (`.md`, `.html`, `.pdf`) MUST always have identical mtimes within sync_issues_docs.sh granularity. Stale exports violate §11.4.53 regardless of whether the underlying `.md` is correct.

Composition: - §11.4.12 — Issues_Summary parity is the sibling rule §11.4.53 symmetrizes. §11.4.12 and §11.4.53 are the canonical pair: edits to one source-of-truth (`Issues.md` or `Fixed.md`) trigger regeneration of BOTH summaries via the same wrapper. - §11.4.19 — atomic Issues→Fixed migration triggers Fixed_Summary regeneration. When an item closes (status `Fixed (→ Fixed.md)` / `Implemented (→ Fixed.md)` / `Completed (→ Fixed.md)` per §11.4.33), the operator moves the entry from `Issues.md` to `Fixed.md` atomically and runs `sync_issues_docs.sh` — both summaries refresh. - §11.4.23 — visual-cue & grouping colorizer post-processes both `Issues_Summary.html` AND

Fixed_Summary.html. Stale Fixed_Summary.html would carry pre-§11.4.23 styling — itself a §11.4.53 violation. - §11.4.33 — type-aware closure-status vocabulary. Fixed_Summary MUST respect the three terminal values (Fixed (→ Fixed.md), Implemented (→ Fixed.md), Completed (→ Fixed.md)) and emit them literally in the Status column. - §11.4.44 — revision-header mandate applies to Fixed_Summary.md exactly as it applies to every other tracked Markdown doc. - §12.10 — CONTINUATION.md resumption guarantee depends on the divergent-summary problem NOT existing; §11.4.53 is the explicit symmetric closure of that risk for the Fixed half.

Pre-build gates:

- CM-FIXED-SUMMARY-SYNC — canonical artifact-level gate (6 invariants): (1) docs/Fixed_Summary.md exists; (2) docs/Fixed_Summary.html exists with mtime ≥ Fixed_Summary.md mtime; (3) docs/Fixed_Summary.pdf exists with mtime ≥ Fixed_Summary.md mtime; (4) Fixed_Summary.md mtime ≥ Fixed.md mtime; (5) scripts/testing/generate_fixed_summary.sh exists + executable; (6) scripts/testing/sync_issues_docs.sh invokes the generator. Pre-build FAILs if any invariant is violated. Partially overlaps with CM-FIXED-COLUMN-ALIGNMENT (§11.4.19 Phase 39.AV gate) and CM-DOCS-EXPORT-SYNC (§11.4.15 4-doc + 6-export scan), but is explicitly the Fixed_Summary derived-doc parity invariant for §11.4.53 enforcement.

- CM-COVENANT-114-53-PROPAGATION — anchor literal §11.4.53 present across all 5 canonical files (constitution/Constitution.md, constitution/CLAUDE.md, constitution/AGENTS.md, parent consumer's CLAUDE.md + AGENTS.md). Same propagation pattern as CM-COVENANT-114-51-PROPAGATION and CM-COVENANT-114-52-PROPAGATION. Consumer projects propagate further (owned submodules, nested submodules, HelixQA deps) at their own per-project gate granularity.

Paired mutations (per §1.1):

- Strip §11.4.53 literal from constitution/CLAUDE.md → CM-COVENANT-114-53-PROPAGATION FAILs.
- Move scripts/testing/generate_fixed_summary.sh aside (so the invariant-(5) executable-check fails) → CM-FIXED-SUMMARY-SYNC FAILs.
- Touch docs/Fixed_Summary.md to a date older than docs/Fixed.md → invariant-(4) violated → CM-FIXED-SUMMARY-SYNC FAILs.

No escape hatch. No --skip-fixed-summary-sync, --issues-only, --summary-not-applicable flag exists. The discipline exists because the user mandate is unambiguous: "like all other docs: ALWAYS in sync and up to date and ALWAYS exported into the PDF and HTML". A divergent Fixed_Summary breaks operator + AI-agent ability to discover what is fixed vs what is still open — and that breakage IS the §11.4 "tests pass but reality differs" pattern at the documentation layer.

Classification: universal (per §11.4.17). Applies to every project consuming the constitution submodule that maintains an Issues.md / Fixed.md split (the Issues-tracking lifecycle is already a §11.4.15 universal mandate, so this clause inherits the same universal scope). Projects that do not maintain a Fixed.md (e.g., single-binary throwaway prototypes) are NOT_APPLICABLE; all projects that DO maintain a Fixed.md MUST land both gates and mutations within one cycle of adopting this Constitution version.

§11.4.54 — ATM-NNN ticket identifier mandate (User mandate, 2026-05-19)

Forensic anchor — verbatim user mandate (2026-05-19):

> "Every workable item in Issues.md / Issues_Summary.md / Fixed.md / Fixed_Summary.md MUST carry a stable, unique, auto-incremental ATM-NNN ticket identifier. ATM- prefix, monotonic, never renumbered, append-only — once assigned to an item it stays bound to that item for the lifetime of the project across every reopen, migration, or rename."

Why this anchor exists. §11.4.15 + §11.4.16 + §11.4.19 + §11.4.33 established the lifecycle vocabulary for tracked items, but the items themselves were identified only by §-letter or Fix-# — both of which are **unstable across migrations**. §-letter is heading-local; when an item moves from Issues.md to Fixed.md the §-letter may be reused for a different item. Fix-# is only assigned on closure for items that touched source code; tasks / features / pure-doc items have no stable identifier at all. Cross-references between commits, test logs, captured-evidence artifacts, Reopens.md history (§11.4.55), README.md doc-link rows (§11.4.57), and external systems (Firebase Crashlytics Issue-IDs, Atlassian-style tracker rows) all need an identifier that is **monotonic, stable, and project-wide unique**. §11.4.54 closes that gap.

Operative rule. Every workable item in `docs/Issues.md` AND `docs/Fixed.md` MUST carry a `[ATM-NNN]` ticket identifier in its heading, in the form `## §X.Y. [ATM-NNN] <title>` (or `### §X.Y.` for nested items). NNN is a positive integer, zero-padded to at least 3 digits (`ATM-001`, `ATM-042`, `ATM-127`, …). Identifiers are allocated by a **canonical helper script** (`scripts/testing/assign_atm_ticket_ids.sh`) that maintains an append-only state file (`scripts/testing/.atm_ticket_state.json`). Once assigned to an item, an ATM-NNN MUST NEVER be:

1. **Renumbered** — the bind is permanent.
2. **Reused** for a different item even if the original item is force-deleted (the helper's state file detects collisions).
3. **Decremented** — counter is monotonic-only.
4. **Skipped intentionally** — gaps are forbidden in the assignment sequence (gaps indicate state-file corruption and trigger gate FAIL).

State file format. `scripts/testing/.atm_ticket_state.json` is a JSON-lines file with one record per allocated ID:

```
{"atm_id": "ATM-001", "heading_hash": "<sha256 of normalized
heading>",
 "type": "Bug|Feature|Task", "current_location": "Issues|Fixed",
 "current_status": "<lifecycle value>", "reopens_count": <int>,
 "created_at": "<ISO 8601 UTC>", "last_modified": "<ISO 8601
UTC>"}
```

The `heading_hash` is the canonical binding key — when the helper scans Issues.md / Fixed.md, it computes the normalized hash of each heading and looks up an existing ATM-NNN before allocating a new one. This protects against accidental

renumbering on heading reflows / wording edits (the hash MUST stay stable; if the heading text changes substantially, the state file's `heading_hash` field is updated in place via an explicit migration command, NOT auto- rebound).

Summary-table column. `docs/Issues_Summary.md` and `docs/Fixed_Summary.md` MUST carry a leftmost `ATM ID` column showing the identifier. Generators (`generate_issues_summary.sh`, `generate_fixed_summary.sh`) MUST emit this column as the first data column so operators / agents can sort + filter on it.

Composition:

- §11.4.15 (Status), §11.4.16 (Type), §11.4.19 (column-alignment Issues⍰Fixed), §11.4.33 (type-aware closure terminal values). All four pre-existing mandates compose with §11.4.54 — the ATM-NNN is the new universal join key.
- §11.4.12 + §11.4.53 (Issues_Summary + Fixed_Summary regen on source changes) — the helper MUST be invoked from `sync_issues_docs.sh` so newly added items receive ATM-NNN identifiers before summary regeneration.
- §11.4.55 (Reopens-history per-item docs) — the per-item Reopens.md lives at `docs/issues/<ATM-NNN>/Reopens.md`.
- §11.4.57 (README.md doc-link section) — every Issues / Fixed item may be linked by ATM-NNN in external references.
- §11.4.44 (revision header) — applies to the state file's audit log if one is maintained.

Pre-build gates:

- `CM-ATM-TICKET-IDS-COMPLETE` — every workable-item heading in Issues.md AND Fixed.md MUST carry an `[ATM-NNN]` token matching `\[ATM-[0-9]{3,}\]`. Headings without one FAIL the gate.
- `CM-ATM-TICKET-IDS-UNIQUE` — no two items share an ATM-NNN. The state file's record count MUST equal the union of Issues.md + Fixed.md `[ATM-NNN]` occurrences (no orphan IDs, no duplicates).
- `CM-ATM-TICKET-IDS-MONOTONIC` — the maximum ATM-NNN in the state file MUST equal `record_count` (no gaps).
- `CM-COVENANT-114-54-PROPAGATION` — anchor literal `§11.4.54` present across canonical files (`constitution/Constitution.md`, `constitution/CLAUDE.md`, `constitution/AGENTS.md`, parent consumer's `CLAUDE.md` + `AGENTS.md`).

Paired mutations (per §1.1):

- Remove `[ATM-NNN]` from one Issues.md heading → `CM-ATM-TICKET-IDS-COMPLETE` FAILs.
- Duplicate an ATM-NNN in the state file → `CM-ATM-TICKET-IDS-UNIQUE` FAILs.
- Delete ATM-NNN=2 from a 5-record state file → `CM-ATM-TICKET-IDS-MONOTONIC` FAILs (gap at 2).
- Strip `§11.4.54` literal from `constitution/CLAUDE.md` → `CM-COVENANT-114-54-PROPAGATION` FAILs.

No escape hatch. No `--skip-atm-assignment`, `--renumber`, or `--no-atm-id-required` flag exists. The discipline exists because unstable item identity is the exact failure mode that produces "the test was passing last week, what changed?" sessions where the operator cannot tell whether a given item is the same one tracked before or a coincidentally similarly-named successor.

Classification: universal (per §11.4.17). Applies to every project consuming the constitution submodule that maintains an Issues / Fixed split (cf. §11.4.53 scoping). Projects that do not maintain such a split are NOT_APPLICABLE.

---

§11.4.55 — Reopens-history tracking + per-item Reopens.md doc (User mandate, 2026-05-19)

Forensic anchor — verbatim user mandate (2026-05-19):

> "Add a Reopens-count column to Issues / Issues_Summary / Fixed / Fixed_Summary. For any item whose reopens-count > 0, create docs/issues/ATM-NNN/Reopens.md (+ HTML + PDF) with comprehensive reopen history + Fixed cycles. Each reopen MUST include By (AI / User), On (date), Reason, Evidence, and each Fixed-marking with its reasoning chain."

Why this anchor exists. §11.4.34 mandated that Reopened status carry a `**Reopened-Details:**` block — but that block lives in the **current** Issues.md heading and is overwritten on the next state change. A complete reopen / fix-cycle audit trail across the lifetime of an item was not previously canonical. §11.4.55 makes the full history first-class: every item that has been reopened at least once gets a dedicated `Reopens.md` companion doc capturing the entire chronological timeline — every reopen, every closure, every reasoning chain — so an operator / agent investigating "why was this reopened five times" has a single, authoritative source to read.

Operative rule. Every workable item with `reopens_count > 0` MUST have a companion document at `docs/issues/ATM-NNN/Reopens.md` (plus its HTML + PDF exports per §11.4.44 + §11.4.53 sync discipline). The document MUST contain:

1. **Revision header** per §11.4.44 (`Revision` integer + `Last modified` ISO 8601 UTC).
2. **Item identification** — `ATM ID`, `Title`, `Type`, `Current Status`, `Current Location` (Issues.md / Fixed.md), link back to the live heading.
3. **Cycle counters** — `Total reopens`, `Total fixed cycles` (items can reopen multiple times, each followed by a re-closure).
4. **Chronological timeline** — one entry per state-change event, each entry capturing:
   - **By:** `AI` or `User` (per §11.4.34 source attribution).
   - **On:** ISO date.
   - **Event:** `Opened` | `Reopened` | `Fixed` | `Implemented` | `Completed`.
   - **Reason:** the closed-vocabulary value from §11.4.34 (`test-failed` / `manual-testing-detected` / `captured-evidence-contradicts` / `end-user-report` / `cycle-re-discovered` / `design-reconsidered`) for reopens; captured-evidence summary for closures.
   - **Evidence:** path to or short description of captured artefact (test log path, recording path, sink-probe report, operator quote).

    - **Outcome:** what the next state transition was.
5. **Reasoning chain for each closure** — for every `Fixed` / `Implemented` / `Completed` event, the reasoning chain that justified marking it closed (root-cause analysis link, captured-evidence under same-conditions per §11.4.7, gate / mutation pair that defends against regression).
6. **Most-recent state-change pointer** — explicit cross-reference to the current Issues.md / Fixed.md heading.

Summary-table column. Issues_Summary.md and Fixed_Summary.md MUST carry a `Reopens` column showing the integer count. When the count > 0 the cell MUST be a hyperlink to `docs/issues/<ATM-NNN>/Reopens.md`. Generators emit the column; the colorizer (§11.4.23) MAY apply a visual cue when reopens > 2 (repeated-reopen signal — investigate before closing again).

Composition:

- §11.4.34 (Reopened-Details on current heading) is the **per-event** capture; §11.4.55 is the **per-item history** aggregation. Both layers MUST be in sync — the state-machine helper updates Reopens.md on every Reopened-Details parse.
- §11.4.54 (ATM-NNN) provides the stable identifier so the per- item doc has a permanent path even after Issues→Fixed migration.
- §11.4.44 (revision header) applies to every Reopens.md.
- §11.4.45 (Status.md per-integration) — the per-item Reopens.md is the item-scoped analogue of the integration-scoped Status.md.
- §11.4.53 (Fixed_Summary parity) — Reopens column appears on BOTH summary tables symmetrically.

Pre-build gates:

- `CM-REOPENS-DOC-EXISTS-WHEN-COUNT-GT-ZERO` — for every state-file record with `reopens_count > 0`, `docs/issues/<ATM-NNN>/ Reopens.md` MUST exist (+ HTML + PDF). Missing doc FAILs.
- `CM-REOPENS-DOC-REVISION-HEADER` — every Reopens.md carries the §11.4.44 revision header.
- `CM-REOPENS-COL-IN-SUMMARIES` — both Issues_Summary.md and Fixed_Summary.md carry a `Reopens` column.
- `CM-COVENANT-114-55-PROPAGATION` — anchor literal `§11.4.55` across canonical files.

Paired mutations (per §1.1):

- Delete `docs/issues/ATM-NNN/Reopens.md` for an item with `reopens_count=2` → `CM-REOPENS-DOC-EXISTS-WHEN-COUNT-GT-ZERO` FAILs.
- Strip revision header from a Reopens.md → `CM-REOPENS-DOC-REVISION-HEADER` FAILs.
- Remove `Reopens` column from Issues_Summary.md table header → `CM-REOPENS-COL-IN-SUMMARIES` FAILs.
- Strip `§11.4.55` literal from `constitution/CLAUDE.md` → `CM-COVENANT-114-55-PROPAGATION` FAILs.

No escape hatch. No `--skip-reopens-doc`, `--collapse-history`, `--reopens-not-applicable` flag. The discipline exists because loss of reopen history is the exact failure mode that produces “why does this keep happening” sessions with no auditable answer.

Classification: universal (per §11.4.17). Applies wherever §11.4.34 applies (every project tracking Reopened status).

---

§11.4.56 — Status_Summary parity + two-audience format (User mandate, 2026-05-19)

Forensic anchor — verbatim user mandate (2026-05-19):

> "Every Status.md doc gets a Status_Summary parity companion ALWAYS in sync, exported to HTML + PDF. Two-page format: page 1 = non-developer audience (team-specific: audio team for audio Status, video team for video Status, etc.), page 2 = software engineer summary. Auto-generated after every main Status update."

Why this anchor exists. §11.4.45 established `Status.md` per integration domain — but Status.md targets engineers (cites gate names, §-letter references, commit hashes, captured-evidence file paths). Non-developer stakeholders (audio team, video team, QA leads, product reviewers) need a plain-language version of the same content, **always derived** from the engineering Status.md so the two views can never diverge. §11.4.56 introduces the symmetric companion doc with a two-audience layout.

Operative rule. For every `docs/<domain>/<integration>/ Status.md` that satisfies §11.4.45, a companion `docs/<domain>/<integration>/ Status_Summary.md` MUST exist with:

1. **Revision header** per §11.4.44.
2. **Page 1 — For the `<team>`** (audience-specific heading; the team name is derived from the integration domain — audio team for `docs/dolby/*`, video team for `docs/video/*`, etc.). Page 1 contains:
   - Plain-language summary of current state (1-3 paragraphs).
   - **What works** (1-3 bullets, end-user-visible behaviour).
   - **What's broken or pending** (1-3 bullets).
   - **Operator / team actions** if any.
   - NO code references, NO §-letter jargon, NO captured-evidence file paths, NO gate / mutation names.
3. **Page 2 — For software engineers** — same content density as Status.md but condensed:
   - §-letter references, gate names, commit hashes, captured- evidence file paths.
   - Cross-references to Issues.md / Fixed.md items by ATM-NNN (§11.4.54).
4. **HTML + PDF exports** travel with the markdown (identical mtimes within sync wrapper granularity).
5. **Auto-generation** — the canonical helper `scripts/testing/ generate_status_summary.sh <Status.md path>` produces both pages. The generator MUST be invoked from a sync wrapper (e.g. `scripts/testing/ sync_integration_status.sh` per §11.4.45) every time the source Status.md is updated.

Page-1 audience derivation. The generator maps domain → audience name via a project-controlled mapping table (e.g. `audio` → `Audio team`, `video` → `Video team`, `network` → `Network team`, `drm` → `DRM / streaming team`). Unmapped domains default to `Project team`. The mapping table lives in the project's generator config, NOT in the Constitution (project-specific data per §11.4.17).

Composition:

- §11.4.45 (Status.md per integration) — Status_Summary.md COMPLEMENTS, never replaces. Both files exist together.
- §11.4.12 + §11.4.53 (parity discipline) — Status_Summary follows the same regeneration / HTML+PDF / mtime-alignment pattern.
- §11.4.44 (revision header) — applies to Status_Summary.md.
- §11.4.23 (colorizer) — Status_Summary.html receives the same type/status visual cues if it embeds tracked-item references.
- §12.10 (CONTINUATION.md) — non-developer stakeholders read Status_Summary; engineers read Status.md + CONTINUATION.md + Issues.md.

Pre-build gates:

- `CM-STATUS-SUMMARY-EXISTS-FOR-EVERY-STATUS` — for every `docs/**/Status.md`, the sibling `Status_Summary.md` MUST exist (+ HTML + PDF) with mtime ≥ Status.md mtime − tolerance.
- `CM-STATUS-SUMMARY-TWO-AUDIENCE` — every Status_Summary.md contains BOTH the `Page 1 — For the <team>` heading AND the `Page 2 — For software engineers` heading. Missing either FAILs.
- `CM-STATUS-SUMMARY-REVISION-HEADER` — §11.4.44 header present.
- `CM-COVENANT-114-56-PROPAGATION` — anchor literal across files.

Paired mutations (per §1.1):

- Delete a Status_Summary.md → `CM-STATUS-SUMMARY-EXISTS-FOR-EVERY-STATUS` FAILs.
- Remove `Page 1` heading from a Status_Summary.md → `CM-STATUS-SUMMARY-TWO-AUDIENCE` FAILs.
- Strip `§11.4.56` literal from `constitution/CLAUDE.md` → `CM-COVENANT-114-56-PROPAGATION` FAILs.

No escape hatch. No `--skip-status-summary`, `--engineer-only`, `--no-audience-split` flag. Plain-language stakeholder access is not negotiable — the project ships to humans, not just to engineers.

Classification: universal (per §11.4.17). Applies wherever §11.4.45 applies (every project that maintains domain-scoped Status.md docs).

---

§11.4.57 — README.md doc-link section + revision metadata (User mandate, 2026-05-19)

Forensic anchor — verbatim user mandate (2026-05-19):

> "Add a doc-link section to README.md — links to Issues + Issues_Summary + Fixed + Fixed_Summary + CONTINUATION + ALL Status docs + their exports. Each link shows revision + last-modified."

Why this anchor exists. Operators, AI agents, and external reviewers arriving at the project repo for the first time need a single discoverable entry point listing every canonical tracked- items + status document, along with each document's freshness (revision + last-modified). Without this surface, agents resort to `find docs -name '*.md'` which returns hundreds of files unranked. §11.4.57 makes the README.md the canonical index.

Operative rule. Every project's top-level `README.md` MUST contain a section titled `Tracked-Items + Status Documents` (or the project's localized equivalent — the section heading MUST contain the literal `Tracked-Items` for gate detection). The section MUST be a markdown table with columns: `Document` (human- readable name), `Last modified` (ISO 8601 UTC from the doc's §11.4.44 revision header), `Revision` (integer from same header), `Markdown` (relative link to `.md`), `HTML` (relative link to `.html`), `PDF` (relative link to `.pdf`).

The section MUST link to:

1. `Issues.md`, `Issues_Summary.md` (§11.4.12, §11.4.15, §11.4.16).
2. `Fixed.md`, `Fixed_Summary.md` (§11.4.19, §11.4.53).
3. `CONTINUATION.md` (§12.10).
4. **Every** `docs/**/Status.md` and its `Status_Summary.md` pair (§11.4.45, §11.4.56). Status docs are auto-discovered by the generator via `find docs -name 'Status.md'`.

Generator. `scripts/testing/update_readme_doc_links.sh` is the canonical helper. It:

1. Scans the canonical doc paths.
2. Extracts each doc's `Revision` + `Last modified` fields from its §11.4.44 header.
3. Renders the markdown table with current values.
4. Replaces the section between explicit markers (`<!-- doc-link-section:begin -->` and `<!-- doc-link-section:end -->`) in README.md.

The wrapper MUST be invoked from `sync_issues_docs.sh` (so every Issues.md / Fixed.md update refreshes the README freshness data) AND from `sync_integration_status.sh` (so every Status.md update likewise refreshes README).

Composition:

- §11.4.12 + §11.4.19 + §11.4.53 (Issues / Issues_Summary / Fixed / Fixed_Summary parity) — README links to all four.
- §11.4.44 (revision header) — README pulls Revision + Last modified from each doc's header.
- §11.4.45 (Status.md) + §11.4.56 (Status_Summary.md) — README enumerates every Status pair.
- §12.10 (CONTINUATION.md) — explicitly linked from README so operators discover it on arrival.

Pre-build gates:

- `CM-README-DOC-LINK-SECTION-PRESENT` — README.md contains the literal `Tracked-Items` heading AND both section markers (`<!-- doc-link-section:begin -->`, `<!-- doc-link-section:end -->`).
- `CM-README-DOC-LINK-ROWS-COMPLETE` — every canonical doc (Issues, Issues_Summary, Fixed, Fixed_Summary, CONTINUATION, every Status.md + Status_Summary.md found under `docs/`) appears as a row inside the section.
- `CM-README-DOC-LINK-FRESHNESS` — the `Last modified` value in each row matches the corresponding doc's §11.4.44 header within sync-wrapper granularity (no row staler than the source doc by more than the sync wrapper's tolerance).
- `CM-COVENANT-114-57-PROPAGATION` — anchor literal across files.

Paired mutations (per §1.1):

- Strip the section markers from README.md → `CM-README-DOC-LINK-SECTION-PRESENT` FAILs.
- Remove the CONTINUATION.md row from the section → `CM-README-DOC-LINK-ROWS-COMPLETE` FAILs.
- Backdate a `Last modified` value in a row → `CM-README-DOC-LINK-FRESHNESS` FAILs.
- Strip `§11.4.57` literal from `constitution/CLAUDE.md` → `CM-COVENANT-114-57-PROPAGATION` FAILs.

No escape hatch. No `--skip-readme-doc-links`, `--collapse-status-rows`, `--no-freshness-check` flag. The discipline exists because invisible docs are non-existent docs from a discoverability standpoint, and the §11.4 covenant specifically prohibits surfaces where claims (here: doc freshness) can drift unnoticed.

Classification: universal (per §11.4.17). Applies to every project that maintains an Issues / Fixed split or Status.md docs (i.e., to every project consuming this Constitution that has any of §11.4.15 / §11.4.45 / §11.4.53 in scope).

---

§11.4.58 — Parallel-development methodology (User mandate, 2026-05-19)

Forensic anchor — verbatim user mandate (2026-05-19T~05:00Z MSK):

> "We MUST DO one comprehensive research and planning in the background in parallel with current mainstream work: our current methodolgy of the development is very slow. It takes us days to fix a bunch of bugs, add some changes or features and all this to be tested, verified and shipped. From iteration to iteration we are slower and slower. We MUST CREATE adjusted improoved version of working methodology where multiple workable items could be done in parallel, all come into the central (main) branch as they are done, then we at particular moment rebuild and reflash the System and in background full testing with validation and verification is done. Each parallel work on one workable (or more) item(s) must use parallel agents as much as possible! We MUST HAVE proper synchronization mechanism between parallel working routines so they do not cause conflicts, break features or create any other types of the problems! Document everything - the whole plan, whole methodology, working flow with in depth diagrams and graphs,

> all user guides and manuals and then add this all into our root (constitution Submodule) Constitution, CLAUDE.MD and AGENTS.MD! Make sure we start using this ASAP since we MUST increase efficiency and productivity a couple of times now! Writing in depth tests (all supported types of the tests) with Challenges and full HelixQA use is MANDATORY! Every test we execute besides executed with success MUST RESULT in proof that actual functionality being tested REALLY DOES WORK with NO BLUFF of any kind! Heavt enforcement of no-bluff / anti-bluff policy IS MANDATORY!"

Why this anchor exists. Sequential phase-chain working pattern (`Phase 39.A → Phase 39.B → … → Phase 39.DT`, observed 2026-05-05 to 2026-05-19: 86 unique Phase 39.X sub-phases across 14 days = ~6/day sub-phase rotation, 224 commits / 14 days = ~16/day) serialises on five distinct bottlenecks: (B1) `commit_all.sh` flock at `.git/.commit_all.lock`, (B2) single-thread sub-phase chaining via `docs/CONTINUATION.md` hand-offs, (B3) rebuild-blocking on every `REQUIRES_REBUILD` fix (§12.7 -j2 cap = 4–6 h rebuild + 30 min flash + 60 min validate per cycle), (B4) single-thread `test_all_fixes.sh` sweep (~30–60 min × 8 phase-device combinations), (B5) subagent dispatch latency (one-at-a-time spawn + wait pattern). Net effect: single-item wall-clock 1–4 days where productive work is < 4 h. The User mandate requires the methodology to multiply throughput several-x.

Operative rule. Project work proceeds through the Parallel Work Unit (PWU) pipeline rather than sequential phase-chain. Each PWU is a self-contained workable item with mandatory components (ATM-NNN identifier per §11.4.54, Issues.md entry per §11.4.15+§11.4.16, file- scope manifest, §11.4.43 RED test, source patch, pre-build gate per §11.4.4(b) layer 1, post-flash test per §11.4.4(b) layer 3, paired §1.1 meta-test mutation, §11.4.4(b) layer 4 HelixQA Challenge bank entry, captured-evidence directory per §11.4.5+§11.4.52). PWUs execute through five pipeline stages: **Stage 1 DEVELOP** (parallel, N PWU agents in isolated worktrees), **Stage 2 MERGE** (serial, conductor processes one PWU at a time via `commit_all.sh` flock + §11.4.41 4- step merge-first pipeline), **Stage 3 REBUILD+FLASH** (parallel where possible: AOSP build inside §12.7 bounded scope, D3+D4 flash serialised on USB hardware), **Stage 4 VALIDATE** (parallel on D3+D4+meta-test+ coverage-audit), **Stage 5 SWEEP** (parallel: HelixQA Challenge bank, Issues→Fixed migration, README.md doc-link refresh per §11.4.57, HTML+PDF exports). Stage 1 of round N+1 starts WHILE Stages 4+5 of round N are still running — this is the primary throughput multiplier.

Synchronization mechanism. Four-layer lock hierarchy: **L1** parent serial flock `.git/.commit_all.lock` (held only during Stage 2, released between PWUs), **L2** per-submodule git operations (implicit via cascade), **L3** contention-path advisory locks at `qa-results/pwu-locks/<sha256(path)>.lock` for the 10 forbidden cross- PWU paths (CLAUDE.md/AGENTS.md/Constitution.md/ CONTINUATION.md/ Issues.md/Fixed.md/pre_build_verification.sh/ meta_test_false_positive_ proof.sh/build.sh/commit_all.sh+push_all.sh/ device.mk+BoardConfig.mk/ atmosphere-.*sh/init.*.rc), **L4** per-PWU git worktree (zero cross- PWU file conflicts during Stage 1). Disjoint-scope PWUs (different test files, different APKs, different drivers, different framework services) run fully parallel. Conflict detection at Stage 2 entry: `git fetch --all --prune --tags` + `git diff main...HEAD -- <scope>` + cross-PWU scope overlap check. Overlap detected → PWU rejected to Stage 1 for rebase (per §11.4.41 step 2 integration-before-force-push).

Anti-bluff enforcement. Conductor REFUSES to merge a PWU lacking all four anti-bluff checks: **C1** §11.4.43 RED test captured-evidence (file exists + non-zero exit code in log + timestamp predates source patch — proves test catches regression), **C2** §1.1 paired meta-test mutation (applied + gate FAILs under mutation + restored + gate PASSes — proves gate is not a bluff gate), **C3** §11.4.50 deterministic- consistency (3 iterations for normal, 10 for cycle-validation, ALL identical exit codes AND identical evidence-hashes — eliminates §11.4.7 flake/ transient/intermittent demotion path), **C4** §11.4.5 captured-evidence (audio: WAV with non-trivial RMS + ffprobe channel count; video: screen recording with ffprobe frame count > 0 + analyzer matched event; UI: uiautomator dump with under-test element state; or HelixQA `result.json` PASS verdict + positive-evidence chain). HelixQA coverage is MANDATORY for every user-visible PWU per User mandate — Challenge bank entry in `tools/helixqa/banks/atmosphere.yaml` referencing the PWU's ATM-NNN + dispatching to the on-device test + scoring PASS only on positive captured evidence. Metadata-only PASS / configuration- only PASS / absence-of-error PASS / grep-without-runtime-evidence PASS all REJECTED at the merge gate.

Agent orchestration. Four roles: **Conductor** (1, main thread) — issues ATM-NNN, dispatches PWU agents, runs Stage 2 merge serially, runs Stages 3-5, owns all contention-path edits, maintains `docs/CONTINUATION.md` §3 live snapshot. **PWU agent** (4–6 background) — one PWU each, Stage 1 work in isolated worktree, signals `qa-results/pwu-<atm>/READY_FOR_MERGE` on completion. **Validator** (4 background) — Stage 4 work in parallel on D3+D4+meta-test+coverage- audit. **Sweeper** (2 background) — Stage 5 work in parallel with next round's Stage 1. Max concurrent agent count ~12 at peak, bounded by §12.6 60% memory budget (~6 GB total ≤ 38 GB budget). Composition with §12 host-session safety: conductor refuses dispatch if `host_check_safety` fails; per §12.7 AOSP build still hard-capped at -j2 inside bounded scope; per §12.8 + §12.8b + §12.8c concurrency gates still block on stray gradle/kotlin daemons.

Composition map (mandatory):

- §11.4.4 — four-layer test coverage per PWU (pre-build + post-build + on-device + HelixQA bank).
- §11.4.5 — audio/video quality analysis comprehensiveness (PWU captured-evidence directory).
- §11.4.6 — no-guessing mandate (every PWU claim backed by captured forensic evidence or explicit `UNCONFIRMED:` tag).
- §11.4.7 — demotion-evidence rule (3/3 deterministic-consistency closes the demotion-bluff path).
- §11.4.9 — batch-source-fixes-before-rebuild (Stage 2 → Stage 3 batch threshold).
- §11.4.15 + §11.4.16 + §11.4.33 — PWU status + type vocabulary + type-aware closure terminal value.
- §11.4.19 — atomic Issues.md → Fixed.md migration on PWU closure.
- §11.4.41 — Pre-Force-Push Merge-First 4-step pipeline at Stage 2 entry.
- §11.4.42 — Iteration-discipline (PWU pipeline IS the iteration conductor; §11.4.58 binds steps 1-5 to PWU stages 1-5).
- §11.4.43 — TDD RED-first per-PWU enforced at merge time.
- §11.4.45 — Integration-status-doc maintenance (PWU's Status.md updated at every stage transition).
- §11.4.49 — Dual-approach testing (PWU MUST ship both UI-driven AND Intent/Broadcast-driven variants when applicable).

- §11.4.50 — Deterministic-consistency (3-iter normal / 10-iter cycle-validation merge-time enforcement).
- §11.4.52 — Autonomous-validation (PWU's on-device test MUST run without operator presence per Stage 4 parallel design).
- §11.4.54 — ATM-NNN ticket identifier (PWU identifier replaces Phase 39.X letters).
- §11.4.57 — README.md doc-link refresh at Stage 5.
- §12.6 — 60% memory-budget ceiling (caps concurrent agent count).
- §12.7 — AOSP `m -j` hard-cap at -j2 (Stage 3 build cap).
- §12.8 + §12.8b + §12.8c — concurrency hardening (Stage 3 preflight refuses on running gradle/kotlin/git-pack-objects/load-per-cpu spike).
- §12.10 — CONTINUATION.md maintenance (conductor owns the file by §3.4 forbidden cross-PWU rule).
- §9.2 — data safety (every history rewrite during Stage 2 merge-first step 2 backed by hardlinked `.git` backup).

Pre-build gates:

- `CM-PWU-LOCK-HIERARCHY` — verify lock hierarchy script exists at `scripts/testing/pwu_acquire_path_lock.sh` and `scripts/testing/pwu_release_path_lock.sh`, both executable, both reference the 10 forbidden cross-PWU paths from §3.4 of `docs/guides/PARALLEL_DEVELOPMENT_METHODOLOGY.md`.
- `CM-PWU-ANTI-BLUFF-COVERAGE` — for every PWU with status "Ready for merge" or beyond, assert all four anti-bluff checks (C1–C4) have captured-evidence files under `qa-results/pwu-<atm>/evidence/`. PWU lacking any of red-test-output / mutation_patch / deterministic- consistency / captured-feature-proof FAILs the gate.
- `CM-PWU-MERGE-QUEUE-DISCIPLINE` — scan `qa-results/pwu-queue/` for PWUs that bypassed the merge queue (commits on main without a corresponding `qa-results/pwu-queue/merged/<atm>` marker). Any bypass FAILs the gate.
- `CM-PWU-PARALLEL-AGENT-LIMIT` — assert no more than 6 concurrent background PWU agents (count tmux/screen sessions or worktree branches) — over-limit FAILs the gate (per §12.6 memory budget).
- `CM-COVENANT-114-58-PROPAGATION` — anchor literal `§11.4.58` present in `constitution/CLAUDE.md` + `constitution/AGENTS.md` + parent CLAUDE.md/AGENTS.md + every owned-submodule CLAUDE.md/AGENTS.md (42 files total in current ATMOSphere consumer family).

Paired mutations (per §1.1):

- Move `scripts/testing/pwu_acquire_path_lock.sh` aside → `CM-PWU-LOCK-HIERARCHY` FAILs.
- Touch a PWU's `READY_FOR_MERGE` marker without `red-test-output.log` → `CM-PWU-ANTI-BLUFF-COVERAGE` FAILs.
- Land a commit on main without a corresponding `qa-results/pwu-queue/merged/<atm>` marker → `CM-PWU-MERGE-QUEUE-DISCIPLINE` FAILs.
- Spawn a 7th concurrent PWU agent → `CM-PWU-PARALLEL-AGENT-LIMIT` FAILs.
- Strip `§11.4.58` literal from `constitution/CLAUDE.md` → `CM-COVENANT-114-58-PROPAGATION` FAILs.

No escape hatch. No `--skip-merge-queue`, `--allow-bypass`, `--no-anti-bluff-check`, `--unlimited-agents`, `--sequential-phase- chain-mode` flag exists in any pipeline helper. The discipline exists because (a) the User mandate explicitly requires the multiplier ASAP and (b) the §11.4 covenant specifically prohibits PASS-bluffs (which the four anti-bluff merge-time checks mechanically prevent). Operators who feel they need to bypass the pipeline for a specific high-urgency fix should split the work into a single trivial PWU and let the pipeline run normally — Stage 1 of a trivial PWU is < 30 min, Stage 2 is < 5 min, Stage 3 is the same rebuild cost they would pay anyway.

Classification: universal (per §11.4.17). Applies to every project consuming this Constitution that has multiple owned-submodules + a sequential commit/push tool + a captured-evidence testing discipline. Project-specific implementations (agent dispatch helpers, worktree layout, batch thresholds) live in consumer-side `docs/guides/PARALLEL_DEVELOPMENT_METHODOLOGY.md` (this Constitution defines the discipline; consumers implement the tooling).

Reference: parent `docs/guides/PARALLEL_DEVELOPMENT_METHODOLOGY.md` (full methodology with diagrams, templates, operator manual, and migration plan).

---

§11.4.59 — README always-sync mandate (User mandate, 2026-05-19)

Forensic anchor — direct user mandate (verbatim, 2026-05-19):

> "fully review and update our main README document. Some points are not valid anymore, some are missing. Make sure main README is among documents we MUST ALWAYS keep updated and in Sync with the projects and other documentation! Make sure we always export it (on every update) into PDF and HTML. This mandatory rules/constraints MUST BE all added into the root (constitution Submodule) Constitution, AGENT.MD and CLAUDE.MD!"

`README.md` at the project root is a §11.4.12-class always-sync document. It MUST be:

1. **Reviewed and updated whenever** new docs / integrations / Status.md entries appear, new submodules land, applied-fixes count changes, or canonical paths shift.
2. **Kept in lockstep** with `docs/CONTINUATION.md` (§12.10) and the Issues / Issues_Summary / Fixed / Fixed_Summary doc set (§11.4.12, §11.4.53) — same always-sync discipline.
3. **Exported to `.html` and `.pdf` on every update**. New helper `scripts/testing/sync_readme_export.sh` performs the pandoc + weasyprint export; it is auto-invoked by `scripts/testing/sync_issues_docs.sh` so a single doc-sync run refreshes the entire doc surface (Issues / Issues_Summary / Fixed / Fixed_Summary / CONTINUATION / README — md + html + pdf).
4. **Carry a §11.4.44 revision header** (Revision N + Last modified ISO timestamp) at the top of the file.
5. **Contain a Documentation Map section** linking to every Status.md
   - Status_Summary.md + spec + plan + guide + script-companion doc + changelog + the constitution submodule, plus a per-audience navigation (end user / developer / QA / agent).

6. **Self-contained** — no hyperlinks to ephemeral external systems as the only source of truth.

Stale exports are §11.4.59 violations regardless of whether the underlying `README.md` is correct — an operator (or future agent) reading the HTML or PDF gets a divergent view, and the §12.10 CONTINUATION resumption guarantee silently breaks. Same discipline as §11.4.12 Issues_Summary and §11.4.53 Fixed_Summary applied to README.md.

Captured-evidence enforcement. Pre-build gate `CM-README-EXPORT-SYNC` locks four invariants: (a) `README.md` exists, (b) `README.html` exists, (c) `README.html` mtime ≥ `README.md` mtime, (d) `README.pdf` mtime ≥ `README.md` mtime (skipped gracefully if weasyprint is unavailable on the host but enforced when the PDF is present). Paired meta-test mutation backdates `README.html` + `README.pdf` so the source is newer → gate FAILs.

Composition. Composes with §11.4.12 (Issues_Summary parity), §11.4.18 (script-companion docs), §11.4.23 (visual-cue HTML colorizer not applicable to README's narrative format), §11.4.44 (revision header on every Status.md and now README.md), §11.4.45 (Status.md integration), §11.4.53 (Fixed_Summary parity — README is the canonical sibling at the project-overview layer), §11.4.56 (Status_Summary parity), §11.4.57 (README.md is the canonical index per Phase 39.EW+1), §12.10 (CONTINUATION.md resumption guarantee depends on README's Documentation Map being current).

No escape hatch. §11.4.59 has NO operator-facing override flag. No `--skip-readme-sync`, `--no-readme-export`, `--readme-stale-OK` flag exists. The discipline exists because README.md is the first file any new agent / operator / contributor reads — letting it diverge from reality is the exact §11.4 PASS-bluff pattern but at the project-overview layer.

Classification: universal (per §11.4.17). Applies to every project consuming this Constitution. The doc-sync helper, pandoc + weasyprint dependencies, and gate name are universal; the specific Documentation Map contents are consumer-side per-project.

Canonical authority: constitution submodule `Constitution.md` §11.4.59.

Non-compliance is a release blocker regardless of context.

§11.4.60 — Documentation always-sync composite covenant (User mandate, 2026-05-19)

Forensic anchor — verbatim user mandate (2026-05-19 ~09:00Z):

> "Double check if all documents are properly tied with our root Constitution, CLAUDE.MD and AGENTS.MD so they are always up to date, always in sync and exported into PDF and HTML! … Issues, Issues_Summary, Fixed, Fixed_Summary, Continuation, Status and Status_Summary for all contexts (areas) — THEY ALL MUST BE REGULARLY UPDATED, IN SYNC AND CONSISTENT without giving at any moment false picture about the state of the project or particular area(s) of it!"

The covenant. Eight documentation classes constitute the project's living state surface. They MUST be in sync at all times across markdown + HTML + PDF artefacts. Per-class anchors §11.4.12 / §11.4.44 / §11.4.45 / §11.4.53 / §11.4.56 / §11.4.57 / §11.4.59 / §12.10 each govern one class; §11.4.60 binds them via a single mechanically-enforced composite invariant so the failure mode "one per-class gate silently disabled while operator reads a divergent HTML/PDF" is structurally impossible.

Eight bound doc classes (artefact triple — `.md` + `.html` + `.pdf`):

1. `docs/Issues.md` — open-item tracker (§11.4.12 + §11.4.15 + §11.4.16)
2. `docs/Issues_Summary.md` — auto-generated short form (§11.4.12)
3. `docs/Fixed.md` — closed-item archive (§11.4.53)
4. `docs/Fixed_Summary.md` — auto-generated short form (§11.4.53)
5. `docs/CONTINUATION.md` — resumption handoff (§12.10)
6. `README.md` — project overview + doc-link index (§11.4.57 + §11.4.59)
7. `docs/**/Status.md` (every domain-scoped instance) — per-integration status (§11.4.45)
8. `docs/**/Status_Summary.md` (every domain-scoped instance) — auto-generated short form (§11.4.56)

Mandatory composite gate. `CM-DOCS-COMPOSITE-SYNC` (pre-build, device/rockchip/rk3588/tests/pre_build_verification.sh):

- For every doc-class instance with `.md` present, verify `.html` mtime ≥ `.md` mtime AND `.pdf` mtime ≥ `.md` mtime.
- For every Status.md instance, also verify the sibling `Status_Summary.md` mtime ≥ `Status.md` mtime (if the summary exists).
- Walks `docs/` recursively for `Status.md` — the Status fleet is not enumerated statically because new integrations land per phase.

Composite gate FAILs the build if ANY single instance fails. The per-class gates (`CM-ISSUES-SUMMARY-SYNC`, `CM-DOCS-EXPORT-SYNC`, `CM-FIXED-SUMMARY-SYNC`, `CM-CONTINUATION-DOC-INSYNC`, `CM-README-EXPORT-SYNC`) remain in place — §11.4.60 is the belt-and-suspenders layer that catches what they miss.

Paired mutation (per §1.1). `meta_test_false_positive_proof.sh` backdates `docs/Issues.html` to year 2000 (`touch -t 200001010000.00`) and asserts `CM-DOCS-COMPOSITE-SYNC` FAILs.

Revision-header coupling. Per §11.4.44 every one of the 8 doc classes carries the `**Revision:** N` + `**Last modified:** ISO 8601 UTC` header directly under the H1. `CM-DOCS-COMPOSITE-SYNC` does NOT re-check the revision header (already covered by `CM-DOC-REVISION-HEADER-PRESENT`) — composite gate is artefact-mtime only, by design narrow.

Auto-sync wrapper coupling. Per §11.4.12 every edit to an Issues/Fixed-class doc flows through `sync_issues_docs.sh`. Per §11.4.45 every edit to a Status.md flows through `sync_integration_status.sh <Status.md>`. Per §11.4.59 every edit to README.md flows through `sync_readme_export.sh`. §11.4.60 does NOT change the wrapper contract — it only catches the failure mode where an operator (or AI agent) bypassed the wrapper and committed markdown without regenerating exports.

No escape hatch. No `--skip-composite-doc-sync`, `--allow-stale-html`, `--summary-not-applicable` flag exists. The covenant exists for the operator's own protection — the moment a single per-class gate is bypassed, divergence accumulates silently until release-time forensics finds it (the exact §11.4 PASS-bluff pattern the canonical covenant was authored to eliminate).

Composes with §11.4.12 (Issues_Summary), §11.4.15 (Status field), §11.4.16 (Type field), §11.4.19 (atomic Issues→Fixed migration), §11.4.23 (colorizer), §11.4.33 (type-aware closure vocabulary), §11.4.44 (revision header), §11.4.45 (Status.md), §11.4.53 (Fixed_Summary), §11.4.56 (Status_Summary), §11.4.57 (README doc-link), §11.4.59 (README always-sync), §12.10 (CONTINUATION maintenance).

Propagation. Pre-build gate `CM-COVENANT-114-60-PROPAGATION` enforces the §11.4.60 anchor literal in every CLAUDE.md / AGENTS.md across parent + 10 owned submodules + HelixQA dependencies (same shape as existing §11.4.5X propagation gates). The propagation gate is OPTIONAL for Phase 39.FC's initial landing — only the composite gate + its paired mutation are mandatory; propagation gates trail per established phase pattern.

Classification: universal (per §11.4.17). Applies to every project consuming this Constitution that maintains any of the 8 doc classes. The composite gate logic is universal; the specific doc-class instance paths are consumer-side per-project.

Canonical authority: constitution submodule `Constitution.md` §11.4.60.

Non-compliance is a release blocker regardless of context.

§11.4.61 — Mandatory Markdown metadata table + structured-doc ToC (User mandate, 2026-05-19)

Forensic anchor — direct user mandate (verbatim, 2026-05-19):

> "For every Markdown document which contains structured content (with headings / sections and sub-sections) make sure that every time we apply change to the structure, table of contents on the top of the document is created or updated! This is MANDATORY for every structured MARKDOWN document. Automatically its PDF and HTML versions MUST BE (re)generated!"
>
> "Introduce … revision number, date and time of creation, date and time of last modification, other useful information we have in documents such as Issues, Issues_Summary, Fixed, Fixed_Summary, Status, Status_Summary, Continuation and similar. … make this mandatory for EVERY Markdown document from now on, update root constitution Submodule with these changes and commit and push it all to all upstreams."

Scope of §11.4.61. This clause adds two universal disciplines that sit alongside (NOT replacing) the §11.4.65 universal export mandate. §11.4.65 governs the existence and freshness of `.html` + `.pdf` siblings; §11.4.61 governs the *content* of the Markdown sources: how revision/status metadata is displayed at the top, and how the document's navigation map (Table of contents) is kept in lockstep with the heading structure. The two clauses compose — §11.4.65 guarantees readers see *something current*; §11.4.61 guarantees what they see communicates its own provenance and is internally navigable.

A. Canonical metadata table (supersedes §11.4.44 format). The §11.4.44 bold-line revision header is superseded by a **Markdown table** placed immediately after the document's H1 title (and any blank line that follows). Pre-existing bold-line headers remain historically valid but MUST migrate to the canonical table at the next substantive edit. The canonical table has these MANDATORY rows:

Field	Value
Revision	positive integer; starts at 1, monotonic, never reset
Created	ISO 8601 date — first commit touching the file
Last modified	ISO 8601 date — most recent commit touching the file
Status	one of `active` / `draft` / `deprecated` / `superseded`

ENCOURAGED rows (REQUIRED when the document tracks workable items or has a known continuation; render with – / `none` when N/A):

Field	Value
Status summary	one-line current-state hook
Issues	comma-separated workable-item IDs
Issues summary	one-line summary of open issues
Fixed	comma-separated workable-item IDs of resolved items
Fixed summary	one-line summary of fixes
Continuation	next action or linked CONTINUATION.md anchor

Additional rows MAY be added for project-specific metadata as long as the MANDATORY rows are present.

B. Structured-document Table of contents. Any tracked `*.md` with **two or more H2 sections** ("structured content") MUST include a `## Table of contents` section immediately after the metadata table. The ToC MUST list every H2 and H3 in document order with anchor links and MUST be regenerated on every structural change (heading added / removed / renamed / reordered). A stale ToC is a §11.4 PASS-bluff: operators reading the rendered Markdown see a navigation map that no longer matches the body.

Scope (IN-scope under §11.4.61). Every tracked `*.md` in the §11.4.65 INCLUDED set (project-root `*.md`, `docs/**/*.md`, `scripts/**/*.md` companion docs, owned-submodule trees, `constitution/**/*.md`, owned HelixQA dependencies). The §11.4.44 exclusions for `CLAUDE.md` / `AGENTS.md` / `README.md` are SUPERSEDED here too — explicit metadata in the file itself is preferred over indirect tracking via a separate VERSION file because operators reading the rendered document see the freshness directly.

Narrow exemptions (NOT IN-scope). Same EXCLUDED set as §11.4.65 (`external/**`, `prebuilts/**`, `out/**`, `build/**`, application- internal `*.md` shipped with source code, non-owned third-party submodule trees). Additional exemptions for the structural rules: `LICENSE`, `LICENSE.md`, `NOTICE`, `VERSION`, `OWNERS`, machine- generated `CHANGELOG.md`.

PDF + HTML siblings. Deferred to §11.4.65. The metadata table and ToC are *content* requirements; their export-freshness guarantee lives in §11.4.65's `CM-UNIVERSAL-MARKDOWN-EXPORT-SYNC` gate. A §11.4.61 metadata or ToC change is a `.md` modification, which transitively triggers §11.4.65 regeneration — the two clauses compose without duplicate enforcement.

Anti-bluff captured-evidence gates (planned):

- `CM-MD-METADATA-PRESENT` — walks every tracked `*.md` (minus exemptions) and asserts (a) H1 present, (b) within 25 lines below H1 a Markdown table contains all four MANDATORY rows (`| Revision |`, `| Created |`, `| Last modified |`, `| Status |`).
- `CM-MD-TOC-PARITY` — for every `*.md` with `≥ 2` H2 sections, asserts a `## Table of contents` is present and its entries' anchor slugs match the document's live H2/H3 set in order.

Paired §1.1 mutation tests (planned):

- Strip the `| Revision |` row from one `*.md` → `CM-MD-METADATA-PRESENT` FAILs.
- Rename one H2 without updating ToC → `CM-MD-TOC-PARITY` FAILs.

Composition with §11.4.44. §11.4.44 remains the authoritative clause for the *fact* that documents must carry revision metadata; §11.4.61 changes the *format* (bold-lines → table) and *scope* (`docs/**/*.md` → every tracked `*.md` per §11.4.65 INCLUDED set). Consumer projects with existing gates that grep for `**Revision:**` MUST update those gates to also accept the canonical table form. Migration period: 30 days from §11.4.61 landing.

Composition with §11.4.45 / §11.4.59 / §11.4.60 / §11.4.65. The per-class always-sync clauses (Status.md §11.4.45, README §11.4.59, composite §11.4.60) and the universal export covenant §11.4.65 govern *artefact existence and freshness*; §11.4.61 governs *content discipline inside the source `.md`*. The clauses are orthogonal layers of the same anti-bluff posture.

No escape hatch. No `--skip-md-metadata`, `--no-metadata-table`, `--toc-stale-OK`, `--allow-missing-revision` flag exists. Divergent metadata and stale ToCs are §11.4 PASS-bluff patterns at the doc-surface layer.

Classification: universal (per §11.4.17). Applies to every project consuming this Constitution. The canonical table format and ToC mandate are universal; the per-project tracker IDs (e.g. `HRD-`) and specific Issues/Fixed-class doc paths are consumer-side per-project.

Canonical authority: constitution submodule `Constitution.md` §11.4.61.

Non-compliance is a release blocker regardless of context.

§11.4.63 — Workable-items procedure docs as single source of truth (User mandate, 2026-05-19)

Forensic anchor — verbatim user mandate (2026-05-19 ~10:30Z):

> "To make workable items tracking and creation through all of the mentioned mandatory documents we MUST create proper procedure document which will be named in Constitution, AGENTS.md and CLAUDE.md as single source of truth for this procedure! Make sure it is created under: docs/procedures/issues/Creation.md which will be always exported to PDF and HTML whenever it is updated. We MUST create procedure documents for Updating.md, Resolution.md, Reopening.md and other workable items actions that we have. … Everything done on project - new features, changes, tasks, bug fixes, investigation, ordinary tasks (writing documentation for example) MUST ALL go through the workable items flow and proper procedures!"

Every workable-item action — opening, updating, closing, reopening, migrating across Issues.md ⍰ Fixed.md — MUST follow the canonical procedure document at `docs/procedures/issues/<Action>.md`. The five procedure docs are MANDATORY references; no agent or operator may invent ad-hoc procedures. The closed-set:

Procedure doc	Covers
`Creation.md`	Opening a new workable item (Bug / Feature / Task)
`Updating.md`	Non-closure, non-reopen edits — status transitions, evidence append, type re-classification
`Resolution.md`	Atomic Issues.md → Fixed.md close (Bug→Fixed, Feature→Implemented, Task→Completed per §11.4.33)
`Reopening.md`	Atomic Fixed.md → Issues.md reopen with §11.4.34 source attribution + §11.4.55 reopens-history
`Migration.md`	Bidirectional atomic-move mechanics + sync_issues_docs.sh internals (consumed by Resolution + Reopening)

Each procedure doc carries the §11.4.44 revision header + HTML + PDF exports synchronised to its `.md` source. New helper `scripts/testing/sync_procedure_docs_export.sh` walks the procedure- docs directory and emits matching `.html` + `.pdf` for each `.md`; invoked as the final stage of `scripts/testing/sync_issues_docs.sh` so a single operator invocation refreshes every doc-side artifact.

Universality of scope. All work — new features, behavioural changes, tasks (refactor / doc / infra / gate / audit / cleanup), bug fixes, investigation, documentation work — MUST flow through these procedures. The "I'll just tweak this one file" escape hatch is forbidden because it produces silent doc drift and reopens the §11.4 covenant's PASS-bluff failure mode at the process-tracking layer.

Composes with §11.4.6 (no-guessing — every procedure step is mechanical, no `likely` / `seems`), §11.4.7 (demotion-evidence — Reopening.md is the canonical implementation), §11.4.11 (file-layout — procedure docs live in `docs/procedures/issues/`, qa-results forensics live in `qa-results/`), §11.4.12 (Issues_Summary sync — all five procedures invoke `sync_issues_docs.sh` whose pipeline is documented in Migration.md), §11.4.15 (Status closed-set — all five

procedures cite the same closed-set), §11.4.16 (Type closed-set), §11.4.19 (atomic Issues?Fixed move — Migration.md is the implementation), §11.4.23 (visual cue + grouping colorization — invoked by sync wrapper, no procedure-side bypass), §11.4.33 (type- aware closure vocabulary — Resolution.md enforces), §11.4.34 (Reopened-source attribution — Reopening.md enforces), §11.4.44 (revision header on every procedure doc), §11.4.53 (Fixed_Summary parity — Migration.md documents the pipeline), §11.4.54 (ATM-NNN identifier — Creation.md allocates), §11.4.55 (Reopens-history per- item doc — Reopening.md authors), §11.4.60 (documentation always- sync composite — procedure docs are an additional doc class binding to the composite covenant).

Pre-build gate `CM-PROCEDURES-DOCS-PRESENT` checks (1) all 5 procedure docs exist at `docs/procedures/issues/{Creation,Updating, Resolution,Reopening,Migration}.md`, (2) each carries the §11.4.44 revision header, (3) each has matching `.html` + `.pdf` exports with mtime ≥ the .md mtime, (4) `scripts/testing/sync_procedure_docs_ export.sh` exists + is executable, (5) `scripts/testing/sync_issues_ docs.sh` invokes `sync_procedure_docs_export.sh`. Paired mutation: rename one procedure doc → gate FAILs.

Propagation gate `CM-COVENANT-114-63-PROPAGATION` enforces this anchor literal in every CLAUDE.md / AGENTS.md across parent + 10 owned submodules + nested submodules + HelixQA dependencies. Paired mutation strips the anchor literal → gate FAILs.

No escape hatch — no `--ad-hoc-procedure`, `--skip-procedure-doc`, `--procedure-not-applicable` flag exists. Inventing an alternate procedure for a "small" change is the exact failure mode this anchor closes.

Classification: universal (per §11.4.17). The closed-set of five procedure docs and the sync-wrapper composition are universal; the specific test-name / fix-name examples within each procedure doc may be consumer-side per-project.

Canonical authority: constitution submodule `Constitution.md` §11.4.63.

Non-compliance is a release blocker regardless of context.

§11.4.65 — Universal Markdown export mandate (User mandate, 2026-05-19)

Forensic anchor — direct user mandate (verbatim, 2026-05-19):

> "Any markdown document inside the project and which is not part of the applications or services source code MUST BE exported (be available) in PDF and HTML! Any already existing Markdown document that fulfills this condition and which does not have HTML or PDF at all or it is not in sync with it MUST HAVE (re)generated PDF and HTML version! Every time when Markdown document (file) is modified, its proper HTML and PDF versions MUST BE regenerated. Markdown documents MUST BE at all times in sync with PDF and HTML versions!"

The covenant generalizes the per-class always-sync discipline of §11.4.12 / §11.4.18 / §11.4.44 / §11.4.45 / §11.4.53 / §11.4.56 / §11.4.57 / §11.4.59 / §11.4.60 / §11.4.63 / §11.4.64 to **every Markdown document in the project that is not part**

of an application or service's source-code tree. Per-class anchors govern specific files (Issues, Fixed, Status, README, etc.); §11.4.65 is the catch- all: any *other* `.md` file that documents the project — guides, research notes, plans, hardware-ID notes, script-companion docs, changelogs, procedure docs, README files inside owned submodules — MUST also have `.html` + `.pdf` siblings, all three artefacts in sync at all times. Stale exports of *any* documentation surface are the exact "operator reads divergent HTML/PDF while .md is correct" PASS-bluff §11.4 forbids — only the per-class enforcement so far prevented it for specifically-named docs; §11.4.65 closes the long tail.

Scope (closed-set):

INCLUDED: - Project root `*.md` (README.md, CLAUDE.md, AGENTS.md, CONTRIBUTING.md, etc.) - `docs/**/*.md` (guides, research, plans, changelogs, procedures, hardware notes) - `scripts/**/*.md` (script-companion docs in documentation format, NOT shebang scripts) - Owned-submodule trees at `device/rockchip/atmosphere/<submodule>/`: top-level README.md / CLAUDE.md / AGENTS.md / CHANGELOG.md and any `docs/**/*.md` - `constitution/**/*.md` (the canonical-root submodule) - `tools/helixqa/<helixqa-submodule>/` top-level README.md / CLAUDE.md / AGENTS.md and any `docs/**/*.md` (owned HelixQA dependencies)

EXCLUDED (NOT subject to §11.4.65 — these are application/service source code or third-party trees we do not own): - `external/**` (AOSP-mainline / upstream-mirror source trees) - `prebuilts/**` (binary prebuilts + their bundled docs) - `packages/modules/**` (AOSP mainline modules) - `kernel-5.10/**` (kernel source tree — has its own upstream docs) - `out/**` (build output) - `build/**` (AOSP build system) - Application / service source-code trees (e.g. Java/Kotlin module-internal `*.md` shipped with library code, README files inside source-only third-party gradle module directories) - Any third-party submodule NOT in the owned-submodule set (presenter, vlc-player, nova-player, mpv-player, gramophone-player, rhythm-player, strep-player, smarttube-player, torrserve, lampa) or HelixQA-owned set (Challenges, Containers, DocProcessor, LLMOrchestrator, LLMProvider, VisionEngine)

Mandatory protections (ALL must hold):

1. **Every INCLUDED `.md` file has `.html` and `.pdf` siblings.** A missing export is a §11.4.65 violation regardless of when the markdown was last touched.
2. **`.html` and `.pdf` mtime ≥ `.md` mtime** (within the same sync-wrapper invocation granularity). Stale exports are violations even if the .md itself is correct.
3. **Every modification triggers regeneration.** Whether via direct sync-wrapper invocation (`sync_all_markdown_exports.sh`), a git pre-commit hook auto-regenerating on staged `.md` changes, or the existing per-class wrappers (§11.4.12 sync_issues_docs.sh, §11.4.18 script-docs sync, §11.4.59 sync_readme_export.sh, etc.) all of which delegate the universal export path to the same canonical helper.
4. **Pre-build gate enforces parity.** `CM-UNIVERSAL-MARKDOWN-EXPORT- SYNC` walks the INCLUDED scope, verifies every `.md` has `.html`
   - `.pdf` siblings with mtime ≥ `.md` mtime, and FAILs the build if any are missing or stale.
5. **No escape hatch.** No `--skip-md-exports`, `--no-pdf-only`, `--md-export-not-applicable`, `--application-internal-doc` flag exists for files inside

the INCLUDED scope. The EXCLUDED scope is the only legitimate path to opt out, and it is closed-set.

Canonical helper. `scripts/testing/sync_all_markdown_exports.sh` walks the INCLUDED scope, invokes pandoc (HTML) + weasyprint (PDF) with `timeout 60` each (graceful per-file degradation), uses `docs/_progress-style.css` for visual consistency, and supports `--check-only` (exit nonzero if any out-of-sync, print list) and `--regenerate-all` (force) modes. Caps at 500 candidates with explicit abort+list if the scope is unexpectedly large (e.g. after a new submodule lands without scope-update review). Idempotent.

Composition. Composes with §11.4.12 (Issues_Summary sync — universal helper is the back-end), §11.4.18 (script-companion docs), §11.4.23 (HTML colorizer continues to post-process the always-sync docs for visual cues), §11.4.44 (revision header on every .md the universal helper exports), §11.4.45 (Status.md auto-sync), §11.4.53 (Fixed_Summary), §11.4.59 (README always-sync — the universal helper covers the long tail of every *other* root-level .md too), §11.4.60 (composite always-sync covenant — §11.4.65 is the structural generalization), §11.4.63 (procedure docs — already always-sync; §11.4.65 adds catch-all for every doc class not yet anchored), §11.4.64 (topic discoverability — summary docs the universal helper exports continue to be post-processed by inject_topic_index.py).

Pre-build gates:

- `CM-UNIVERSAL-MARKDOWN-EXPORT-SYNC` — invariants: (a) `scripts/testing/sync_all_markdown_exports.sh` exists + executable,
    1. running it in `--check-only` mode returns 0 (no out-of-sync file in the INCLUDED scope).
- `CM-COVENANT-114-65-PROPAGATION` — anchor literal `11.4.65` in every CLAUDE.md / AGENTS.md across canonical-root + parent + 10 owned submodules + nested submodules + HelixQA dependencies (42-file fleet — same propagation set as §11.4.58).

Paired meta-test mutations: one strips the `CM-UNIVERSAL-MARKDOWN- EXPORT-SYNC` gate's enforcement literal; one strips the §11.4.65 anchor literal from a sentinel propagation file. Both mutations assert the corresponding gate FAILs when applied.

Classification: universal (per §11.4.17). The mandate to export every project-doc Markdown to HTML+PDF is universal across every project that consumes this Constitution; the specific INCLUDED / EXCLUDED scope path lists are consumer-side per-project (since each consumer has different submodule topology, build trees, and source- code roots).

Canonical authority: constitution submodule `Constitution.md` §11.4.65.

Non-compliance is a release blocker regardless of context.

---

§11.4.66 — Blocker-resolution interactive-clarification mandate (User mandate, 2026-05-19)

Forensic anchor — direct user mandate (verbatim, 2026-05-19):

> "If any blockers which can be resolved with interactive response ever happen again, perform in depth research on options doable by your side and how much inputs from us you really need, then create options and present them to us. After we answer, preferrably you will be unblocked and be able to continue work on blocked items. Let us make this main approach when such situations (blockers) do happen!"

When any task is blocked (waiting on operator decision, hardware access, external-service authorization, ambiguous scope, or any input the agent cannot mechanically derive), the agent MUST follow this five-step discipline before idling or asking free-form questions:

1. **In-depth research on agent-side options.** Identify the maximum scope that can land unilaterally without operator input — research the codebase, upstream documentation, existing tooling, the current state, what files/components are involved, what tests/gates already exist. Surface every agent-actionable path even if it requires a small operator confirmation.

2. **Calculate minimum-viable operator input.** Reduce the question set to the smallest closed set of operator-only decisions (preference, authorization, scope-bounding, hardware availability, schedule). Anything that can be researched MUST NOT be asked.

3. **Construct 2–4 mutually-exclusive options.** Each option carries

    1. a short label, (b) a one-line trade-off description, (c) an explicit statement of what the agent will do *after that answer*. One option marked "Recommended" with rationale. Options MUST genuinely differ — restating one option as three close variants is itself a §11.4.66 violation.

4. **Present via the platform's interactive question mechanism.** On Claude Code that is `AskUserQuestion` (max 4 questions, each with 2–4 options; supports multi-select for non-mutually-exclusive sets). Other platforms (Copilot CLI, Codex, Gemini CLI) use their equivalents. NEVER inline free-text "what would you like?" when interactive options would do — that wastes operator attention.

5. **After the operator answers, resume work without additional round-trips.** Every option's promised action MUST be sufficient to unblock — if a follow-up clarifying question is needed, the prior options were insufficiently researched and that's itself a §11.4.66 violation. The contract is: ask once, unblock, continue.

Composes with:

- §11.4.6 (no-guessing — interactive options replace agent guessing about operator preference)
- §11.4.7 (demotion-evidence — operator preference is captured- evidence for direction changes)
- §11.4.40 (full-suite retest — uninterrupted work post-answer means the test cycle resumes cleanly without context drift)
- §11.4.41 (merge-first — when operators make conflicting choices across sessions, the merge-first discipline preserves both)
- §11.4.42 (iteration-discipline — interactive-question loop is built into the per-cycle methodology)

- §11.4.52 (autonomous-validation — most blockers can be researched to autonomous-path level, leaving only true operator preference to interactive-ask)

Mandatory protections:

- **No silent waiting.** If blocked, ask. If genuinely no operator input is needed, do not ask — proceed.
- **No bulk-text questions when interactive options would do.** Free-text prompts are reserved for truly open-ended ideation, never for closed-set decisions.
- **Each interactive question minimizes operator cost** — the operator should be able to answer all questions in under 30 seconds total.
- **Recommended option is highlighted explicitly** — the operator should know what the agent would default to if no answer comes.
- **Out-of-band channels (slack, voice, email) NEVER substitute** — the interactive mechanism creates an audit trail that free-text channels do not.

Pre-build gate `CM-COVENANT-114-66-PROPAGATION` enforces the anchor literal is present across the 42-file consumer fleet (parent CLAUDE.md / AGENTS.md + 10 owned-submodule pairs + HelixQA-owned pairs). Paired meta-test mutation strips the literal → gate FAILs. No escape hatch — no `--skip-ask`, `--silent-wait`, `--free-form-only` flag is permitted.

Canonical authority: constitution submodule `Constitution.md` §11.4.66.

Non-compliance is a release blocker regardless of context.

---

§11.4.67 — Shell-script target-shell-parseability mandate (User mandate, 2026-05-19)

Forensic anchor — direct user mandate (verbatim, 2026-05-19):

> "any issue we spot must be fixed, bash scripts as well if they are broken!"
> "Make sure that this is mandatory rule!"

Forensic incident — Phase 39.FJ.67 D3 post-flash cycle block: `device/rockchip/rk3588/tests/test_all_fixes.sh:114` invoked on Android via `sh script.sh` used bash-only process substitution `exec > >(tee -a "$file") 2>&1`. Android's `/system/bin/sh` is mksh, which parses the *entire* script BEFORE executing it — so the runtime guard `if [ -n "${BASH_VERSION:-}" ]` could not save the script: mksh rejected `>(...)` at parse time and the test cycle failed to launch on D3. The runtime guard correctly tried to gate the unsafe path, but mksh's compile-time parser made the guard useless. Fixed by wrapping the offending `exec` in `eval` so the parser sees only a string and the parse step defers to runtime where the guard can take effect.

This is the canonical class of script-bug FAIL-bluff: a test cycle PASSes on the developer host (bash) and FAILS at parse time on the target (mksh), producing a FAIL-bluff that masks every product defect the cycle would have caught. §11.4.1 already forbids script-bug FAILs. §11.4.67 mechanically prevents this specific sub-class.

The mandate. Every shell script that may be invoked under a target shell other than the one in its shebang MUST parse cleanly under that target shell. Specifically:

1. **Closed-set scope.** Every tracked `.sh` file under `device/rockchip/rk3588/tests/`, `scripts/`, and `scripts/testing/` (and equivalent paths in owned submodules) is in scope. Explicitly OUT of scope: `external/`, `prebuilts/`, `packages/modules/`, `kernel-5.10/`, `out/`, `build/`, `scripts/legacy/` (legacy quarantine), and any third-party vendored shell directory the project does not maintain.
2. **Mandatory parseability invariant.** Every in-scope script MUST parse under POSIX `sh -n` on the developer host. POSIX `sh` is the closest portable parser to Android mksh — passing `sh -n` on the host gives high confidence the script parses on mksh.
3. **Bash-only syntax requires deferral.** Process substitution `>(...)` / `<(...)`, `[[ ]]`, `<<<` here-strings, indexed arrays `arr=()`, associative arrays `declare -A`, `${var^^}` / `${var,,}` case-fold expansions, `${var/pattern/replacement}` with bash-only operators, `coproc`, `function name { ... }` (vs POSIX `name() { ... }`), and any other bash-only construct MUST EITHER be wrapped in `eval '...bash-only string...'` so the parser sees only a string OR be guarded by sourcing the file only on bash- detected hosts (`[ -n "${BASH_VERSION:-}" ]` BEFORE the script loads — runtime guards inside a single mksh-parsed script do not work).
4. **Honest shebangs.** A script that contains bash-only constructs without `eval`-deferral MUST declare `#!/bin/bash` (or equivalent) AND its callers MUST invoke it as `bash script.sh` rather than `sh script.sh`. A script with `#!/system/bin/sh` or `#!/bin/sh` shebang MUST be POSIX-clean — no bash-only syntax, no `local` keyword without an mksh fallback, no `read -p`, no `echo -e` without portable equivalent.
5. **`eval`-deferral is the safe pattern.** When a bash-only construct is genuinely needed inside a script that may be parsed by mksh (e.g. `sh script.sh` from a callsite outside our control), wrap the construct in single-quoted `eval`: `eval 'exec > >(tee -a "$f") 2>&1'`. The parser sees a string; the runtime guard around the `eval` decides whether to execute.

Captured-evidence enforcement. Pre-build gate `CM-SCRIPT-TARGET-SHELL-PARSEABLE` walks every `.sh` file under the in-scope directories, runs `sh -n` on each with a 30-second per-file timeout, and FAILs if any script fails to parse. The gate's diagnostic output cites the failing file + the `sh -n` error message so the fix location is unambiguous.

Propagation gate. `CM-COVENANT-114-67-PROPAGATION` verifies the §11.4.67 anchor literal is present across the 44-file consumer fleet (parent CLAUDE.md / AGENTS.md + Containers + 10 owned atmosphere submodules + nested SmartTube SharedModules / MediaServiceCore / MSC-SharedModules + 7 HelixQA submodules). Constitution submodule files (constitution/{Constitution,CLAUDE,AGENTS}.md) are canonical authority — they carry §11.4.67 by definition.

Paired meta-test mutations. (a) Inject a bash-only construct (`exec > >(/bin/cat)`) outside `eval` into a fresh location inside a test script and assert `CM-SCRIPT-TARGET-SHELL-PARSEABLE` FAILs. (b) Strip `11.4.67` literal from one consumer file and assert `CM-COVENANT-114-67-PROPAGATION` FAILs.

Composes with:

- §11.4.1 (FAIL-bluffs equally forbidden — script-bug FAILs that mask product defects are the exact failure mode §11.4.67 mechanically closes)
- §11.4.4 (test-interrupt-on-discovery — a parse FAIL at cycle start IS the discovery event; cycle stops, fix lands, retest)
- §11.4.6 (no-guessing — `sh -n` produces fact, not opinion)
- §11.4.50 (deterministic consistency — a script that parses on one shell and not another fails the N-iteration identical-result requirement)
- §11.4.51 (live-ADB-first — shell-script fixes are LIVE_ADB_TESTABLE: push the fix, re-run on device, capture evidence before commit)

Mandatory protections (no escape hatch):

- **No `--skip-parseability-check` flag.** A script that does not parse under the target shell is broken regardless of intent.
- **No "this script only runs on bash" exception.** If a script may be invoked via `sh script.sh` (and almost all of them can — Android's default `sh` is mksh), it MUST parse under mksh.
- **No "the runtime guard catches it" exception.** Runtime guards inside a mksh-parsed script cannot prevent mksh parse-time rejection. Use `eval`-deferral or shebang-controlled invocation.
- **Fix at source, not in callsites.** A broken script is fixed in the script itself (per §11.4.1), not by individual callsites defensively detecting parse failures.

Pre-build gate `CM-SCRIPT-TARGET-SHELL-PARSEABLE` + paired mutation.
Propagation gate `CM-COVENANT-114-67-PROPAGATION` + paired mutation.

Canonical authority: constitution submodule `Constitution.md` §11.4.67.

Non-compliance is a release blocker regardless of context.

§11.4.68 — Positive sink-side / downstream evidence mandate (User mandate, 2026-05-20)

Forensic anchor — direct user mandate (verbatim, 2026-05-20):

> "We still do not hear any audio played from D3 device! Arvus Web Dashboard when we play music from D3 shows nothing for Codec In Use! This MUST BE investigated and fixed! How come we passed the tests with Arvus validation? What were values for the Codec In Use field? Empty means nothing! This is not working! It MUST BE FIXED, TESTED AND VERIFIED WITH FULL AUTOMATION TESTING ASAP!!!"

§11.4.13 already mandates consuming the Arvus sink-side report when available. §11.4.68 extends this with a stricter contract: **a test that asserts audio or video routing PASS MUST capture and verify positive sink-side or downstream evidence — never config-only, never metadata-only, never PCM-open-state-only.** The §11.4.13 path described what SHOULD be captured; §11.4.68 fixes the failure mode where tests "PASSed" because the helpers silently SKIPped the very evidence the assertion depended on.

Positive sink-side / downstream evidence — closed enumeration (at least one MUST be captured for every audio/video routing PASS; none alone is sufficient if the canonical sink/downstream is reachable):

1. **Sink-side codec-state** (Arvus / Sonos / Yamaha / Denon / Marantz / WiSA REST API) showing a **non-empty Codec-In-Use** value matching the test's expected codec regex during the playback window. Empty / `<unreachable>` / `<N.E.>` / `<None>` placeholders are NOT positive evidence.
2. **PCM frames-written delta** from `/proc/asound/cardN/pcmMp/sub0/status hw_ptr` — the delta over the playback window MUST be strictly positive AND consistent with the expected sample-rate × channel-count × duration. Zero or negative delta is FAIL.
3. **ALSA ELD / EDID-Like-Data** from `/proc/asound/cardN/eld#X.Y` demonstrating the sink negotiated the expected channel count + format. Empty ELD is NOT evidence the sink received audio — it only proves the cable / EDID-channel is up.
4. **ffprobe-on-captured-mp4** for video routing — non-zero frame count, expected codec, expected resolution, expected fps. 0-byte capture (Bug #24 pattern) is FAIL.
5. **Recording-analyzer event match** per §11.4.2 / §11.4.5 timeline — at least one matched event in the analyzer findings.
6. **Tinycap RMS amplitude** for ALSA-loopback audio recordings — non-trivial RMS in the captured WAV (>= -60 dBFS for line-level playback).

Failure-mode taxonomy — what §11.4.68 specifically forbids:

- `arvus_probe_present` returns 1 → test reports SKIP → suite reports all-green. **FORBIDDEN.** When the sink-side evidence is REQUIRED by the assertion, missing sink is `OPERATOR-BLOCKED` (an explicit release-blocker, NOT a SKIP, NOT a PASS).
- `arvus_probe_codec_state` returns empty string → test logs "codec_state:" (blank) → assertion proceeds on HAL-side metadata. **FORBIDDEN.** Empty codec-state IS a defect signal — either the audio is not arriving at the sink (genuine product defect per current User mandate) OR the sink is unreachable (operator-blocked). Either way, the test cannot PASS.
- Test reads HAL `dumpsys media.audio_flinger` output-thread device field showing `0x400 (AUDIO_DEVICE_OUT_HDMI)` → reports "HDMI routing PASS". **FORBIDDEN.** That field reflects what the audio POLICY chose, NOT what the audio HAL physically opened. The HDMI ALSA card may still be closed (PCM open failure swallowed by the HAL's `ALOGW` fallback) — silent silence at the user's ear. Sink-side codec-state or PCM `hw_ptr` delta is the ONLY proof audio actually arrived.
- Test asserts `/vendor/etc/audio_policy_configuration.xml` contains the `deep_buffer` route to `HDMI Out`. **FORBIDDEN as the only evidence.** Config-only PASS is the canonical §11.4 PASS-bluff — config can be perfect AND audio still inaudible (e.g., the HAL may still misroute). MUST be paired with at least one runtime positive evidence from the enumeration above.

Mandatory protections (all four):

1. **Library contract.** Sink-side helper libraries (`arvus_probe.sh`, `sonos_probe.sh`, etc.) MUST expose a `*_require_reachable` function that returns exit code 2 (OPERATOR-BLOCKED) when the sink cannot be probed. Tests asserting sink-side codec-state MUST call this function at probe entry. Silent skip via `*_probe_present` is permitted ONLY for topology-dispatch

fan-out logic that already has an alternative on-SoC + downstream-amplitude probe path.
2. **Exit-code propagation.** `arvus_require_reachable` / equivalent helpers return 2; the test harness MUST propagate 2 to the suite summary (counted separately from FAIL/SKIP) as `OPERATOR-BLOCKED`. `test_all_fixes.sh` orchestrator MUST exit non-zero when any test reported `OPERATOR-BLOCKED` AND release tags MUST NOT be cut while any `OPERATOR-BLOCKED` test exists.
3. **Test rewrite.** Every test currently asserting `audio output routing PASS` via metadata-only checks (e.g., the historical `test_audio_output_routing.sh` config-only invariants) MUST be rewritten to capture at least one positive sink/downstream evidence per the enumeration above. The §11.4 PASS-bluff pattern `tinymix says route is on → PASS` is the failure mode this anchor closes.
4. **Anti-stickiness post-stop.** After the test stops the playback, it MUST re-probe the sink/downstream and assert the codec-state transitioned to N.E. / N/A / 0-frames-delta. A persistent codec- state across test boundaries indicates orphan playback (§11.4.14 violation) AND would falsely PASS the next test.

Pre-build gate `CM-COVENANT-114-68-PROPAGATION` enforces this anchor literal across canonical files + per-consumer propagation. Paired mutation strips the anchor literal → gate FAILs. Pre-build gate `CM-POSITIVE-SINK-EVIDENCE-PER-AUDIO-TEST` walks every audio-routing on-device test asserting PASS and verifies it cites at least one positive evidence call (one of `arvus_require_reachable` / `arvus_assert_codec_format` / `pcm_hw_ptr_delta_positive` / `tinycap_rms_above_floor` / `ffprobe_frames_nonzero`). Paired mutation strips one such call → gate FAILs.

Composes with §11.4.2 (recorded-evidence — sink-side is the captured-evidence channel), §11.4.5 (audio + video quality analysis comprehensiveness — codec / channel-count / RMS), §11.4.13 (sink-side captured-evidence mandate — §11.4.68 closes its silent-skip gap), §11.4.14 (test cleanup — anti-stickiness post-stop), §11.4.46 (recent-work validation — sink-side evidence required at validation entry), §11.4.49 (dual-approach testing — both UI + Intent variants need sink-side evidence), §11.4.50 (deterministic consistency — sink-side evidence must be consistent across N iterations), §11.4.52 (autonomous-validation — sink probe IS the canonical autonomous path for HDMI audio).

No escape hatch — no `--skip-sink-evidence`, `--allow-empty-codec`, `--sink-unreachable-is-pass`, `--metadata-only-suffices` flag exists anywhere. The discipline exists because the operator/end-user forensic on 2026-05-20 demonstrated the exact failure this anchor forbids: tests reported audio-routing PASS while the user heard nothing and Arvus Codec-In-Use was empty.

Canonical authority: constitution submodule `Constitution.md` §11.4.68.

Non-compliance is a release blocker regardless of context.

---

§11.4.69 — Universal Sink-Side Positive-Evidence Taxonomy + Mechanical Enforcement (User mandate, 2026-05-20)

Forensic anchor — direct user mandate (verbatim, 2026-05-20):

> "THIS MUST HAPPEN NEVER AGAIN!!! We MUST HAVE this all working! Not just for audio but for every single piece of the System!!! Proper full automation when executed with success MUST MEAN that manual testing will be as much positive at least regarding the success results! Dive deep into the root causes and how this had happened! The root causes of such omission MUST BE TRACKED, and proper solution applied! Solution MUST BE universal, generic that solves working flows for all System components and for all future and all existing projects! Make sure everything is added we change as mandatory rules we must follow and critical constraints into ours root (constitution Submodule) Constitution.md, CLAUDE.md and AGENTS.md!!! Everything we do MUST BE validated and verified with rock-solid proofs and anti-bluff policy enforcement and fulfillment! THIS IS MOST CRITICAL POINT WE HAVE NOW with all audio issues!"

Escalation from the same operator session (2026-05-20, hours earlier):

> "We still do not hear any audio played from D3 device! Arvus Web Dashboard when we play music from D3 shows nothing for Codec In Use! … How come we passed the tests with Arvus validation? What were values for the Codec In Use field? Empty means nothing! This is not working!"

Forensic incident — 2026-05-19→20 D3 audio-routing PASS-bluff generalised. §11.4.68 closed the audio-specific failure path where Arvus reported an empty Codec-In-Use field and the test PASSed anyway. The same shape of bug had previously shown up intermittently across multiple subsystems (display routing on D1 secondary HDMI, BT A2DP codec advertisements, WiFi connection quality, video playback frame-count, touch-input event delivery). The audio incident was the canonical trigger because the operator hears silence at the soundbar even while the cycle reports green. §11.4.69 is the universal generalisation — closing the bluff class for every user-visible feature, not only audio.

Root-cause analysis — why existing anchors §11.4.2 / §11.4.5 / §11.4.13 / §11.4.27 / §11.4.52 / §11.4.68 did NOT mechanically catch the failure across the System:

1. **Aspirational text without a pre-build gate.** §11.4.2 (recorded-evidence) required captured visual/audio evidence per PASS but had no gate walking the test corpus to enforce a per-PASS evidence file. Tests called `ab_pass "description"` and the evidence requirement was honour-system.
2. **SKIP-fail-open hiding missing evidence.** §11.4.13 (Arvus sink-side) had a silent-skip escape when the sink returned an empty / unreachable response — the empty response was the very defect signal but it was classified as "sink unreachable, SKIP the assertion." §11.4.68 closed this for audio; §11.4.69 closes it universally across every feature class with a downstream sink.
3. **PASS-by-default vs FAIL-by-default helper contract.** The bare `ab_pass` helper accepted a free-text description and incremented the PASS counter — no evidence path required, no shape enforced. The default failure mode was therefore PASS- bluff. The correct default is FAIL-by-default — a PASS must prove itself with a captured-evidence artefact path.
4. **Missing closed-set taxonomy.** Individual test authors made one-off decisions about what evidence shape was acceptable. Without a single canonical table mapping every user-visible feature class to its required sink-side probe, aggregate enforcement across the System was mechanically impossible.

5. **Helper functions that mapped a multi-step probe to one bool.** `arvus_probe_present` collapsed reachable-but-empty into the same value as unreachable. The §11.4.68 fix added `arvus_require_reachable` for audio; §11.4.69 codifies the equivalent for every sink (Sonos, Yamaha, Denon, video display analyzer, WiFi sink, BT peer, etc.).
6. **No per-feature evidence ledger.** §11.4.52 supplied an autonomous-validation classification (`AUTONOMOUS_VERIFIED` etc.) but did not require each test to cite a captured-evidence artefact path matching the §11.4.69 taxonomy.

The §11.4.69 mandate — universal, generic, applies to every System component, every owned project, every consumer subscribing via the constitution submodule:

Element 1: Closed-set sink-side / downstream evidence taxonomy. Every user-visible feature class in every consuming project MUST map to exactly one entry in the following table. The taxonomy is the canonical authority on the required evidence shape per feature class. Tests annotate themselves with `# §11.4.69 FEATURE: <class>` so the pre-build gate can match expected evidence shape.

Feature class	Required probe	Required evidence shape
`audio_output`	sink-side codec/channel REST probe (Arvus / Sonos / Yamaha / Denon / Marantz / WiSA) + on-device `tinycap` capture + ffprobe channel assertion	non-empty `codec_in_use` matching expected codec AND captured WAV with RMS > -60 dBFS AND ffprobe `channels` matches claim
`audio_input`	tinycap capture from intended device + ffprobe RMS	captured WAV with RMS > -60 dBFS AND `audio_devices_for_attr` shows the claimed source device
`video_display`	`screenrecord` on intended display + `ffprobe -count_frames` + analyzer event-match	non-zero mp4 size AND `nb_read_frames > 0` AND analyzer reports a matched event for the claimed display
`network_throughput`	`iperf3` / `curl --speed` / `dd` over network	throughput ≥ per-link-type floor AND TCP buffer / congestion-control matches claim
`network_connectivity`	ping + DNS resolve + TCP connect probe	captured ping reply AND DNS answer AND TCP handshake to known port
`bluetooth_a2dp`	codec query + sink peer state + A2DP TX queue depth	`dumpsys bluetooth_manager` reports `CONNECTED+A2DP_PLAYING` AND codec field non-empty AND queue depth < overflow threshold
`bluetooth_pair`	`dumpsys bluetooth_manager` + `BluetoothDevice.getBondState()`	

Feature class	Required probe	Required evidence shape
		bond state = BOND_BONDED AND device MAC matches expected
touch_input	getevent -lt capture during scripted input tap	≥1 EV_ABS ABS_MT_* event per scripted tap AND dumpsys input shows the touch device active
sensor	dumpsys sensorservice + getevent on sensor device	event count > 0 between BEFORE and AFTER snapshots for the claimed sensor
gpu_render	SurfaceFlinger frame-rate dump + dumpsys gfxinfo <pkg>	Janky frames < threshold AND Total frames rendered > 0 during test window
storage_read	dd if=<path> of=/dev/null + /proc/diskstats delta	throughput ≥ floor AND read_sectors delta confirms IO landed on intended device
storage_write	dd of=<path> + sync + /proc/diskstats delta	throughput ≥ floor AND write_sectors delta confirms IO landed on intended device
mediacodec_decode	media.metrics dump + dumpsys media.codec	decoder lifecycle events present (configure, start, ≥1 output buffer) AND codec name matches claim
mediacodec_encode	media.metrics dump + non-zero output file	encoder lifecycle events present AND output file non-zero AND ffprobe confirms codec matches claim
miracast / cast	sink-side WFD / Cast announce + session state	sink dashboard reports active session AND session ID present in dumpsys media_session
boot_service	getprop init.svc.<name> + log capture	service state = running AND log shows expected boot-time output literal
package_install	pm path <pkg> + signature match	non-empty pm path AND APK signature matches expected platform key
permission_grant	dumpsys package <pkg> permission state	permission state = granted: true for the claimed permission
wifi_link	iw dev wlan0 link + RSSI + frequency	captured BSSID AND RSSI within range AND frequency matches claim

Feature class	Required probe	Required evidence shape
`wifi_throughput`	`iperf3` to LAN server	throughput ≥ floor for the negotiated PHY rate
`ethernet_link`	`ip link show` + carrier state + throughput probe	`carrier=1` AND `speed` matches claim AND throughput probe succeeds
`display_topology`	DRM connector dump + display ID enumeration	connector status `connected` for claimed connector AND display ID present in `dumpsys SurfaceFlinger`
`drm_playback`	Widevine session + decoded-frame count	active DRM session AND decoded-frame count > 0
`subtitle_render`	analyzer + Tesseract OCR on captured frames	OCR matches expected subtitle text on the claimed display

The taxonomy is **open to additions** via constitution-submodule commits + propagation. Removal of a feature class is forbidden without a tracked work item explaining the supersession. Consumer projects MAY extend the taxonomy with project-specific feature classes (e.g., HelixCode-specific build-pipeline classes) but MUST NOT contract it.

Element 2: New helper contracts (additive during grace period; mandatory after grace-period end-date 2026-06-19).

The project anti-bluff helper library (`anti_bluff.sh` in ATMOSphere; functionally equivalent helper in every consuming project) MUST provide three new helpers:

1. **`ab_pass_with_evidence <description> <evidence_path>`** — the new canonical PASS helper:
   - REQUIRES a non-empty `evidence_path` argument
   - Verifies the path exists AND is non-empty (`[ -s "$path" ]`)
   - If the path is missing or empty, the helper EXITs the test as FAILed with a diagnostic citing the missing evidence path
   - On success, increments the PASS counter AND emits a result line `PASS: <description> [evidence: <path>]`
   - Composes with `ab_send_action` — a single test issues many `ab_send_action` calls and one terminal `ab_pass_with_evidence`
2. **`ab_skip_with_reason <description> <reason_code>`** — the new canonical SKIP helper:
   - REQUIRES `reason_code` from a CLOSED set: `geo_restricted`, `operator_attended`, `hardware_not_present`, `topology_unsupported`, `network_unreachable_external`, `feature_disabled_by_config`
   - REJECTs ad-hoc reasons — any string not in the closed set fails the helper at call time
   - FORBIDs `network_unreachable_external` for any feature class in the §11.4.69 taxonomy that lists a sink-side probe — that escape was the §11.4.13 fail-open path §11.4.68 closed for audio, and §11.4.69 closes universally
   - SKIP remains mechanically distinct from PASS in cycle math (per §11.4.1)

3. **Bare `ab_pass` deprecation wrapper.** The legacy `ab_pass` helper MUST emit a WARNING line on every call AND emit a deprecation event to a tracked log file (`/data/local/tmp/ab_pass_deprecations.log`). After the grace period (defined in Element 4) the wrapper MUST FAIL the test outright, forcing all callers to migrate.

Element 3: Mechanical enforcement via three pre-build gates + three paired meta-test mutations (per §1.1).

1. **`CM-SINK-EVIDENCE-PER-FEATURE`** — walks every `.sh` test in `device/rockchip/rk3588/tests/` (and equivalent project paths), parses for `# §11.4.69 FEATURE: <class>` annotation comments, verifies the test invokes the corresponding sink-side probe from the taxonomy AND uses `ab_pass_with_evidence` (or `ab_skip_with_reason` with a permitted reason). Pre-grace: tests lacking the annotation WARN. Post-grace: they FAIL.
2. **`CM-NO-FAIL-OPEN-SKIP`** — audits sink-side probe helpers (`arvus_probe.sh`, `sonos_probe.sh`, equivalents) for any code path that converts an empty / unreachable sink response into a PASS-counting SKIP for a feature class with a sink-side probe in the taxonomy. The gate FAILs if any such path exists. Remediation: escalate to FAIL (the empty response IS the defect) or to OPERATOR-BLOCKED per §11.4.68.
3. **`CM-AB-PASS-WITH-EVIDENCE-EVERYWHERE`** — pre-grace WARNs on every bare `ab_pass` call in in-scope tests; post-grace (2026-06-19) FAILs the gate. Tests that legitimately have no evidence-capable PASS path MUST migrate to `ab_skip_with_reason operator_attended` and add a tracked work item per §11.4.52's `OPERATOR_ATTENDED_ONLY` classification.

Paired mutations (per §1.1):

- **Mutation 1:** strip the literal `ab_pass_with_evidence` from the project anti-bluff library → assert `CM-AB-PASS-WITH-EVIDENCE-EVERYWHERE` FAILs.
- **Mutation 2:** inject a fresh test that calls bare `ab_pass` without the `# §11.4.69 FEATURE:` annotation → assert `CM-SINK-EVIDENCE-PER-FEATURE` FAILs post-grace, WARNs pre-grace.
- **Mutation 3:** inject a fresh test that calls `ab_skip` (no `_with_reason` suffix) OR `ab_skip_with_reason network_unreachable_external` for an `audio_output` feature → assert `CM-NO-FAIL-OPEN-SKIP` FAILs.

Element 4: Grace period mechanics — additive, not destructive.

- **Grace period:** 30 days from anchor land date — 2026-05-20 through 2026-06-19 inclusive. The end date is hardcoded in the helper library and in this anchor; no environment variable, no command-line flag, no operator override can extend or shorten it.
- **Pre-grace behaviour:** new helpers exist and are usable; the three gates emit WARN for legacy patterns; cycle math counts WARNs separately from PASS / FAIL / SKIP.
- **Post-grace behaviour:** WARN promotion to FAIL is automatic via a date check (`[ "$(date -u +%Y%m%d)" -ge 20260619 ]`) inside each gate's enforcement logic.

Element 5: Inheritance via HelixConstitution. This anchor lives in the constitution submodule and applies to every consuming project that subscribes — including HelixCode, Catalogizer, Yole, HelixPlay, HelixTranslate, HelixFlow, MeTube, ATMOSphere-Android-15, and every future consumer. Each consumer

MUST implement its own equivalent helper library + pre-build gates; the taxonomy in Element 1 is the canonical authority across all consumers. Consumer-specific extensions are allowed only as additions to the taxonomy.

Composes with:

- §11.4.1 (FAIL-bluffs equally forbidden — `ab_skip_with_reason` closed-set prevents script-bug FAILs hiding real defects)
- §11.4.2 (recorded-evidence requirement — Element 2 helper contracts make captured evidence mechanically required per PASS)
- §11.4.5 (audio + video quality 5-layer — §11.4.69 enforces that the layers are exercised before PASS via the taxonomy probe)
- §11.4.6 (no-guessing — sink-side evidence eliminates "we think audio is routing" guessing; the codec field reads concretely)
- §11.4.13 (Arvus sink-side evidence — §11.4.69 mechanically closes the SKIP-fail-open escape §11.4.13 left open)
- §11.4.27 (no-fakes-beyond-unit — §11.4.69 adds the runtime hook §11.4.27 lacked)
- §11.4.50 (deterministic consistency — sink-side evidence is what `ab_run_n_times` hashes for divergence detection)
- §11.4.52 (autonomous-validation — §11.4.69 supplies the evidence-shape contract per feature class; §11.4.52 supplies the classification ledger)
- §11.4.68 (positive sink-side / downstream evidence for audio + video — §11.4.69 is the universal generalisation across all feature classes)

No escape hatch:

- No `--skip-evidence` flag.
- No `--config-only-pass` flag (configuration-only PASS is the exact bluff pattern this anchor closes).
- No `--allow-fail-open-skip` flag.
- No `--legacy-ab-pass-permitted` flag post-grace.
- No taxonomy-bypass shortcut — every feature class in scope MUST have its evidence shape enforced.

Propagation gate. `CM-COVENANT-114-69-PROPAGATION` verifies the §11.4.69 anchor literal is present across the ~44-file consumer fleet (parent CLAUDE.md / AGENTS.md + Containers + 10 owned atmosphere submodules including the smarttube-player nested set + 7 HelixQA submodules). Constitution submodule files (constitution/{Constitution,CLAUDE,AGENTS}.md) are canonical authority — they carry §11.4.69 by definition.

Canonical authority: constitution submodule `Constitution.md` §11.4.69.

Non-compliance is a release blocker regardless of context.

§11.4.70 — Subagent-Driven Execution Is The Default (User mandate, 2026-05-20)

Forensic anchor — direct user mandate (verbatim, 2026-05-20):

> "Always do if possible Subagent-driven! Add this into our root (constitution Submodule) Constitution.md, CLAUDE.md and AGENTS.md. This should be the default choice ALWAYS!"

When executing implementation plans authored via `superpowers:writing-plans` (or any equivalent task-decomposed execution flow), the **default execution model is subagent-driven** per `superpowers:subagent-driven-development`. Inline execution via `superpowers:executing-plans` is permitted ONLY when (a) the task is trivial AND fits in a single sub-300-line edit, OR (b) the operator explicitly requests inline execution at brainstorm-handoff time per the `superpowers:writing-plans` skill's explicit-execution-mode prompt.

Why subagent-driven is the default:

1. **Isolated context per task** — each subagent gets a fresh context window scoped to its task, avoiding the conductor's context bloat that degrades reasoning quality after ~100 k tokens.
2. **Two-stage review naturally enforced** — fresh subagent does the work, conductor reviews, optional second subagent verifies. This composes with §11.4.4 four-layer coverage + §11.4.43 TDD-fix- discipline.
3. **Parallel-PWU compatible** (§11.4.58) — subagents are the unit of parallelism in the PWU pipeline; making them the default aligns the writing-plans handoff with the parallel-development methodology.
4. **Anti-bluff seam** (§11.4) — the conductor's review of a subagent's output is structurally separated from the work itself, eliminating self-review blind spots.
5. **Survives operator absence** — subagents resume from their on-disk plan + spec inputs; the conductor session can be lost without losing in-flight task state.

Composition with §11.4.4 (four-layer coverage), §11.4.6 (no- guessing — subagent's captured output IS the evidence), §11.4.42 (iteration discipline), §11.4.43 (TDD-fix), §11.4.50 (deterministic consistency — the subagent's deterministic exit code), §11.4.51 (LIVE_ADB_FIRST — subagents classify rebuild-requirement before commit), §11.4.58 (parallel-development PWU — subagents ARE the parallel work units).

No escape hatch. `--inline-execution-required`, `--no-subagents`, `--monolithic-execution` are NOT permitted flags. Operator may request inline at brainstorm-handoff time per `superpowers:writing-plans` skill; absent that, subagent-driven is default. Skipping subagent-driven for non-trivial work without recorded operator authorisation is itself a §11.4 PASS-bluff (the conductor self-reviews, the anti-bluff seam disappears, and the forensic class of incident this Constitution catalogues becomes mechanically possible again).

Captured-evidence enforcement. Pre-build gate `CM-COVENANT-114-70-SUBAGENT-DEFAULT-PROPAGATION` enforces this anchor literal in every CLAUDE.md / AGENTS.md across parent + 10 owned submodules + nested submodules + HelixQA dependencies (the ~44-file consumer fleet). Paired §1.1 meta-test mutation strips the anchor literal → gate FAILs.

Canonical authority: constitution submodule `Constitution.md` §11.4.70. Default applies to every `superpowers:writing-plans` skill terminal handoff and every multi-task execution flow with independent subtasks.

Non-compliance is a release blocker regardless of context.

§11.4.71 — Pre-Push Fetch + Investigate + Integrate Mandate (User mandate, 2026-05-20)

Forensic anchor — direct user mandate (verbatim, 2026-05-20):

> "before pushing changes to any upstream for any repository - main repo or Submodule, we MUST fetch and pull all changes. Once these are obtained WE MUST investigate what is different compared to head position we were on last time before fetching and pulling new changes! We MUST understand what is done and for what purpose, easpecially how that does affect our project and our System in general! Any mandatory changes or improvements required by fresh changes we just have brough in MUST BE incorporated, covered with all supported types of the tests which will produce as a result of its success execution REAL PROOFS of working for all componetns and functionalities covered and work fully in anti-bluff manner!"

This anchor is the **everyday-push variant** of §11.4.41 (Pre-Force-Push Merge-First). §11.4.41 governs the destructive force-push case; this anchor governs EVERY push including ordinary fast-forwards. The complementary discipline closes the regression-via-stale-baseline gap: a project committing against a HEAD that's behind any upstream's main risks pushing changes that conflict with, regress, or fail to incorporate accepted work landed in parallel by other consumers (HelixCode, Catalogizer, Yole, HelixPlay, HelixTranslate, HelixFlow, MeTube on the constitution submodule; sibling projects on other shared submodules).

The mandatory 5-step pre-push cycle (every push, every repository — main + every submodule):

1. **Fetch all remotes** — `git fetch --all --prune --tags` (or per-remote loop if `--all` is unavailable). Capture stdout for the audit trail.
2. **Pull all upstream branches** — `git pull --no-rebase <remote> <branch>` for each configured remote whose tip differs from the local branch. Resolve any merge conflict per §11.4.41 step 2 (rebase / merge / cherry-pick per consumer judgment, NOT auto-`--ours` or `--theirs`).
3. **Investigate the diff vs OUR previous HEAD** — `git log --oneline <previous-head>..HEAD` + `git diff --stat <previous-head>..HEAD` + read EVERY commit's body. For each foreign commit understand: (a) what changed, (b) why (forensic anchor + cited user mandate or §-anchor), (c) how it affects OUR project + OUR System (does it touch surfaces we ship? does it introduce constraints we now MUST honor? does it deprecate or supersede patterns we use?).
4. **Integrate mandatory changes + cover with full anti-bluff test coverage** — if the pulled-in work mandates anything (new §-anchor, new gate, new propagation requirement), implement it per §11.4.4(b) four-layer coverage (pre-build gate + post-build inspection + on-device test + HelixQA Challenge) + §11.4.43 TDD-fix RED-test-first discipline. Every PASS MUST carry §11.4.5 captured-evidence (REAL PROOFS — not metadata-only).
5. **Then push** — only after Steps 1–4 land. Push to every configured remote in the per-repo cascade order (canonical first, then mirrors). Verify the push landed with `git ls-remote <remote> <branch>` post-push.

Composition with §11.4.26 (constitution-submodule update pipeline — per-submodule specialisation) + §11.4.32 (post-pull validation) + §11.4.37 (fetch-before-edit) + §11.4.40 (full-suite retest before tag) + §11.4.41 (pre-force-push

merge-first — the force-push case of this anchor) + §11.4.42 (iteration discipline) + §11.4.43 (TDD-fix-discipline) + §11.4.4(b) (four-layer coverage) + §11.4.5 (audio + video quality analysis comprehensiveness) + §11.4.6 (no-guessing — every integration decision cites the foreign commit by SHA, not "we think it does X").

No escape hatch — there is no `--skip-fetch`, `--no-investigate`, `--fast-push`, `--trust-upstream` flag. The discipline exists because:

- Pushing-without-fetching produces silent stale-baseline regressions (consumer A's PR diverges from consumer B's PR; neither saw the other; both merge; resulting tree corrupts).
- Pulling-without-investigating destroys the foreign work's traceability (we now carry someone else's code without understanding it; future debugging crosses the §11.4.6 no-guessing line).
- Integrating-without-testing reproduces the §11.4 PASS-bluff pattern (we adopt a change but don't prove it works in OUR system).
- Pushing-after-integration-without-evidence reproduces §11.4 itself (untested integration = release-blocker by definition).

Per-repository scope: this anchor applies to (a) the parent ATMOSphere-Android-15 repo, (b) the constitution submodule (across all 6 remotes), (c) every owned submodule (presenter, vlc-player, nova-player, mpv-player, gramophone-player, rhythm-player, strep-player, smarttube-player, torrserve, lampa), (d) every nested submodule (smarttube-player/SharedModules / smarttube-player/MediaServiceCore / smarttube-player/MediaServiceCore/SharedModules), (e) every HelixQA dependency (Challenges, Containers, DocProcessor, LLMOrchestrator, LLMProvider, VisionEngine, HelixQA).

Audit-trail requirement: every push event MUST be reconstructable post-hoc from `docs/changelogs/<tag>.md` + the per-repo `git log` evidence. Operators auditing a release MUST be able to: (1) see what each repo's `HEAD..@{u}` was BEFORE this push, (2) see what foreign work was pulled in, (3) see what new tests + captured-evidence were generated by the integration. Anchor enforcement may be partially automated by extending `scripts/commit_all.sh` to refuse pushes without an audit-trail log entry.

Pre-build gate `CM-COVENANT-114-71-PROPAGATION` enforces this anchor literal across the consumer fleet (parent + 10 owned submodules + nested + HelixQA). Paired mutation strips the anchor literal → gate FAILs. Paired meta-test also asserts the audit-trail-log file produced by a recent push exists + lists the foreign-commit SHAs that were pulled in.

Canonical authority: constitution submodule `Constitution.md` §11.4.71.

Non-compliance is a release blocker regardless of context.

§11.4.72 — Audio Top-Priority Mandate (User mandate, 2026-05-20)

Forensic anchor — direct user mandate (verbatim, 2026-05-20):

> "Make sure all fixes for audio are always top priority in main working stream!"

The conductor (main working stream — whether a Claude Code session, an AI agent, or a human operator) MUST treat audio fixes as the highest-priority class on the serial dispatch queue. Audio fixes include but are not limited to §EU HDMI rejection, §EM multichannel HDMI, §ET Arvus integration, §EV/§EW/§EX D3 audio defects, HiFi (Fix #74 pcm_config_deep, #76 HDMI/SPDIF resilience), AC3 (Fix #106 FFmpeg Codec2), ES8388 (Fix #103/§104/#105 mutual-output + DAPM), multichannel LPCM (Fix #112), and every future audio-stack improvement.

Composition with §11.4.42 (iteration-discipline mandate) + §11.4.58 (parallel-development methodology). Per §11.4.42, the conductor selects work items in priority order; this anchor pins audio at the apex of that order. Per §11.4.58 PWU pipeline, parallel BACKGROUND subagents (research, refactors, infrastructure documentation) MAY run concurrently with audio work — they do NOT preempt audio on the main-stream serial dispatch queue.

Operative rule: any time the conductor faces a choice between dispatching an audio task vs a non-audio task on the SAME serial resource (most commonly the pre_build_verification.sh + meta_test_false_positive_proof.sh edit-and-commit critical path), the audio task wins.

No escape hatch — there is no "but this non-audio task is faster" or "but this research is more interesting" override. The discipline exists because audio-stack regressions are user-perceptible and high-impact (D3 silent post-flash is a release blocker), while research and refactors can wait.

Canonical authority: constitution submodule `Constitution.md` §11.4.72.

Non-compliance is a process violation regardless of context.

§11.4.73 — Main-specification document versioning + revision discipline (User mandate, 2026-05-20)

Forensic anchor — direct user mandate (verbatim, 2026-05-20):

> "Make sure everything we add now in previous and upcoming requests IS ALWAYS applied to the main specification — if we have one. Since all these are not major changes we could increase Specification version per change for secondary version instead of the primary. Primary version MUST BE increased for much bigger levels of changes! Add this into root (constitution Submodule) Constitution.md, CLAUDE.md and AGENTS.md as mandatory rule / constraint applicable ONLY IF we have something like the main specification document or we do recognize something like the main specification document. Document MUST BE updated ALWAYS to follow the versioning rules we are appling here + revision and other properties we have!"

Scope condition. This clause applies **only when a project recognises a main specification document** — i.e. a project-level Markdown file (or set of files) that constitutes the canonical "what this system does" reference, typically at `docs/specs/**/specification*.md` or a comparable path. Projects with no main specification (small libraries, internal tooling) are exempt. Projects that operate per-feature spec files (e.g. ADRs only) are exempt unless they elect to adopt the main-spec pattern.

The mandate. When a project DOES have a main specification document:

1. **Every additive operator requirement, refinement, or accepted recommendation MUST be applied to the spec** before, or as part of, the work that implements it. The spec is the source of truth; downstream code and tracking docs reference it.
2. **Spec versioning has two axes** — *primary* and *secondary* (where "secondary" maps to the existing §11.4.61 metadata-table `Revision` row):
   - **Primary (V1 / V2 / V3 / …)** bumps for major rewrites — substantial architecture changes, foundational scope shifts, deprecating large sections, replacing the technology stack. Triggered explicitly by operator decision; old primary versions move to `archive/` per §11.4.61.
   - **Secondary (`Revision`)** bumps for every other change — section additions, mandate landings, structural reorganisation, polish, type-contract refinements. The metadata table's `Revision` integer is the secondary version (matches §11.4.44 monotonic-integer rule).
3. **The metadata table on the spec MUST stay current** — `Revision` bumps in lockstep with the change; `Last modified` updates to the change date; `Status summary` describes the bump's content; `Fixed` and `Fixed summary` reference the relevant work IDs.
4. **Cross-document propagation** — when a project has propagated copies of the spec-change rule (Herald uses CLAUDE.md, AGENTS.md, HERALD_CONSTITUTION.md per its §106), those copies MUST also reference the active spec file (`specification.V<primary>.md`) and not a stale archived version.
5. **Archive semantics** — when the primary version bumps, the old `specification.V<n>.md` moves to `<spec-dir>/archive/specification.V<n>.md` and gains `Status: superseded` with a `Continuation` pointer to the new active file. Per the §11.4.65 universal-export mandate the archived `.html` and `.pdf` siblings travel with it.

Anti-bluff captured-evidence gate (planned). `CM-SPEC-VERSION-DISCIPLINE`: - If the project advertises a main spec (operator marks via `[project].main_spec_path = "docs/specs/.../specification.V<n>.md"` in a canonical config file), the gate asserts: - The file at `main_spec_path` exists. - Its metadata table's `Revision` row is ≥ the highest `Revision` referenced anywhere else in the repo (Issues.md, Fixed.md, CLAUDE.md, AGENTS.md). - If commits exist that touch the spec since the last `Revision` bump, the gate FAILs (operator forgot to bump).

Paired §1.1 mutation. Backdate the `Last modified` row to a prior date while leaving real-content edits in place → gate FAILs.

Composition. Composes with §11.4.44 (revision header — `Revision` is the secondary version here), §11.4.61 (metadata table + ToC), §11.4.59 (README always-sync — if README cites the spec, the citation MUST point at the current primary version), §11.4.65 (universal export — archived versions stay exported).

Classification: universal (per §11.4.17), applicable conditionally per the scope condition above.

Canonical authority: constitution submodule `Constitution.md` §11.4.73.

Non-compliance (forgetting to bump, or letting the spec drift behind operator-mandated requirements) is a release blocker.

§11.4.74 — Submodule-catalogue-first discovery + extend-don't-reimplement (User mandate, 2026-05-20)

Forensic anchor — direct user mandate (verbatim, 2026-05-20):

> "We MUST ALWAYS check which already developed features / functionalities do exist as a part of our comprehensive Submodules catalogue located in vasic-digital and HelixDevelopment organizations on GitHub and GitLab both! Project MUST BE aware of all its existence so we do not implement same things multiple times if they are already done as some of existing universal, reusable general development purpose Submodules! For any missing features that some Submodules we incorporate may be missing we MUST IMPLEMENT the properly and extend those Submodules furter! We do control all of the and we CAN and MUST maintain and extend the regularly! All development cycle rules we have MUST BE applied to them and fully respected!"

The mandate. Before scaffolding ANY new module, package, helper, or utility, the contributor (human or AI agent) MUST:

1. **Survey the canonical Submodule catalogue.** Two organisations are authoritative:
   - `vasic-digital` on GitHub (`https://github.com/vasic-digital`) AND GitLab (`https://gitlab.com/vasic-digital`).
   - `HelixDevelopment` on GitHub (`https://github.com/HelixDevelopment`) AND GitLab (`https://gitlab.com/HelixDevelopment`). Both organisations mirror across all four supported hosts per §2.1 — surveying one mirror MUST yield the full catalogue.
2. **Inventory existing Submodules.** The canonical repo INDEX is `submodules-catalogue.md` in this submodule (landed 2026-05-20 per user follow-up mandate). It enumerates every owned-by-us repository under `vasic-digital` + `HelixDevelopment` (142 repos at landing time) with a one-line capability description, categorised so consuming projects can locate "is there already a Submodule for X?" in one glance. The catalogue MUST be re-generated quarterly (or after any non-trivial repo create/rename) per its §3 regeneration recipe.
3. **Reuse before reimplement.** If a Submodule already provides the functionality (or 80%+ of it), the consuming project MUST add the existing Submodule as a Git submodule, not write the functionality fresh.
4. **Extend in-place when 80%+ matches but features are missing.** When an existing Submodule is close-but-not-complete, the missing features MUST be added TO THAT SUBMODULE (PR upstream + bump consuming project's submodule pointer) — never as a separate consuming-project helper that duplicates the 80% already in the Submodule.
5. **Same development-cycle rules apply.** Every Submodule under the two orgs is subject to: §11.4 anti-bluff covenant, §1.1 paired-mutation tests, §11.4.10 credentials handling, §11.4.61 metadata table + ToC, §11.4.65 universal Markdown export, §11.4.73 spec versioning (if the Submodule has its own spec), §2.1 multi-mirror push, §3 propagation order. Maintenance + extension of those Submodules MUST follow these rules without exception.

6. **Document the survey result.** Each new feature's tracker entry (Issues.md row, ADR, PR description, or equivalent) MUST contain a `Catalogue-Check:` field with one of three values:
   - `Catalogue-Check: reuse <org/repo>@<sha>` — the existing Submodule covers this.
   - `Catalogue-Check: extend <org/repo>@<sha>` — extending an existing Submodule; reference the upstream PR.
   - `Catalogue-Check: no-match <date>` — surveyed both orgs on the named date; no existing Submodule covers this functionality; the new code is justified to live in-project. Operators reviewing the PR re-validate this claim.

Anti-bluff captured-evidence gate (planned). `CM-SUBMODULE-CATALOGUE-CHECK`: - For every new module / package introduced in a non-trivial PR (heuristic: ≥ 200 LOC of new non-test code), the gate scans the PR description and any linked tracker row for a `Catalogue-Check:` line. Absence is a FAIL. - The gate's paired mutation strips the line from a PR and asserts the gate FAILs.

Composition. Composes with §3 (submodule commits propagate first), §4 (every tag on the main repo MUST be mirrored on every owned submodule), §11.4.36 (mandatory `install_upstreams` on clone/add), §11.4.31 (Submodule-Dependency-Manifest), §11.4.32 (post-pull validation), §11.4.28 (Submodules-As-Equal-Codebase).

Why this matters. Without the catalogue-check, consuming projects re-implement UUIDv7 helpers, RLS-tenant-context wrappers, Pandoc-multi-format exporters, install_upstreams shell scripts, Markdown ToC generators, fnv1a32 hash wrappers, etc. — fragmenting the catalogue and forcing the next project to choose between three subtly-different implementations of the same thing. The mandate makes the catalogue load-bearing.

Classification: universal (per §11.4.17). Applies to every project that consumes this Constitution.

Canonical authority: constitution submodule `Constitution.md` §11.4.74.

Non-compliance is a process violation; severe cases (duplicate implementation landed without catalogue check) are release blockers.

---

§11.4.75 — Mechanical Enforcement Without Exception (User mandate, 2026-05-20)

Forensic anchor — direct user mandate (verbatim, 2026-05-20):

> "Why do these violations still happen!? This is a serious problem! We cannot rely on stability nor consistency if we cannot respect our Constitution, mandatory rules and constraints! Is there a way to make this always respected, followed and applied without exception fully and unconditionally!? WE MUST HAVE THIS WORKING FLAWLESSLY!!! Do investigate the root causes of such problems! Once all problems are identified WE MUST apply proper mechanisms for this not to happen NEVER EVER AGAIN!"

§-slot history note. This anchor was originally drafted as §11.4.74 but renumbered to §11.4.75 per the §11.4.41 merge-first mandate after upstream landed a different §11.4.74 (Submodule-catalogue- first discovery) concurrent with this work. Same renumber pattern as Phase 39.DH §11.4.41 collision resolution (commit `8ad51547115` of the parent ATMOSphere-Android-15 repo).

The §11.4 covenant ("tests pass while feature broken-for-end-user") historically relied on agent and operator vigilance to enforce. Three forensic incidents in 2026-05-19→20 demonstrated the failure: subagent `a84d2c86f90fdc297` committed `docs/research/workstation/Configurations.md` without HTML+PDF siblings (§11.4.65 violation) when it stalled at the 600s watchdog before its pandoc-export step; subagent `a6b38eedf4ef796e8` dispatched to remediate ALSO stalled at the same operational seam; the parent CLAUDE.md / AGENTS.md drift after §11.4.66 propagation required a dedicated catch-up commit `f93b25a92eb`. The common failure mode: late-binding enforcement that fires at `pre_build_verification.sh` time — hours-to-days after the violator commit reached every remote.

The mandate. Constitutional invariants MUST be enforced mechanically at FIVE independent layers — bypassing any single layer does not bypass the discipline. The five layers are:

1. **Local `pre-commit` git hook** — refuses staged `.md` lacking sibling `.html`+`.pdf` (and other staged-only invariants).
2. **`commit_all.sh` integration** — the canonical project commit script invokes the same checks + auto-runs `sync_all_markdown_exports.sh` to self-repair before the commit.
3. **Local `pre-push` git hook** — re-runs siblings + propagation gate subset on every commit in the push.
4. **`post-commit` auto-repair hook** — detects orphan `.md` in the just-committed manifest, auto-generates siblings via pandoc + weasyprint, creates a `chore(§11.4.75): auto-export ...` follow-up commit. Idempotent + recursion-guarded.
5. **Local-only equivalent** (Phase 39.GF, User mandate 2026-05-20). Remote CI surfaces (GitHub Actions, GitLab pipelines, Jenkins, CircleCI, etc.) are DISABLED per User mandate 2026-05-20 — the workflow file is preserved at `.github/workflows/constitution-compliance.yml.disabled-local-only` so its contents survive for re-enable but GitHub Actions does NOT execute it (only `.yml` / `.yaml` files under `.github/workflows/` are honoured). Layer 5 enforcement migrated to the LOCAL pre-build- verification + meta-test run that the operator MUST execute before tagging: `bash device/rockchip/rk3588/tests/pre_build_verification.sh`
   - `bash scripts/testing/meta_test_false_positive_proof.sh`. Layers 1-4 remain authoritative; Layer 5 is now the operator's local final-gate ritual. A future re-enable PWU may re-establish remote CI.

Helper contracts (mandatory):

- `scripts/install_git_hooks.sh` — idempotent installer that copies hooks from tracked `scripts/git_hooks/` into `.git/hooks/`. Hooked into `scripts/setup.sh` so fresh clones auto-install. Verifies itself with `--verify` flag.
- `scripts/git_hooks/pre-commit` — Layer 1 implementation.
- `scripts/git_hooks/pre-push` — Layer 3 implementation.
- `scripts/git_hooks/post-commit` — Layer 4 implementation.
- `scripts/git_hooks/commit-msg` — Bypass-rationale enforcement.

- `_constitution_sibling_check` function in `scripts/commit_all.sh` — Layer 2 implementation.

Bypass policy. No layer is perfectly bypass-proof in isolation — that is the design. Bypassing ALL FIVE simultaneously is mechanically impossible because:

- Layers 1 + 3 can be bypassed by `--no-verify`. Layer 5 (now local pre-tag ritual after Phase 39.GF) re-runs the same check before tag creation; tags are NEVER created without it per §11.4.40.
- Layer 2 can be bypassed by direct `git commit`. Layer 1 catches that.
- Layer 4 cannot be bypassed by `--no-verify` (runs after commit lands).
- Layer 5 is now local-only (Phase 39.GF — User mandate 2026-05-20). The operator runs `pre_build_verification.sh` + meta-test before every tag per §11.4.40 full-suite-retest mandate. Skipping the local Layer 5 ritual blocks tag creation.

Legitimate bypasses MUST be audited. When `--no-verify` is detected via touch-file marker `.git/ATMO_LAST_BYPASS_ATTEMPT`, the `commit-msg` hook REQUIRES a `Bypass-rationale: <reason>` footer in the commit message. `docs/audit/bypass_events.md` accumulates the audit trail per §11.4 captured-evidence requirement.

Captured-evidence enforcement. Five pre-build gates with paired §1.1 meta-test mutations:

- `CM-COVENANT-114-75-PROPAGATION` — anchor literal across canonical files.
- `CM-GIT-HOOKS-INSTALL-SCRIPT` — installer present + executable.
- `CM-GIT-HOOKS-SOURCE-DIR` — 4 hook bodies present + executable.
- `CM-COMMIT-ALL-SIBLING-CHECK` — `_constitution_sibling_check` in `commit_all.sh`.
- `CM-CI-WORKFLOW-PRESENT` — CI workflow + ≥3 required jobs.

Composes with §1.1 (paired meta-test mutations), §9 (data safety), §11.4 (end-user-quality covenant — Layer 5 CI enforcement is the §11.4 promise made mechanical), §11.4.1 (FAIL-bluffs forbidden), §11.4.4 (test-interrupt-on-discovery), §11.4.5 (audio + video quality), §11.4.6 (no-guessing), §11.4.41 (pre-force-push merge-first — this very anchor's renumber from §11.4.74 → §11.4.75 was driven by §11.4.41 merge-first discipline), §11.4.43 (TDD-fix), §11.4.51 (live-ADB-first), §11.4.52 (autonomous-validation), §11.4.58 (PWU pipeline), §11.4.65 (universal Markdown export — Layer 1+3+4 are §11.4.65's mechanical seam), §11.4.66 (interactive-clarification), §11.4.67 (target-shell- parseability — hooks parse under bash AND mksh), §11.4.69 (universal sink-side positive-evidence), §11.4.70 (subagent-driven execution), §11.4.71 (pre-push fetch + integrate — Layer 3 + 5 honour the merge-first pipeline), §11.4.72 (audio top-priority — hooks do not race against in-flight audio commits because they hold no lock themselves), §11.4.73 (main-spec versioning — when spec exists, hooks include it), §11.4.74 (submodule-catalogue-first — hooks can be added to the canonical catalogue for cross-project reuse).

No escape hatch. No `--skip-hooks`, `--bypass-enforcement`, `--allow-orphan-md`, `--ci-not-applicable`, `--mechanical-enforcement-not-needed` flag exists. The `--no-verify` route IS the deliberate audit-trail bypass; §11.4.75 makes the audit trail mechanical via the `commit-msg` footer requirement.

The mandate exists because the User mandate of 2026-05-20 is unambiguous: violations MUST NEVER EVER AGAIN happen. Reliance on vigilance has demonstrably failed three times in 24 hours. Only mechanical enforcement at every layer can deliver the discipline the project requires.

Propagation gate `CM-COVENANT-114-75-PROPAGATION` enforces this anchor literal across the ~44-file consumer fleet. Paired mutation strips the literal → gate FAILs.

Canonical authority: constitution submodule `Constitution.md` §11.4.75.

Non-compliance is a release blocker regardless of context.

§11.4.76 — Containers-submodule mandate (User mandate, 2026-05-20)

Forensic anchor — direct user mandate (verbatim, 2026-05-20):

> "For any work or requirements of running services or codebase inside the Containers (Docker / Podman / Qemy / Emulators, and so on) we MUST USE / INCORPORATE the Containers Submodule properly: https://github.com/vasic-digital/containers (git@github.com:vasic-digital/containers.git). Containers Submodule contains all means for us to Containerize our code and services! If any feature or Containing System is missing or not supported we MUST EXTEND IT properly like we do all of our projects! All these details MUST BE added to our root constitution (constitution Submodule) - Constitution.md, AGENTS.md, CLAUDE.md, QWEN.md - and followed and respected! No bluff work is allowed of any kind!"

§-slot history note. This anchor was originally drafted as §11.4.75 but renumbered to §11.4.76 per the §11.4.71 fetch-before-push mandate after a concurrent upstream commit (`0a70083`) landed §11.4.75 (Mechanical Enforcement Without Exception). Same collision-resolution pattern as §11.4.75's own renumber-from-§11.4.74.

The mandate. For ANY containerized workload — Docker, Podman, Qemu, Kubernetes, container-backed emulators (Android emulator, VM-based testbeds, etc.) — every consuming project MUST:

1. **Use the canonical Containers Submodule.** `https://github.com/vasic-digital/containers` (`digital.vasic.containers`) is the authoritative library for runtime auto-detection, endpoint discovery, lifecycle/health management, compose orchestration, cross-build, emulator integration, and on-demand service boot. Mirrored across the four §2.1-canonical hosts (GitHub + GitLab + GitFlic + GitVerse).
2. **Install as a Git submodule.** Container-orchestration concerns enter the consuming project as a `git submodule add git@github.com:vasic-digital/containers.git <path>/containers`. Project go.mod (or equivalent dependency manifest) consumes via `replace digital.vasic.containers => ./<path>/containers` during development; production builds resolve through pinned commit SHAs.
3. **Boot infrastructure on demand.** Tests, CLI doctor commands, and local-development workflows that depend on Postgres / Redis / OTel / message-brokers / emulators MUST invoke the Containers Submodule's `pkg/boot` + `pkg/compose` + `pkg/health` APIs to bring infra up automatically. Operators

MUST NOT be required to manually start `podman machine` / `docker compose up` before tests — the boot is part of the test entry point. This is the **on-demand-infra invariant**.

4. **Extend the Containers Submodule, never reimplement.** Per §11.4.74's extend-don't-reimplement rule applied specifically to containerization: if a runtime (e.g., a new emulator type) or lifecycle primitive (e.g., a new health-check kind) is missing, the consuming project MUST add it to `vasic-digital/containers` (PR upstream + bump the consuming project's submodule pointer) — never as a parallel implementation inside the consuming project.
5. **Anti-bluff captured-evidence requirement.** Every integration test that claims to exercise a containerized component MUST actually boot that component via the Containers Submodule. Tests that pass without containers actually running (e.g., short-circuit fakes that bypass the boot path) are a §11.4 bluff violation. A passing test MUST imply the infra was up.
6. **Composition with §11.4.74.** The Containers Submodule itself is one of the Submodules surveyed by `Catalogue-Check`. A project tracker row touching containerization MUST record `Catalogue-Check: extend vasic-digital/containers@<sha>` (or `reuse`) — `no-match` is only valid if the consuming project demonstrates the Containers Submodule's API cannot model the workload (rare).

Anti-bluff captured-evidence gate (planned). `CM-CONTAINERS-USED`: - For every PR that touches `Dockerfile*`, `*compose*.yml`, `*podman*.yml`, `Vagrantfile`, or any code under a directory matching `(test|integration|e2e)/.*infra` heuristic, the gate scans for an import of `digital.vasic.containers/...`. Absence in a non-trivial container-touching PR is a FAIL. - The gate's paired mutation strips the import and asserts the gate FAILs.

Composition. Composes with §11.4.74 (catalogue-first discovery), §11.4.75 (mechanical enforcement — this clause IS one of the things mechanical enforcement protects), §3 (submodule propagation), §11.4.31 (Submodule-Dependency-Manifest), §11.4.36 (`install_upstreams` on clone), §11.4.28 (Submodules-As-Equal-Codebase: the Containers Submodule itself follows all dev-cycle rules), §11.4.65 (multi-format export for docs maintained inside the Containers Submodule), §1.1 (paired-mutation gates protect the Containers Submodule's own boot / health primitives).

Why this matters. Without the on-demand-infra invariant, integration tests silently degrade to "pass without infra running" (a §11.4 bluff) or break with cryptic "connection refused" errors that operators waste hours debugging. The Containers Submodule turns infra-boot into a typed, programmable, anti-bluff-tested capability rather than a sequence of imperative shell commands. Consuming projects converge on a single shared lifecycle/health model instead of each project hand-rolling their own.

Classification: universal (per §11.4.17). Applies to every project that runs code inside any containerization or virtualization runtime.

Canonical authority: constitution submodule `Constitution.md` §11.4.76.

Non-compliance is a process violation; landing containerized infra without consuming the Containers Submodule (i.e., reinventing compose orchestration in-project) is a release blocker.

---

§11.4.77 — Regeneration-mechanism-required mandate (User mandate, 2026-05-20)

Forensic anchor — direct user mandate (verbatim, 2026-05-20):

> "We must be sure that after excluding anything from Git versioning we still have the mechanism which will out of the box obtain or re-generate missing content! Add this mandatory safety rule / constraint into ours root (constitution Submodule) Constitution.md, CLAUDE.md, AGENTS.md, QWEN.md or any other required document / file from the constitution Submodule! Do not forget to fetch and pull Submodule first! Commit and push all Submodule changes to all upstreams!"

Forensic incident (2026-05-20T15:00Z, parent project ATMOSphere-Android-15). The parent's audio Tier 1 `commit_all.sh` stalled four hours on `git add -A` while scanning 274 GiB of untracked `.git-backup-20260520T084610Z-pre-orphan-kill/` + 159 GiB `RKTools/linux/` + 167 GiB `qa-results/` content — all of which should never have been tracked. The natural remediation is to add those paths to `.gitignore`, BUT doing so without a regeneration mechanism would silently orphan fresh clones: a new operator running `git clone && cd <project>` would find missing RKTools, missing test infrastructure, missing build outputs they cannot reproduce. §11.4.77 codifies the universal rule that closes this gap.

The mandate. Every `.gitignore` entry that excludes either (a) more than ~100 MiB of content, OR (b) any artifact essential to building / running / testing the project, MUST be accompanied by a documented + automated mechanism that re-obtains or re-generates the excluded content on a fresh clone. The closed-set choices are:

1. **Re-obtain** — download from an authoritative source (vendor tarball, SDK installer, npm / pip / cargo / go-mod registry, container image registry, dedicated git submodule, S3, GCS, vendor-internal artifact server) via a script that runs deterministically on a fresh clone.
2. **Re-generate** — produce the content from tracked source code via a script (build pipeline, code generation, asset rendering, captured-evidence replay, container build, kernel build, image packaging) that runs deterministically on a fresh clone.

Required artefacts per qualifying `.gitignore` entry (each is independently mandatory):

1. **Metadata file at `.gitignore-meta/<entry-slug>.yaml`** (or equivalent project-canonical location declared in the project's `CLAUDE.md`) carrying: the gitignore-pattern verbatim, `mechanism-type: re-obtain | re-generate`, `script-path: <repo-relative path>` (the deterministic regeneration entrypoint), `expected-disk-usage: <bytes-or-human-readable>`, `vendor-url-or-source: <URL or in-tree path>` when applicable, `integrity: { algorithm: sha256 | md5, value: <hex> }` when the content is fetchable from a known-stable mirror, `requires-network: true | false`, `requires-credentials: true | false` (and which credentials slot per §11.4.10).
2. **Entry in the post-clone bootstrap script** (`scripts/setup.sh` or the project-canonical equivalent). The script MUST run the regeneration mechanism non-interactively on a fresh clone or be invocable as an idempotent step (`scripts/setup.sh --regenerate <entry-slug>`).

3. **Pre-build gate** that verifies either the regenerated content is present at the expected path OR the regeneration script ran successfully (e.g., a stamp file under `.gitignore-meta/.regenerated/<entry-slug>.ok` with mtime ≥ source freshness window).
4. **Documentation update** — README plus the relevant `docs/guides/*.md` guide describing the mechanism, including the manual fallback (vendor URL, alternative download mirrors), the time + disk-usage budget, and per-§11.4.10 credentials requirements.

No escape hatch. Adding a bare `.gitignore` entry that excludes significant or essential content without the accompanying mechanism is itself a §11.4 PASS-bluff variant — the codebase appears complete to the casual eye, every pre-build / post-build gate goes green, but a fresh clone cannot build / run. There is no `--skip-regen-mechanism`, `--gitignore-is-enough`, `--operator-already-has-content` flag.

Composition with sister anchors:

- **§11.4.6 (no-guessing)** — the mechanism MUST be verified working end-to-end on a sandbox clone before the `.gitignore` entry lands. `LIKELY works on a fresh clone` is the exact bluff §11.4.6 forbids.
- **§11.4.65 (universal Markdown export)** — generated HTML / PDF siblings are an instance of `re-generate`; the auto-regeneration helper (`sync_all_markdown_exports.sh`) is the §11.4.77 mechanism for that gitignored content where applicable.
- **§11.4.66 (interactive clarification)** — if the agent is uncertain whether a candidate `.gitignore` addition qualifies under §11.4.77, the agent MUST ASK the operator via the platform's interactive question mechanism rather than guess.
- **§11.4.71 (pre-push fetch + integrate)** — the regeneration mechanism's integrity hash and vendor URL MUST be re-validated against upstream before pushing the §11.4.77-governing commit; stale hashes are a §11.4.6 violation.
- **§11.4.74 (catalogue-first + extend-don't-reimplement)** — if the regeneration mechanism can be implemented by extending a Submodule already in `vasic-digital/HelixDevelopment` (e.g., a reusable downloader / decompressor helper), the project MUST extend rather than reimplement.
- **§11.4.75 (Mechanical Enforcement Without Exception)** — the local `pre-commit` hook (Layer 1) MUST refuse a commit that adds new `.gitignore` lines without a corresponding `.gitignore-meta/*.yaml`; the CI workflow (Layer 5) MUST replay the gate on every push. Paired meta-test mutation: strip a metadata YAML entry → gate FAILs.
- **§11.4.76 (Containers-submodule mandate)** — container images excluded from VCS are regenerated via `vasic-digital/containers` `pkg/boot` + Dockerfiles tracked in-tree; the `.gitignore-meta/<entry-slug>.yaml` references the Containers Submodule build path.
- **§9 / §9.2 (zero-risk data safety)** — the regeneration mechanism MUST be pre-tested in a sandbox clone with a hardlinked-backup safety net BEFORE relying on it for live operator clones; a regeneration script that silently corrupts content on a fresh clone is a §9 violation.
- **§3 (propagation order)** — submodules consuming this constitution submodule inherit §11.4.77; their own `.gitignore` entries are subject to the same mechanism requirement, propagated via their own `CLAUDE.md` / `AGENTS.md` / `QWEN.md`.

Anti-bluff captured-evidence gate (planned). `CM-GITIGNORE-REGEN-MECHANISM`:

- For every PR that adds a line to `.gitignore` (or to any `*.gitignore` file in a sub-tree), the gate scans for a matching entry in `.gitignore-meta/`, verifies the `script-path` exists and is executable, verifies the metadata YAML parses and carries the required keys, and verifies the post-clone bootstrap references the mechanism.
- Bare-line additions (e.g., `echo 'qa-results/' >> .gitignore`) without the metadata sibling are a FAIL.
- The gate's paired §1.1 mutation strips one required YAML key (e.g., `script-path:`) and asserts the gate FAILs.

Why this matters. Without §11.4.77, the natural `.gitignore` ergonomics — "this is too big to track, exclude it" — leads to fresh-clone orphanage: operators or CI runners (or new developers, or end-user mirror sites) clone the project and discover essential content is missing, with no documented way to obtain it. The cost is paid at every fresh-clone event, every CI bootstrap, every operator onboarding, every disaster-recovery exercise. §11.4.77 forces the cost to be paid once, at the `.gitignore`-addition commit, where it is bounded and visible.

Forensic anchor crosswalk to today's incident. Without §11.4.77, the natural fix to today's `commit_all.sh` 4-hour-stall — adding `.git-backup-*`, `RKTools/linux/`, `qa-results/` to `.gitignore` — would have orphaned every fresh ATMOSphere clone. With §11.4.77, each of those entries now requires: (a) a `.gitignore-meta/<slug>.yaml` declaring the mechanism (e.g., `.git-backup-*` is a transient artefact safe to re-create per §9.2 hardlinked-backup; `RKTools/linux/` is re-obtainable from Rockchip's vendor tarball; `qa-results/` is re-generatable by re-running the on-device test cycle); (b) bootstrap-script entries; (c) gates; (d) docs. The bare `.gitignore` addition is now mechanically blocked.

Classification: universal (per §11.4.17). Applies to every project that excludes content from VCS for size / regenerability / vendor-provenance reasons.

Canonical authority: constitution submodule `Constitution.md` §11.4.77.

Non-compliance is a release blocker regardless of context.

---

§11.4.78 — CodeGraph code-intelligence mandate (User mandate, 2026-05-20)

Forensic anchor — direct user mandate (verbatim, 2026-05-20):

> "All gathered knowledge and steps MUST BE added as future reusable experience into our constitution Submodule on setting up / configuring and using codegraph with all supported CLI agents we may use for our codebase! Make codegraph MANDATORY CHOICE for this purpose for all of our project and add proper comprehensive details about everything codegraph related into our root constitution (constitution Submodule) Constitution.md, AGENTS.md, CLAUDE.md, QWEN.md and other constitution Submodule relevant files! All project which do not have configured and installed codegraph yet MUST DO IT and MUST USE IT!"

The mandate. Every consuming project under this Constitution MUST install, initialize, and use **CodeGraph** (`https://github.com/colbymchenry/codegraph`, npm package `@colbymchenry/codegraph`) as its code-intelligence layer for AI coding agents. CodeGraph builds a local SQLite semantic knowledge graph of a codebase and exposes it to agents over the Model Context Protocol (MCP) — 100% local, no cloud, no external API (compatible with the Local-Only CI/CD posture). A project worked on by AI coding agents that has not yet installed and wired CodeGraph MUST do so; using it is not optional.

1. **Install.** `npm install -g @colbymchenry/codegraph` (Node.js 18+). The npm global prefix MUST be user-writable — installation MUST NOT require `sudo` / `su` (§9 / §12 host-safety + the no-privilege-escalation posture). Native modules (`better-sqlite3`, `tree-sitter`) compile on install with a WASM fallback.

2. **Initialize + index.** `codegraph init` at the project root creates `.codegraph/config.json` (tracked) and `.codegraph/codegraph.db` (a regenerable build artifact — gitignored per §11.4.30 / §11.4.77, with `codegraph index` as its declared regeneration mechanism). The `config.json` `exclude` list MUST exclude (a) other-owned submodules and vendored trees — index the consuming project's own domain code — and (b), non-negotiably, every credential/secret path (`.env*`, keystores, signing keys, service-account JSON) per §11.4.10. A secret reaching the index is a §11.4.10 violation. `codegraph index` builds the graph; `codegraph sync` keeps it fresh.

3. **Wire every supported CLI agent.** The CodeGraph MCP server (`codegraph serve --mcp`, stdio transport) MUST be registered with every AI coding CLI agent the project's developers use. Registration is project-scoped and committed where the agent supports it (e.g. Claude Code `.mcp.json`, OpenCode `opencode.json`, Qwen Code `.qwen/settings.json`, Crush `.crush.json`); host-local where the agent stores MCP config outside the repository (e.g. Kimi CLI `~/.kimi/mcp.json`). Every MCP config MUST reference the bare `codegraph` command resolved on `PATH` — never a hardcoded host path (the no-hardcoding posture). Claude Code is the canonical primary agent; the others MUST work too.

4. **Anti-bluff verification is mandatory.** CodeGraph integration MUST be covered by an anti-bluff verification suite (the canonical reference implementation is Lava's `scripts/verify-codegraph.sh` + `tests/codegraph/`, six layers): index reality, query correctness, MCP-protocol JSON-RPC, per-agent connectivity, per-agent end-to-end LLM drive, and a falsifiability rehearsal that mechanically proves the suite FAILs when the index is deliberately broken. The per-agent end-to-end layer MUST use an **unforgeable challenge** — a fact obtainable only by calling a CodeGraph MCP tool (e.g. the index node count via `codegraph_status`) — so an agent answering from its own file-reading tools cannot produce a false PASS. An agent that genuinely cannot be driven end-to-end in the test environment (missing credentials, exhausted quota, environment incompatibility) is recorded as a documented SKIP gap per §11.4.3 — never a faked PASS.

5. **Comprehensive documentation.** Every project MUST carry a `docs/CODEGRAPH.md` (or canonical equivalent) describing install, initialization, indexing, per-agent wiring, the verification suite, and troubleshooting — kept in sync per §11.4.12 / §11.4.65.

6. **Catalogue-first.** CodeGraph is a third-party developer tool, not an owned submodule — it does NOT enter the project as a `git submodule` (per §11.4.74 it is consumed as the published npm package) and it does NOT add a Git remote.

Anti-bluff captured-evidence gate (planned). `CM-CODEGRAPH-WIRED`: for every consuming project, the gate verifies `.codegraph/config.json` exists with the §11.4.10 secret-exclusions present, every developer-used agent carries a CodeGraph MCP registration, and the verification suite exists and is executable. The paired §1.1 mutation removes a secret-exclusion from `config.json` and asserts the gate FAILs.

Composition. Composes with §11.4.3 (per-environment-topology SKIP for un-runnable agents), §11.4.10 (credentials never indexed), §11.4.12 + §11.4.65 (CODEGRAPH.md kept in sync + exported), §11.4.30 / §11.4.77 (the `.codegraph/codegraph.db` artifact is gitignored with `codegraph index` as its regeneration mechanism), §11.4.74 (catalogue-first — CodeGraph is consumed, not reimplemented), §11.4 (the verification suite is anti-bluff by construction — CI green is necessary, never sufficient), §1.1 (paired-mutation gate).

Why this matters. AI coding agents that explore a codebase by repeated file scanning consume tokens and tool calls wastefully and build a shallow, drift-prone mental model. A pre-indexed semantic graph gives every agent instant, consistent symbol / caller / callee / impact resolution. Standardising on one tool — CodeGraph — across every Helix project means the capability is wired once, verified anti-bluff once, and reused everywhere, instead of each project hand-rolling ad-hoc context tooling.

Classification: universal (per §11.4.17). Applies to every project that is worked on by AI coding CLI agents.

Canonical authority: constitution submodule `Constitution.md` §11.4.78.

Non-compliance is a process violation; a project worked on by AI agents without CodeGraph installed, wired, and anti-bluff-verified is in breach of this mandate.

§11.4.79 — Own-org submodules MUST be included in the CodeGraph index (User mandate, 2026-05-21)

Forensic anchor — direct user mandate (verbatim, 2026-05-21):

> "All Submodules we use in the project and that are part of organizations to which we have the full access via GitHub, GitLab and other CLIs MUST BE included into the codegraph database and initialized / scanned / synced!"

The mandate. This extends §11.4.78 step 2's exclude-list refinement with a per-submodule-ownership split. Every consuming project's `.codegraph/config.json` MUST distinguish:

Submodule class	Treatment in CodeGraph index
Own-org — full write access via the project's CLIs (canonical orgs: `vasic-digital` on GitHub/GitLab/GitFlic/	**MUST be INCLUDED.** Per-submodule sources MUST be indexed so AI coding agents resolve symbols, callers, and impact across the whole

Submodule class	Treatment in CodeGraph index
GitVerse + `HelixDevelopment` on GitHub)	repo graph — not just the consuming project's domain code.
Third-party — no write access, vendored only (the §11.4.74 `no-match → vendor` path, e.g. an external SDK like `gopkg.in/telebot.v3`)	**MUST be EXCLUDED.** Indexing third-party code wastes the local index budget and creates symbol-noise the project's contributors cannot fix.

This refines, NOT contradicts, §11.4.78 step 2 (which spoke of "other-owned submodules" generically). The classifier is **write-access via the project's own CLIs**, not "internal vs external" subjectively — own-org submodules are the project's own code under another path; third-party submodules are upstream code Herald extends without owning.

Operational steps for every consuming project:

1. **Fetch + pull latest of every submodule** before re-indexing — `git submodule update --remote --merge` from the project root, then verify the project still builds. Third-party submodule pins (e.g. `gopkg.in/telebot.v3` pinned at `v3.3.8` to keep the Go import path stable) MUST be respected and rolled back if a `--remote` advances past the load-bearing pin.

2. **Adjust `.codegraph/config.json` `exclude` to keep own-org submodules in scope.** Third-party submodule paths MUST be explicitly listed in `exclude` (per §11.4.78 step 2's per-credential exclusion principle); own-org submodule paths MUST NOT appear in `exclude`.

3. **Re-index.** Run the project's canonical CodeGraph initialiser (e.g. `scripts/codegraph_setup.sh`) so `codegraph index` (or `codegraph sync`) rebuilds the index with the corrected exclude list.

4. **Verify symbols across own-org submodules.** The project's `scripts/codegraph_validate.sh` (per §11.4.78 anti-bluff verification) MUST probe at least one symbol that lives ONLY inside an own-org submodule (e.g. `bg.TaskQueue` from `submodules/background/interfaces.go`). A successful query proves the include path actually reached the submodule's sources — not a §107 PASS-bluff where validate runs against a stale index.

5. **Paired §1.1 mutation**: temporarily add the own-org submodule path to `exclude`, re-index, run validate — it MUST FAIL on the cross-submodule probe. Restore. This mutation proves the validate is not bluffing about the include reach.

Composition. Composes with: - §11.4.74 (catalogue-first): the `own-org` classifier matches the `reuse/extend` catalogue-check disposition; `third-party` matches `no-match → vendor`. - §11.4.78 (CodeGraph mandate): this §11.4.79 refines step 2's exclude-list contract without weakening any other step. - §11.4.10 (credentials never tracked): the exclude list MUST still cover every `.env*`, keystore, signing key, service-account JSON regardless of submodule ownership. - §1.1 (paired mutation): every validate probe added per step 5 above MUST land with a mutation pair. - §107 (end-user usability): an indexed graph that lies about which symbols are reachable is a §107 PASS-bluff against the AI coding agents that consume it.

Why this matters. When AI coding agents work across a project that vendors own-org capability modules (e.g. `commons_constitution` consuming the Helix-stack 9-module surface under `submodules/`), excluding those modules from the index forces the agents back into shallow file-scan exploration of half the codebase — exactly the failure mode §11.4.78 exists to prevent. The agents lose call-graph resolution across the seam between domain code and own-org infra code. Including own-org submodules is the difference between agents seeing the project as a unified graph and agents tripping over an invisible boundary at the submodule directory.

Classification: universal (per §11.4.17). Applies to every project under this Constitution that has CodeGraph configured and own-org submodules vendored.

Canonical authority: constitution submodule `Constitution.md` §11.4.79.

Non-compliance is a process violation; an AI-agent-worked project whose own-org submodules are excluded from CodeGraph (when they could be included) is in breach of this mandate. Severe cases (own-org submodules silently excluded WITHOUT an audit trail in `.codegraph/config.json` comments) are release blockers.

§11.4.80 — CodeGraph regular-update + sync automation mandate (User mandate, 2026-05-21)

Forensic anchor — direct user mandate (verbatim, 2026-05-21):

> "We MUST regularly check for the updates and execute codegraph npm updates so the latest version of it is always installed on the host machine! After update executes with success use the following commands (and validate and verify them using codegraph help) to sync all: [list of codegraph commands]. Make sure we have proper full automation bash scripts which will run regularly and that these are part of the constitution Submodule since they MUST BE available to all projects which do respect and follow our constitution rules and mandatory constraints! Make sure all updates, sync processes we do and important codegraph related events are all documented under docs/codegraph in Status and Status_Summary documents (another area / context to regularly update and sync) and regularly export them like all other Status docs into the PDF and HTML!"

The mandate. Every consuming project under this Constitution MUST regularly check for CodeGraph npm-package updates, install the latest stable version, and re-sync its local index. The automation lives in the constitution submodule itself so every consuming project gets it via the inherited tree without reimplementation. Three deliverables:

1. **`<constitution>/scripts/codegraph_update.sh`** — checks the installed `@colbymchenry/codegraph` version against npm's latest, runs `npm update -g @colbymchenry/codegraph` (or `npm install -g @colbymchenry/codegraph@latest`) if a newer version exists, captures the old + new version + timestamp into the constitution's `docs/codegraph/Status.md` audit trail, then exits 0 only when `codegraph --version` produces the expected post-update output. Idempotent: re-runs on the same day are no-ops. Anti-bluff (§107): the "update succeeded" PASS MUST observe the new version, not just `npm update` exit code.

2. **`<constitution>/scripts/codegraph_sync.sh`** — after a successful update, runs the canonical sync sequence inside the consuming project (passed as an argument or auto-resolved via `find_constitution.sh` parent walk): `codegraph status` (baseline) → `codegraph sync .` (incremental update) → `codegraph status` (post-sync) → `scripts/codegraph_validate.sh` (the project's anti-bluff verifier, per §11.4.78 step 4). Each step's output is appended to the project's `docs/codegraph/Status.md` ledger. The sync MUST consult `codegraph help` BEFORE invoking any command to verify the subcommand still exists with the same shape (tools evolve). The canonical sync sequence covered by `codegraph help`:

```
codegraph                          # Run interactive installer
codegraph install                  # Run installer (explicit)
codegraph init [path]              # Initialize in a project (--
index to also index)
codegraph uninit [path]            # Remove CodeGraph from a
project (--force to skip prompt)
codegraph index [path]             # Full index (--force to re-
index, --quiet for less output)
codegraph sync [path]              # Incremental update
codegraph status [path]            # Show statistics
codegraph query <search>           # Search symbols (--kind, --
limit, --json)
codegraph files [path]             # Show file structure (--
format, --filter, --max-depth, --json)
codegraph context <task>           # Build context for AI (--
format, --max-nodes)
codegraph affected [files...]      # Find test files affected by
changes
codegraph serve --mcp              # Start MCP server
```

3. **`<constitution>/docs/codegraph/Status.md`** + **`docs/codegraph/Status_Summary.md`** — append-only ledgers tracking every CodeGraph-related event across the consuming project: install events, npm-update events, sync runs, index regenerations, validate runs (with PASS/FAIL counts), HRDs opened/closed against CodeGraph. Both ledgers MUST be exported to `.html` and `.pdf` siblings on every edit (per §11.4.65 multi-format export mandate). The `Status_Summary.md` is the operator-readable derived index (per §11.4.53 fixed-summary backfill convention).

Operational cadence. The automation MUST run at least weekly (per §11.4.45 status-digest cadence). Consuming projects MAY wire it more frequently (e.g. daily via cron or per-CI-run) but never less. Cadence is not contractually fixed because environments differ (development workstation vs CI vs production-locked host); the floor is "weekly" because slower than that risks accumulated drift.

Scripts are inherited by reference. Per §3 submodule inheritance, consuming projects do NOT copy `codegraph_update.sh` / `codegraph_sync.sh` into their own `scripts/` directory. They invoke the constitution submodule's scripts directly (e.g. `bash ${CONST_DIR}/scripts/codegraph_update.sh && bash ${CONST_DIR}/scripts/codegraph_sync.sh .`). This is the single-source-of-truth pattern — when the constitution updates the scripts, every consuming project picks up the change on its next submodule update.

Composition. Composes with §11.4.78 (CodeGraph parent mandate), §11.4.79 (own-org submodule inclusion), §11.4.10 (credential exclusions stay applied), §11.4.45 (status-digest cadence), §11.4.53 (Fixed_Summary backfill), §11.4.65 (multi-format export), §107 (each sync run's "ok" MUST carry observed-version evidence, not just exit code), §1.1 (paired mutation: temporarily downgrade installed codegraph version → script MUST detect drift and self-correct → restore).

Why this matters. A CodeGraph index built against an older version of the indexer may miss symbols that newer versions resolve correctly. Stale indexes silently lie to AI agents about what symbols exist and how they connect. Regular automated updates + sync close the loop: tooling stays current, the index reflects current source, AI agents see the truth. Manual "run codegraph index when I remember" is exactly the failure mode this mandate forbids.

Classification: universal (per §11.4.17). Applies to every project under this Constitution that has CodeGraph configured per §11.4.78.

Canonical authority: constitution submodule `Constitution.md` §11.4.80.

Non-compliance is a process violation; severe cases (consuming project has not run `codegraph_update.sh` in >2 weeks AND has open AI-agent work) are release blockers — the agents' index reflects a stale tool version and may produce silently wrong reasoning.

---

§12. Host-session safety — directly OR indirectly signing the user out is FORBIDDEN

Every script, test, helper, and AI agent governed by this Constitution MUST respect host-session safety. Non-compliance is a release blocker.

§12.1 Forbidden operations — directly OR indirectly

1. **Suspending the host**: `systemctl suspend`, `pm-suspend`, `loginctl suspend`, DBus `org.freedesktop.login1.Suspend`, GNOME idle-suspend, lid-close handler.
2. **Hibernating / hybrid-sleeping**: any `Hibernate` / `HybridSleep` / `SuspendThenHibernate` method.
3. **Logging out the user**: `loginctl terminate-session`, `pkill -u <user>`, `systemctl --user --kill`, anything that signals `user@<uid>.service`.
4. **Unbounded-memory operations** inside `user@<uid>.service` cgroup. Any single command expected to exceed ~4 GiB RSS MUST be wrapped in a bounded execution scope (e.g. `bounded_run` from a project helper library that wraps `systemd-run --user --scope -p MemoryMax=...`).
5. **Programmatic rfkill toggles, lid-switch handlers, or power-button handlers** — these cascade into idle-actions.
6. **Disabling systemd-logind, GDM, or session managers** "to make things faster" — even temporary stops leave the system unable to recover.

§12.2 Required safeguards

Every script in this project that performs heavy work (build, transcription, model inference, large compression, multi-GB git operations) MUST:

1. Source the project's host-safety helper library at the top.
2. Call its pre-flight check and **abort if it fails**.
3. Wrap any subprocess expected to exceed ~4 GiB RSS in a bounded execution scope so the kernel OOM killer is contained to that scope and cannot escalate to user.slice.
4. Cap parallelism (`-j`) to fit available RAM (estimate per-job-peak-RSS and divide into the budget).

§12.3 Container hygiene

Containers (Docker / Podman) the project owns or relies on MUST:

1. Declare an explicit memory limit (`mem_limit` / `--memory` / `MemoryMax`).
2. Set `OOMPolicy=stop` in their systemd unit to avoid retry loops.
3. Use exponential-backoff restart policies, never immediate retry.
4. Be clean-slate destroyed and rebuilt after any host crash or session loss so stale lock files don't keep producing failures.

§12.6 Memory-Budget Ceiling — 60% MAXIMUM

Forensic anchor — direct user mandate:

> "First make sure that whatever we do through our procedures related to this project MUST NOT use more than 60% of total system memory! All processes MUST be able to function normally!"

The mandate. Project procedures MUST NOT use more than **60% of total system RAM**. The remaining 40% is reserved for the operator's other workloads so the host can keep serving them while project work proceeds.

The protections:

1. `HOST_SAFETY_MAX_MEM_PCT` defaults to 60.
2. `HOST_SAFETY_BUDGET_GB` is computed at source-time from `MemTotal × MAX_PCT/100`, in GiB.
3. Bounded execution scopes clamp `MemoryMax` down to the budget if the caller asks for more.
4. The build script's parallelism is computed as `min(nproc, floor(budget_gb / per_job_peak_rss_gb))`.
5. Heavy work MUST be wrapped in a bounded execution scope so the kernel OOM-kills only the scope — `user@<uid>.service` stays alive.

No escape hatch. §12.6 has NO operator-facing override flag. The cap exists for the operator's own protection; bypassing it is the bluff the §11.4 covenant specifically prohibits.

§12.10 Continuation document — sacred invariant

Forensic anchor — direct user mandate:

> "during any work we perfrom, during Phases implementation, debugging and fixing, during ANY effort we have the Continuation document MUST BE maintained and it MUST NOT BE out of sync with current work we are doing! If for any reson we stop our work, we MUST BE able to continue any time, with current work, exactly where we have left of and from any CLI agent or any LLM model we chose! Nothing can be broken or faulty in maintained Continuation document!"

The mandate. A single, canonical, machine-readable handoff document — `docs/CONTINUATION.md` — must always reflect the live state of the project. Any agent (human, Claude Code, Cursor, Aider, Codex, Gemini CLI, any future LLM) must be able to resume work **exactly where the previous session left off** by reading this single file.

Mandatory protections:

1. **`docs/CONTINUATION.md` MUST exist** at the project root. Its absence is a release blocker.
2. **Every non-trivial state change** MUST update this document in the same commit as the work itself.
3. **Top-of-file timestamp** is updated on every edit. Stale timestamps trigger gate failure.
4. **Section §3 "Active work"** lists every IN PROGRESS / BLOCKED item with enough detail that any agent can resume without conversation context — concrete commands, file paths, monitor IDs.
5. **Section §0 "How to use this document"** contains the verbatim resumption prompt — a single block any operator can paste into any CLI agent.
6. **Document MUST be self-contained.** No hyperlinks to ephemeral external systems as the only source of truth.

No escape hatch. §12.10 has NO operator-facing override flag for the existence requirement. The discipline exists for the operator's own protection.

Appendix A — Mutation testing — academic and industrial foundations

The anti-bluff / paired-mutation policy in §1.1 is rooted in the **mutation testing** literature. References that the consuming project SHOULD cite when explaining its meta-test harness:

- Jia, Y. & Harman, M. (2011). *An Analysis and Survey of the Development of Mutation Testing*. IEEE Transactions on Software Engineering, 37(5).
- DeMillo, R.A., Lipton, R.J., Sayward, F.G. (1978). *Hints on Test Data Selection: Help for the Practicing Programmer*. IEEE Computer.
- Open-source mutation testers worth studying:
  - **PIT** (Java) — https://pitest.org/
  - **Stryker** (JS / C# / Scala) — https://stryker-mutator.io/
  - **Cosmic Ray** (Python) — https://github.com/sixty-north/cosmic-ray

- **mutmut** (Python) — https://mutmut.readthedocs.io/
- **mull** (LLVM-IR) — https://mull.readthedocs.io/

The §1.1 paired-mutation policy is a **lightweight** variant of mutation testing applied at the gate-assertion layer rather than at the production-code layer. It does not need a mutation framework; it needs ONE mutation per gate that proves the gate catches the break. This is computationally cheap (O(gates), not O(production code lines)) and produces a strong "bluff-immunity" guarantee.

Appendix B — Recursive inheritance & path-independence

Projects that include this constitution submodule MUST tolerate **arbitrary submodule depth**. Between the main project root and any consuming submodule there may be N intermediate levels. To locate this constitution submodule from any depth, every project SHOULD provide a helper that walks up parents until it finds `constitution/Constitution.md` (the file you are reading right now) OR follows `git rev-parse --show-superproject-working-tree` recursively. The constitution submodule ships `find_constitution.sh` for exactly this purpose; nested submodules can source it without knowing their own depth.

Appendix C — Multi-upstream push

This constitution submodule is hosted on multiple Git providers, and consuming projects often are too. The submodule ships an `install_upstreams.sh` helper (or invokes the `install_upstreams` system command from the parent toolkit) that reads `Upstreams/*.sh` declarations and configures all remotes locally.

Every commit to the constitution submodule MUST be pushed to ALL configured upstreams. Use a shell helper that iterates `git remote` and pushes one by one, OR configure `origin` with multiple push URLs (`git remote set-url --add --push origin <url>` per remote).
